I'll add all the remaining topics to the AWS book as new chapters. This will complete the comprehensive coverage from compute scaling to multi-account disaster recovery.

AWS FOR DEVELOPER & DEVOPS ENGINEER

ADDITIONAL CHAPTERS (Compute Scaling, IaC, Containers, Observability, Security, EKS, Networking, Cost Optimization, Event-Driven, Multi-Account)

Version 2.0 - Expanded Edition

About These Additional Chapters

This document contains the remaining topics from the comprehensive AWS curriculum. These chapters cover:

- **Compute Scaling:** Auto Scaling Groups, scaling policies, health checks, Multi-AZ design
- **Infrastructure as Code:** Terraform, state management, lifecycle
- **Container Ecosystem:** ECR, image management, scanning
- **Deployment Strategies:** Blue-green, rolling, canary, rollback
- **Observability:** CloudWatch Logs vs Metrics, X-Ray, centralized logging
- **Auditing & Compliance:** CloudTrail, AWS Config
- **Threat Detection:** GuardDuty, Security Hub
- **Advanced IAM:** Policy evaluation, least privilege, cross-account access
- **Encryption:** At rest vs in transit, envelope encryption, key rotation
- **Secure CI/CD:** Pipeline security, secrets handling, scanning
- **Kubernetes (EKS):** Node groups, IRSA, Ingress, networking
- **Advanced Networking:** Public/private subnets deep dive, NAT Gateway, Internet Gateway

- **Cost Optimization:** Spot instances, reserved instances, right-sizing
- **Event-Driven Architecture:** SQS, SNS, EventBridge
- **Multi-Account & Disaster Recovery:** AWS Organizations, backup, DR strategies

---

[Page 2]

TABLE OF CONTENTS (Additional Chapters)

PART ELEVEN: COMPUTE SCALING & HIGH AVAILABILITY
- Chapter 30: Auto Scaling Groups (ASG)
- Chapter 31: Scaling Policies (Target Tracking, Step Scaling)
- Chapter 32: Health Checks & Self-Healing Systems
- Chapter 33: Multi-AZ High Availability Design

PART TWELVE: INFRASTRUCTURE AS CODE (IaC)
- Chapter 34: Terraform Fundamentals
- Chapter 35: State Management & Remote Backends (S3 + DynamoDB)
- Chapter 36: Infrastructure Lifecycle Management

PART THIRTEEN: CONTAINER ECOSYSTEM
- Chapter 37: ECR (Elastic Container Registry)
- Chapter 38: Image Management & Versioning
- Chapter 39: Image Scanning Basics

PART FOURTEEN: DEPLOYMENT STRATEGIES
- Chapter 40: Blue-Green Deployment
- Chapter 41: Rolling Deployment
- Chapter 42: Canary Releases
- Chapter 43: Rollback Strategies

PART FIFTEEN: OBSERVABILITY & MONITORING (EXPANDED)
- Chapter 44: CloudWatch Logs vs Metrics

- Chapter 45: Alarms & Alerting
- Chapter 46: AWS X-Ray (Distributed Tracing)
- Chapter 47: Centralized Logging Architecture

PART SIXTEEN: AUDITING & COMPLIANCE
- Chapter 48: CloudTrail (API Auditing)
- Chapter 49: AWS Config (Resource Compliance & Drift Detection)

PART SEVENTEEN: THREAT DETECTION & SECURITY MONITORING
- Chapter 50: GuardDuty
- Chapter 51: Security Hub

PART EIGHTEEN: ADVANCED IAM & SECURITY PRACTICES
- Chapter 52: IAM Policy Evaluation Logic
- Chapter 53: Least Privilege Design
- Chapter 54: IAM Roles vs Users (Best Practices)
- Chapter 55: Cross-Account Access

PART NINETEEN: ENCRYPTION & SECRETS MANAGEMENT (EXPANDED)
- Chapter 56: Encryption at Rest vs In Transit
- Chapter 57: Envelope Encryption
- Chapter 58: Key Rotation Strategies

PART TWENTY: SECURE CI/CD PIPELINES
- Chapter 59: Pipeline Security Best Practices
- Chapter 60: Secrets Handling in Pipelines
- Chapter 61: Image & Dependency Scanning
- Chapter 62: Access Control in CI/CD

PART TWENTY-ONE: KUBERNETES (EKS) DEEP DIVE
- Chapter 63: Node Groups & Scaling
- Chapter 64: IAM Roles for Service Accounts (IRSA)
- Chapter 65: Ingress (ALB Controller)
- Chapter 66: Networking in EKS

PART TWENTY-TWO: ADVANCED NETWORKING CONCEPTS

- Chapter 67: Public vs Private Subnets (Deep Dive)
- Chapter 68: NAT Gateway
- Chapter 69: Internet Gateway
- Chapter 70: Secure Network Design Patterns

PART TWENTY-THREE: COST OPTIMIZATION STRATEGIES

- Chapter 71: Spot Instances
- Chapter 72: Reserved Instances
- Chapter 73: Resource Right-Sizing

PART TWENTY-FOUR: EVENT-DRIVEN ARCHITECTURE

- Chapter 74: SQS (Queueing)
- Chapter 75: SNS (Pub/Sub)
- Chapter 76: EventBridge

PART TWENTY-FIVE: MULTI-ACCOUNT & DISASTER RECOVERY

- Chapter 77: AWS Organizations
- Chapter 78: Multi-Account Strategy
- Chapter 79: Backup & Disaster Recovery (RTO/RPO)

[Page 3]

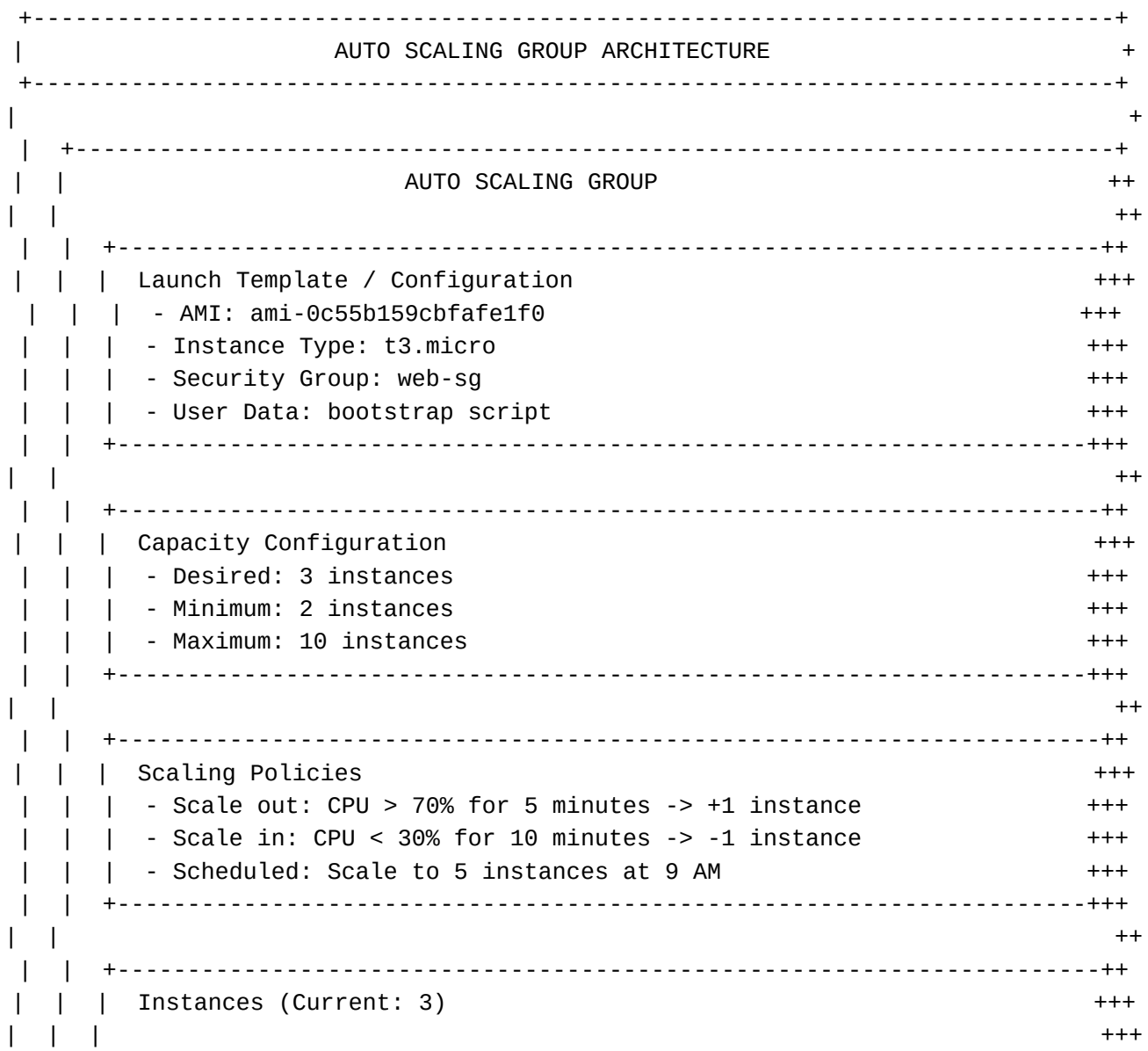
PART ELEVEN: COMPUTE SCALING & HIGH AVAILABILITY

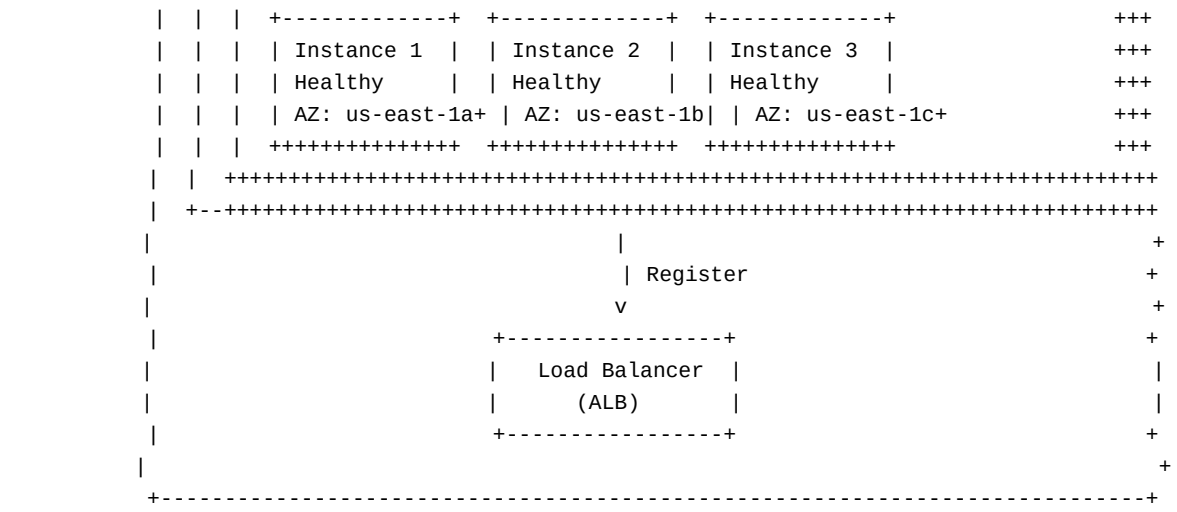
Chapter 30: Auto Scaling Groups (ASG)

What is an Auto Scaling Group (ASG)?

An Auto Scaling Group is a collection of EC2 instances that automatically adjusts capacity based on demand. It ensures you have the right number of instances running at all times.

Real Scenario
Your web application experiences traffic spikes during lunch hours and evenings. You don't want to over-provision (wasting money) or under-provision (users experience slowdowns).
Without ASG:
- Manual scaling (you add/remove instances yourself)
- Over-provision for peak (expensive)
- Under-provision during spikes (poor performance)
With ASG:
- Automatically adds instances when CPU > 70%
- Automatically removes instances when CPU < 30%
- Maintains minimum (2) and maximum (10) instances
Diagram: Auto Scaling Group Architecture





ASG Components

Component

Description

Launch Template

Configuration for new instances (AMI, instance type, security groups, user data)

Desired Capacity

Target number of instances

Min/Max Size

Boundaries for scaling

Scaling Policies

Rules that trigger scaling actions

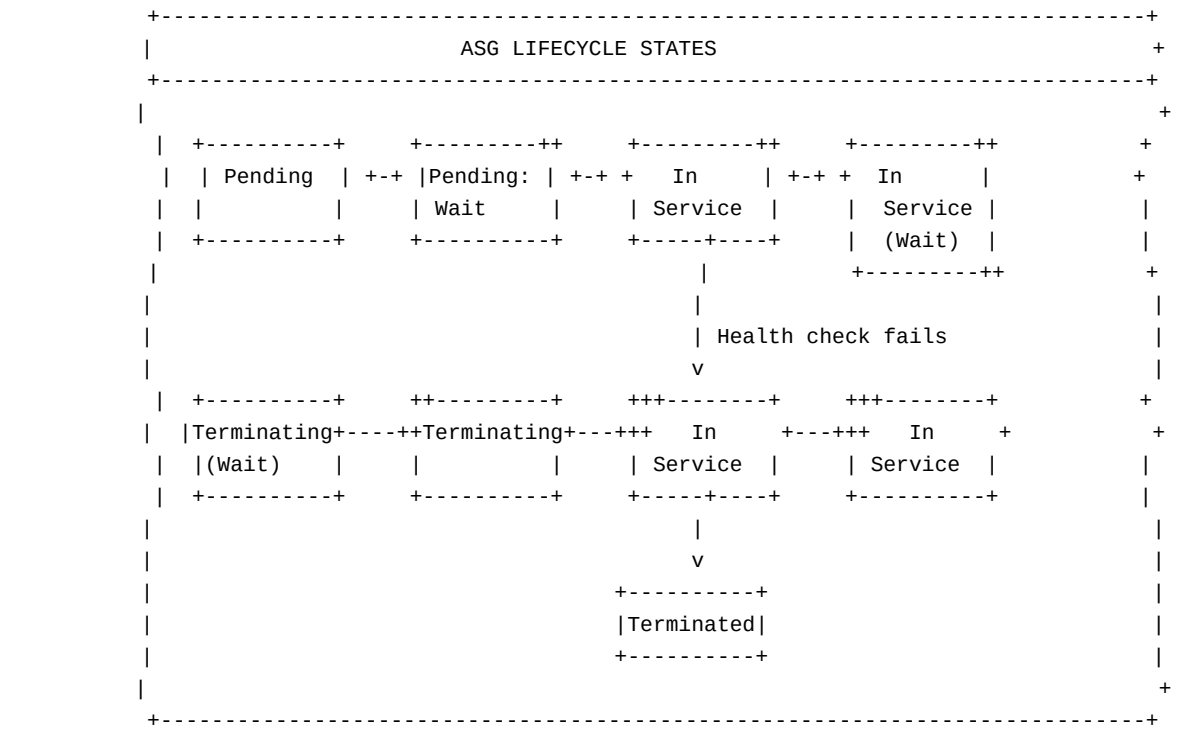
Health Checks

Determine if instances are healthy

Lifecycle Hooks

Custom actions before instance starts/terminates

ASG Lifecycle States



Launch Template vs Launch Configuration

Aspect	Launch Template	Launch Configuration
--------	----------	----------
Versioning	Yes (multiple versions)	No
Parameters	More options	Limited
T2 Unlimited	Yes	No
Spot instances	Yes	No
Recommendation	Preferred	Legacy

ASG with Load Balancer Integration

```
# CloudFormation snippet
AutoScalingGroup:
  Type: AWS::AutoScaling::AutoScalingGroup
  Properties:
    VPCZoneIdentifier:
      - subnet-12345
      - subnet-67890
    LaunchTemplate:
      LaunchTemplateId: !Ref WebServerLaunchTemplate
      Version: !GetAtt WebServerLaunchTemplate.LatestVersionNumber
    MinSize: 2
    MaxSize: 10
    DesiredCapacity: 3
    HealthCheckType: ELB
    HealthCheckGracePeriod: 300
    TargetGroupARNs:
      - !Ref WebTargetGroup

ScalingPolicy:
  Type: AWS::AutoScaling::ScalingPolicy
  Properties:
    AutoScalingGroupName: !Ref AutoScalingGroup
    PolicyType: TargetTrackingScaling
    TargetTrackingConfiguration:
      PredefinedMetricSpecification:
        PredefinedMetricType: ASGAverageCPUUtilization
      TargetValue: 70.0
```

When to Use Auto Scaling Groups

Use Case	Why ASG
----------	---------
Web applications	Handle variable traffic
Batch processing	Scale based on queue size
Microservices	Scale each service independently
Dev/Test environments	Schedule scaling to save costs

Advantages of ASG

1. **Automatic scaling:** Responds to demand changes
2. **Cost optimization:** Scale down during low demand
3. **High availability:** Maintain minimum instances across AZs
4. **Self-healing:** Replace failed instances automatically
5. **Integration:** Works with load balancers

Disadvantages of ASG

1. **Cold starts:** New instances take minutes to be ready
2. **Configuration complexity:** Many parameters to tune
3. **Scale-in risk:** Removing instances during load spikes
4. **Stateful applications:** Hard to scale (use stateless design)

[Page 4]

Chapter 31: Scaling Policies

What are Scaling Policies?

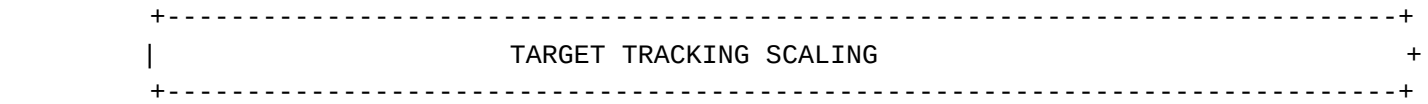
Rules that define when and how an Auto Scaling Group should change its capacity.

Types of Scaling Policies

Policy Type
Description
Use Case

Target Tracking
Maintain metric at target value
Simple, set-and-forget
Step Scaling
Add/remove based on metric thresholds
Gradual response
Simple Scaling
Single adjustment per alarm
Legacy, not recommended
Scheduled Scaling
Scale at specific times
Predictable patterns
Predictive Scaling
ML-based forecasting
Daily/weekly patterns

Diagram: Target Tracking Scaling



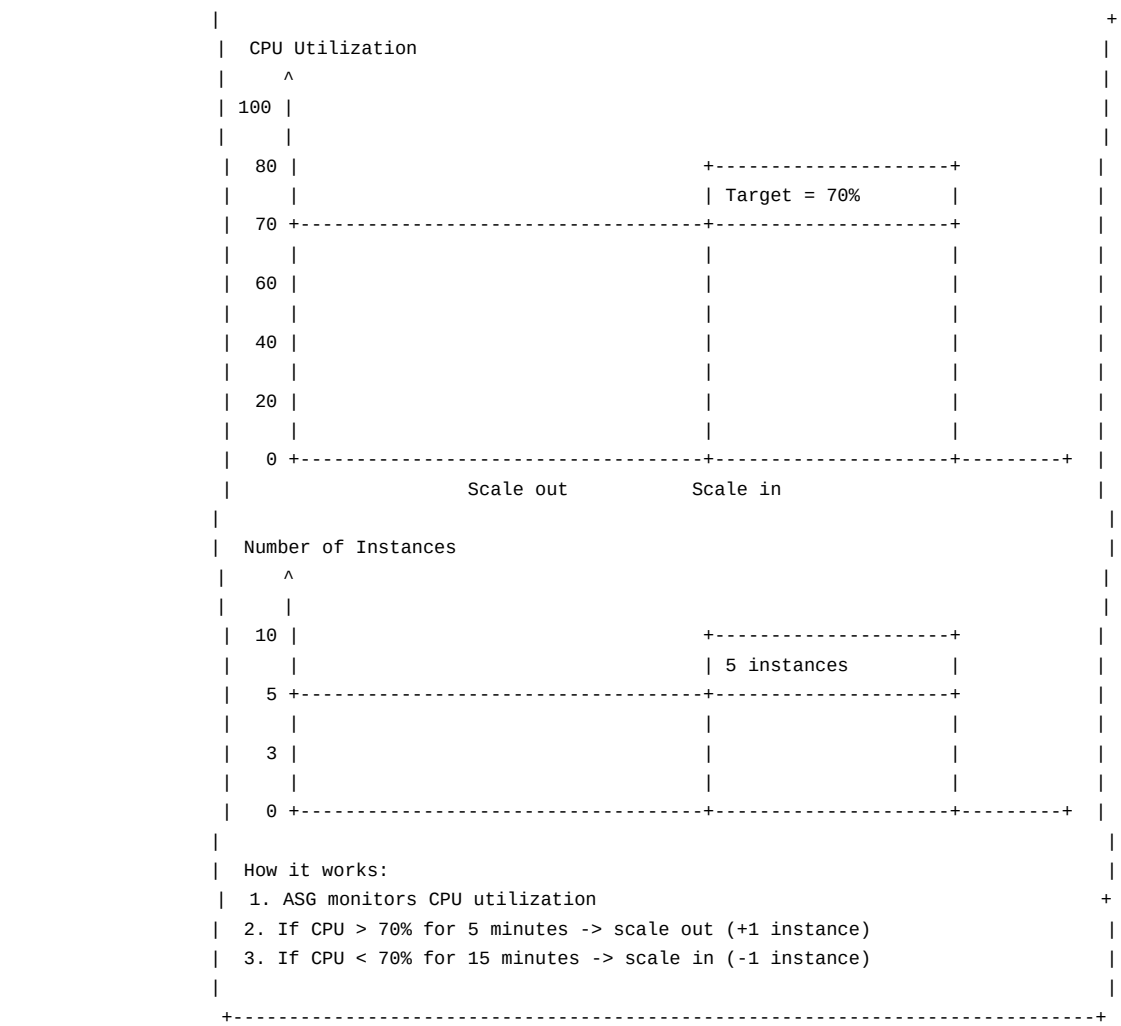
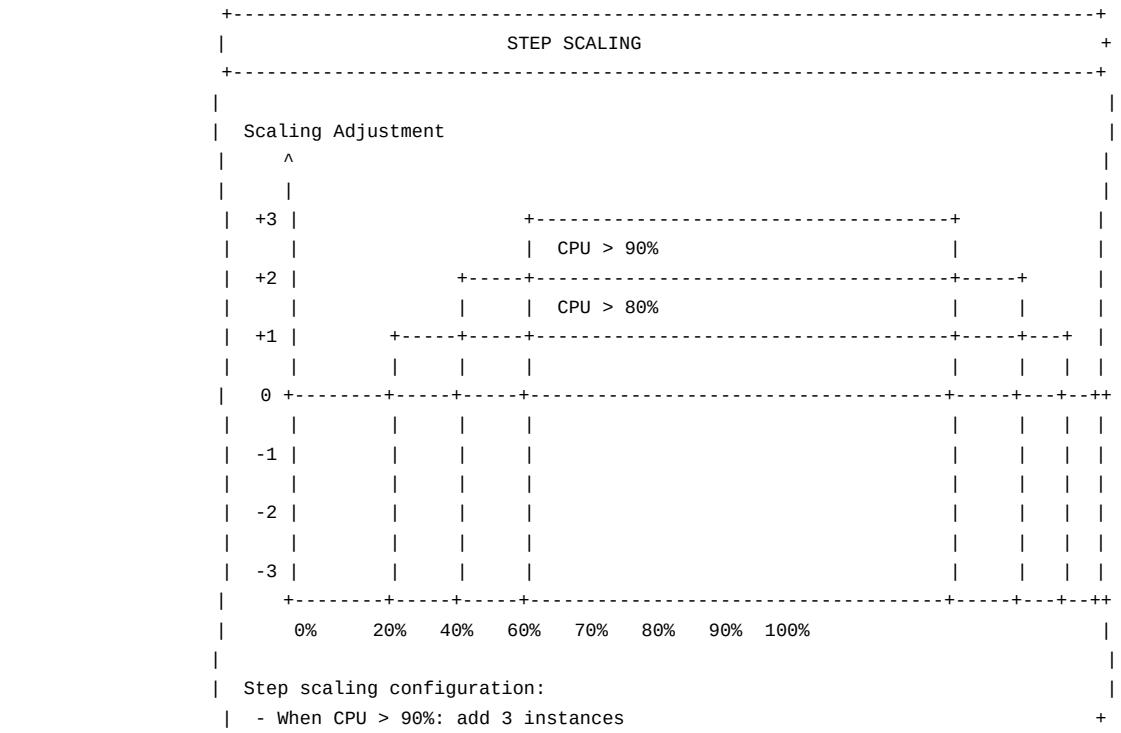


Diagram: Step Scaling



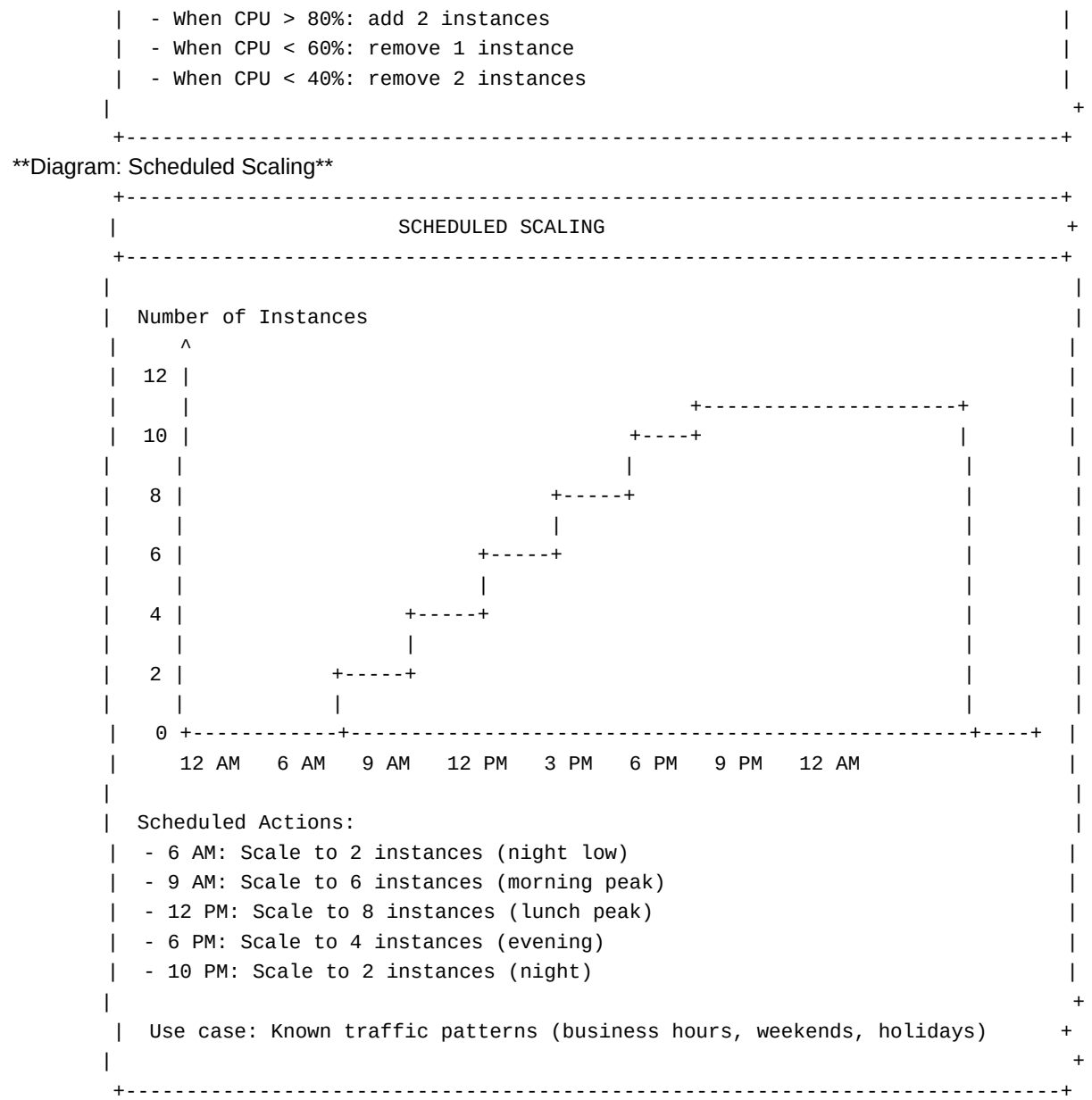


Diagram: Scheduled Scaling

Target Tracking Metrics

Metric

Description

ASGAverageCPUUtilization

Average CPU across instances

ALBRequestCountPerTarget

Requests per target

ASGAverageNetworkIn

Network receive throughput

ASGAverageNetworkOut

Network transmit throughput

Step Scaling Configuration Example

{
"PolicyName": "cpu-step-scaling",
"AutoScalingGroupName": "web-asg",
"PolicyType": "StepScaling",

```
  "StepAdjustments": [
    {
      "MetricIntervalLowerBound": 0,
      "MetricIntervalUpperBound": 10,
      "ScalingAdjustment": 1
    },
    {
      "MetricIntervalLowerBound": 10,
      "MetricIntervalUpperBound": 20,
      "ScalingAdjustment": 2
    },
    {
      "MetricIntervalLowerBound": 20,
      "ScalingAdjustment": 3
    }
  ],
  "MetricAggregationType": "Average",
  "AdjustmentType": "ChangeInCapacity"
}
```

Cooldown Periods

Concept	Description
---	---
Default cooldown	Wait time after scaling (300 seconds)
Scale-in cooldown	Separate cooldown for removing instances
Health check grace period	Wait before health checks (300 seconds)

Scaling Policies Best Practices

Practice	Why
---	---
Use target tracking	Simplest, works for most workloads
Set warm-up time	Instances need time to become ready
Use multiple metrics	Combine CPU and memory
Scheduled scaling for predictable	Prevents lag in response
Test scaling scenarios	Verify behavior under load

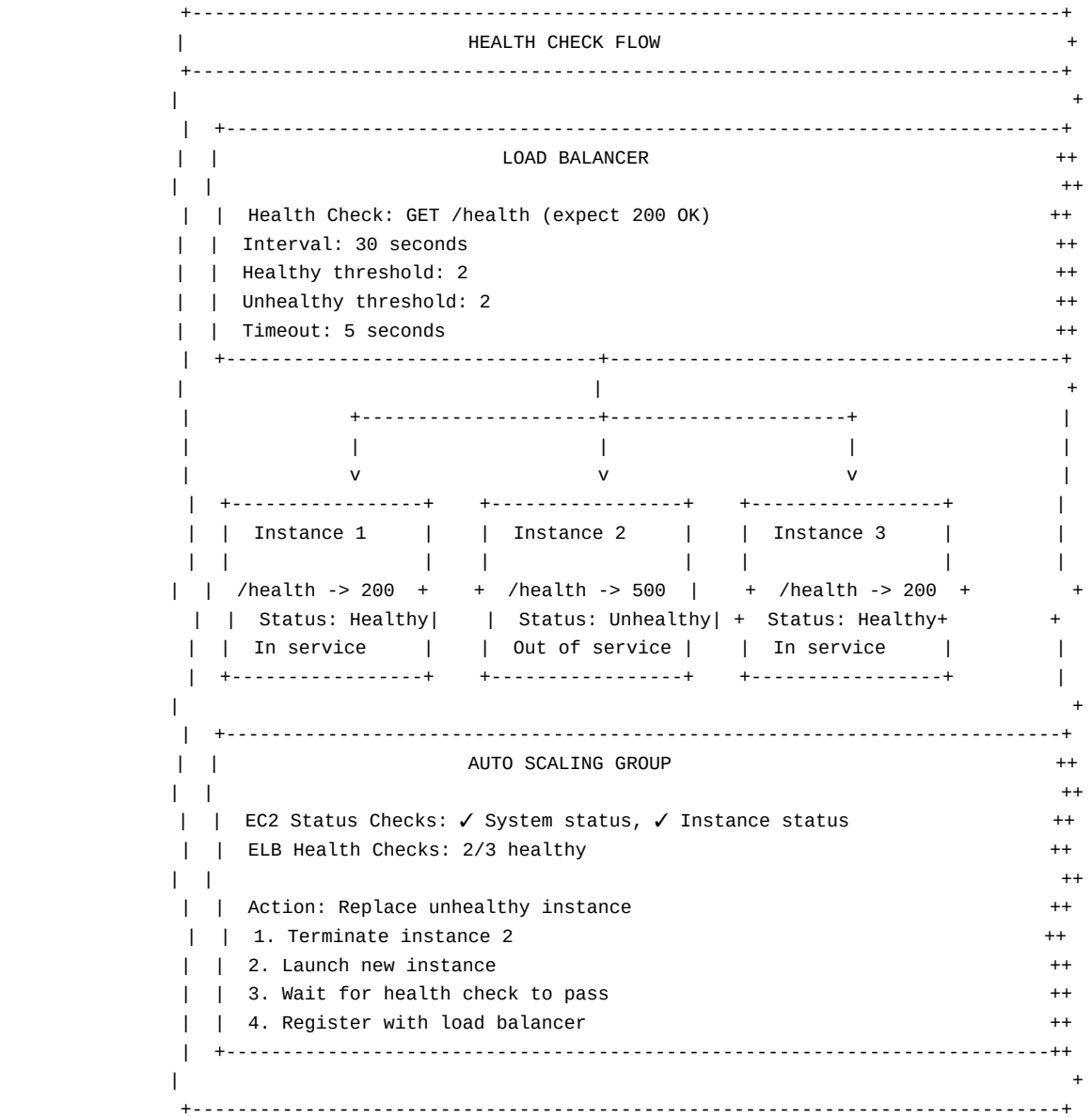
[Page 5]

Chapter 32: Health Checks & Self-Healing Systems

What are Health Checks?

Mechanisms to determine if an application instance is healthy and able to serve requests.
Types of Health Checks
Type
Level
Description

EC2 Status Checks
Instance
System status (hardware) + Instance status (software)
ELB Health Checks
Application
HTTP/HTTPS endpoint check
Custom Health Checks
Application
Custom logic via Lambda
CloudWatch Alarm
Metric
Based on metric thresholds
Diagram: Health Check Flow



Health Check Endpoint Implementation

```
# Flask health check endpoint
@app.route('/health')
def health_check():
                                          health_status = {
'status': 'healthy',
'checks': {
                                       'database': check_database(),
                                          'redis': check_redis(),
                                         'disk': check_disk_space()
}
}

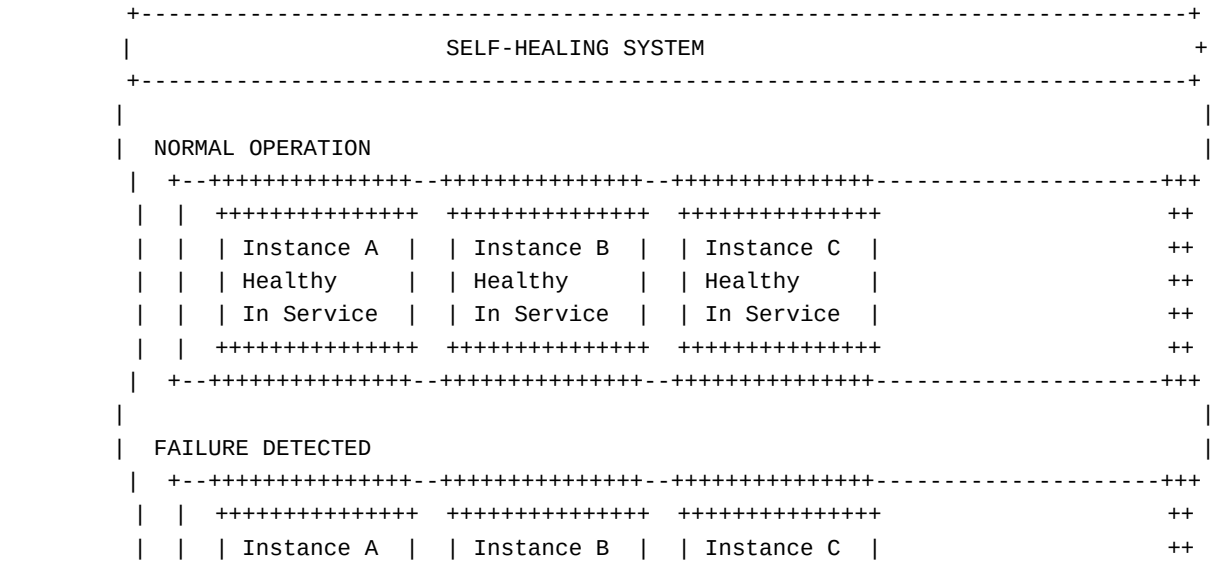
# If any dependency is unhealthy, overall status is unhealthy
                           for check, status in health_status['checks'].items():
if status != 'healthy':
                                        health_status['status'] = 'unhealthy'
                                         return jsonify(health_status), 500

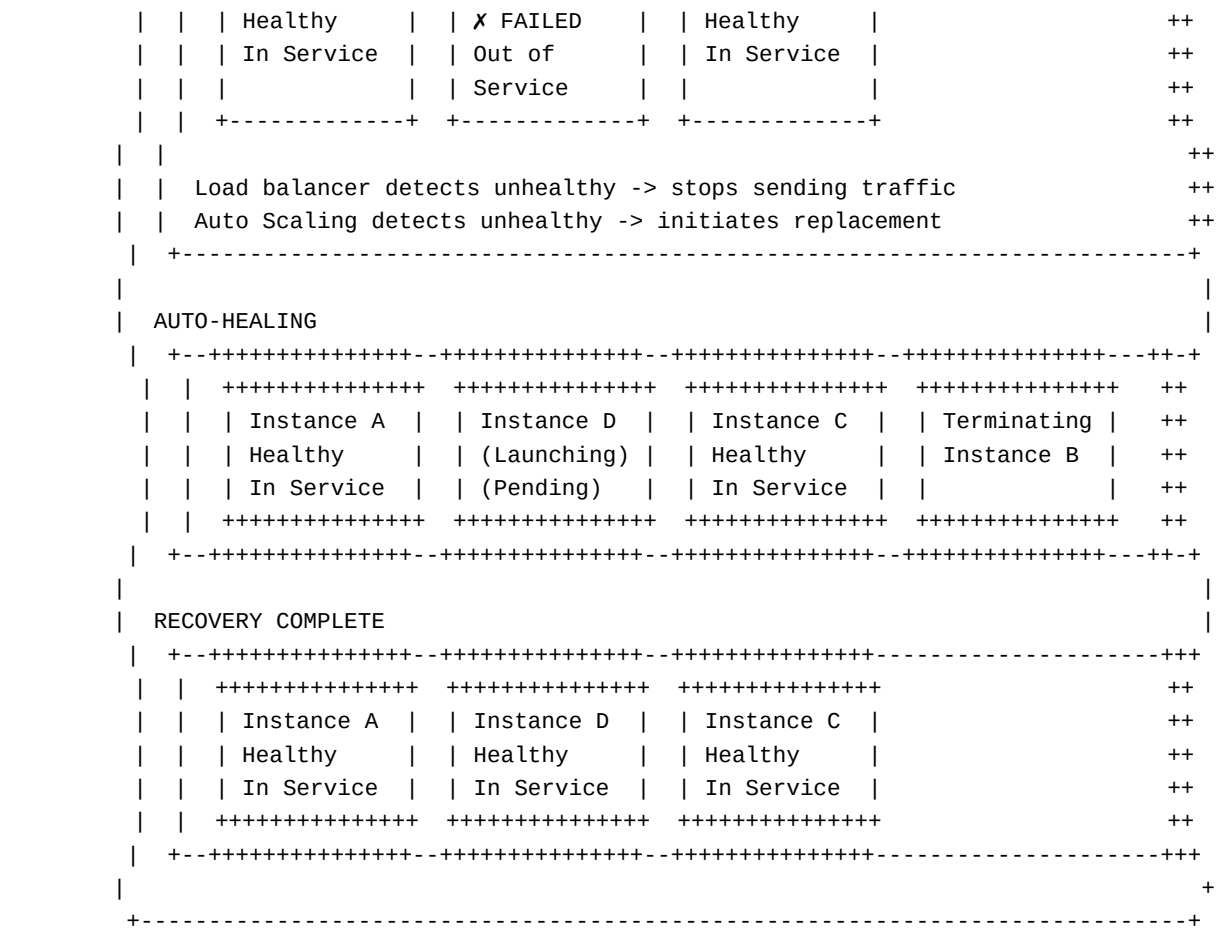
                                    return jsonify(health_status), 200
def check_database():
try:
db.session.execute('SELECT 1')
return 'healthy'
except:
return 'unhealthy'
```

Self-Healing Mechanisms

Mechanism	Description
Auto Scaling	Replaces failed instances automatically
Load Balancer	Stops sending traffic to unhealthy instances
Route 53	DNS failover to healthy endpoints
Multi-AZ	Automatic failover to standby
RDS Multi-AZ	Automatic database failover

Diagram: Self-Healing System





Custom Health Checks with CloudWatch

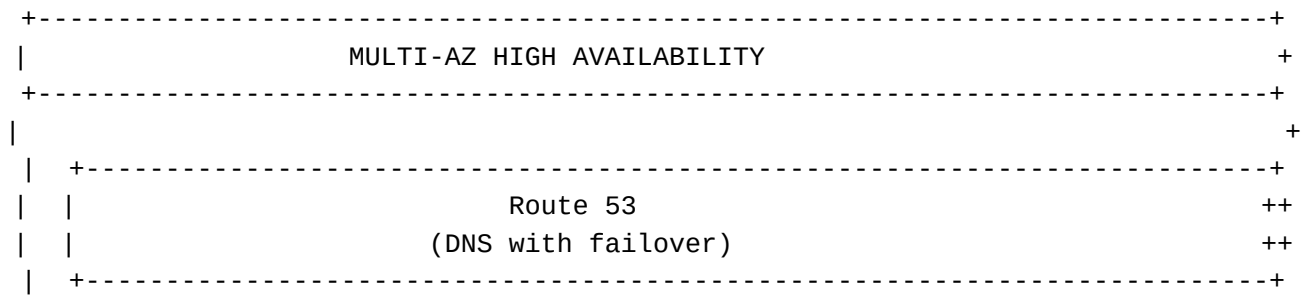
```
{
"Type": "AWS::AutoScaling::AutoScalingGroup",
"Properties": {
"HealthCheckType": "ELB",
"HealthCheckGracePeriod": 300,
"TargetGroupARNs": [...]
}
}
# CloudWatch alarm for custom metric
{
"AlarmName": "custom-health-alarm",
"MetricName": "CustomHealthCheck",
"Namespace": "MyApp",
"Statistic": "Average",
"Period": 60,
"EvaluationPeriods": 3,
"Threshold": 1,
"ComparisonOperator": "LessThanThreshold"
}
```

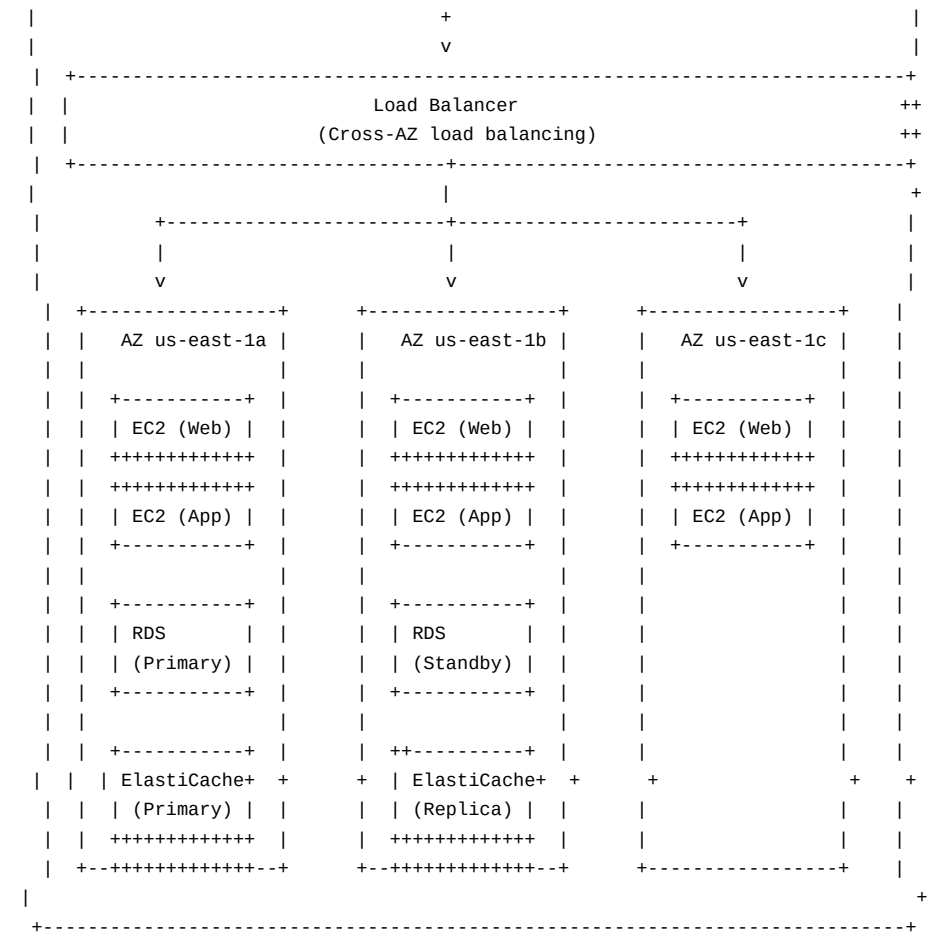
Health Check Best Practices

Practice	Why
Create dedicated /health endpoint	Lightweight, fast response
Check dependencies	Database, cache, external APIs

Practice
Why
Use short timeouts
Don't wait for long operations
Implement grace period
Allow instance to warm up
Log health check failures
Debugging and monitoring
Don't implement complex logic
Keep health checks simple

[Page 6]
Chapter 33: Multi-AZ High Availability Design
What is Multi-AZ High Availability?
Deploying resources across multiple Availability Zones (AZs) so that if one AZ fails, the system continues to operate.
Real Scenario
Your e-commerce application cannot afford downtime. If a data center loses power, your system must still serve customers.
Without Multi-AZ:
- Single AZ: one data center failure = entire application down
- Manual recovery (hours of downtime)
With Multi-AZ:
- Resources spread across 2+ AZs
- Automatic failover
- Zero downtime (if designed properly)
Diagram: Multi-AZ Architecture

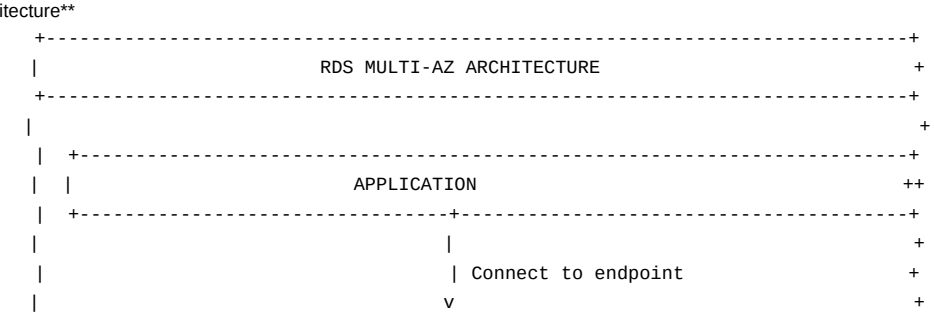


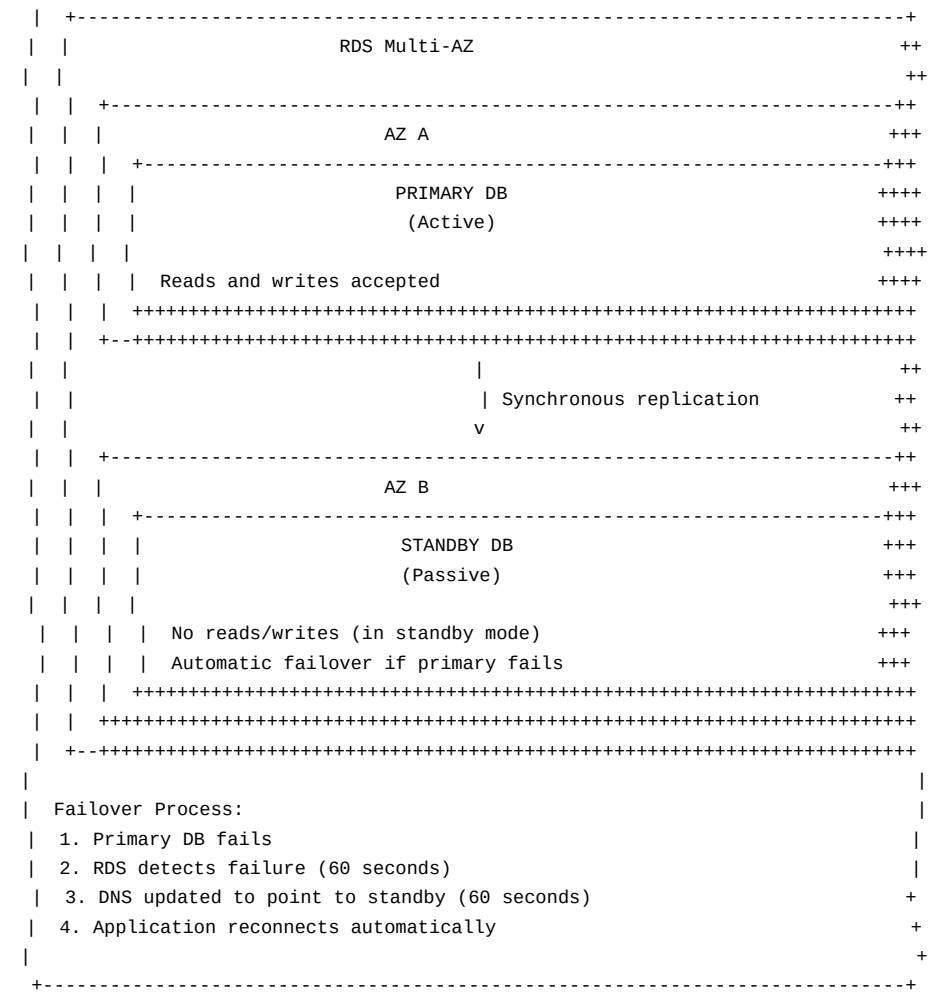


Multi-AZ Services

Service	Multi-AZ Configuration	Failover Time
EC2	Spread across AZs via ASG	Automatic (ASG)
Load Balancer	Cross-AZ enabled	Automatic
RDS	Multi-AZ (synchronous replication)	60-120 seconds
ElastiCache	Multi-AZ with replica	60-120 seconds
S3	11 nines durability (built-in)	Automatic
Route 53	Global service	Automatic

RDS Multi-AZ Architecture





Cross-AZ vs Single-AZ Load Balancer

Aspect	Single-AZ ALB	Cross-AZ ALB
Traffic distribution	Only to instances in same AZ	To all instances across AZs
AZ failure impact	100% loss if that AZ fails	Partial loss (other AZs continue)
Data transfer cost	Lower (no cross-AZ)	Higher (cross-AZ traffic costs)
Resilience	Low	High

Design Pattern: Active-Active vs Active-Passive

Pattern	Description	Failover	Use Case
Active-Active	All AZs handle traffic	Immediate	Stateless apps
Active-Passive	One AZ handles traffic, others standby	60-120 seconds	Stateful apps (databases)

Multi-AZ Best Practices

Practice	Why
----------	-----
Minimum 3 AZs	If one AZ fails, you still have 2
Spread instances	Use ASG across AZs
Use cross-AZ load balancer	Distribute traffic across AZs
RDS Multi-AZ	Automatic failover for databases
Test failover	Regularly test AZ failure scenarios
Monitor across AZs	Track health per AZ

[Page 7]

PART TWELVE: INFRASTRUCTURE AS CODE (IaC)

Chapter 34: Terraform Fundamentals

What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool that allows you to define and provision infrastructure using a declarative configuration language (HCL - HashiCorp Configuration Language).

Real Scenario

Your team needs to create consistent development, staging, and production environments. Manual clicking in AWS console is error-prone and not repeatable.

Without Terraform:

- Manual AWS console (slow, error-prone)

- Scripts (boto3, AWS CLI) - imperative, hard to manage

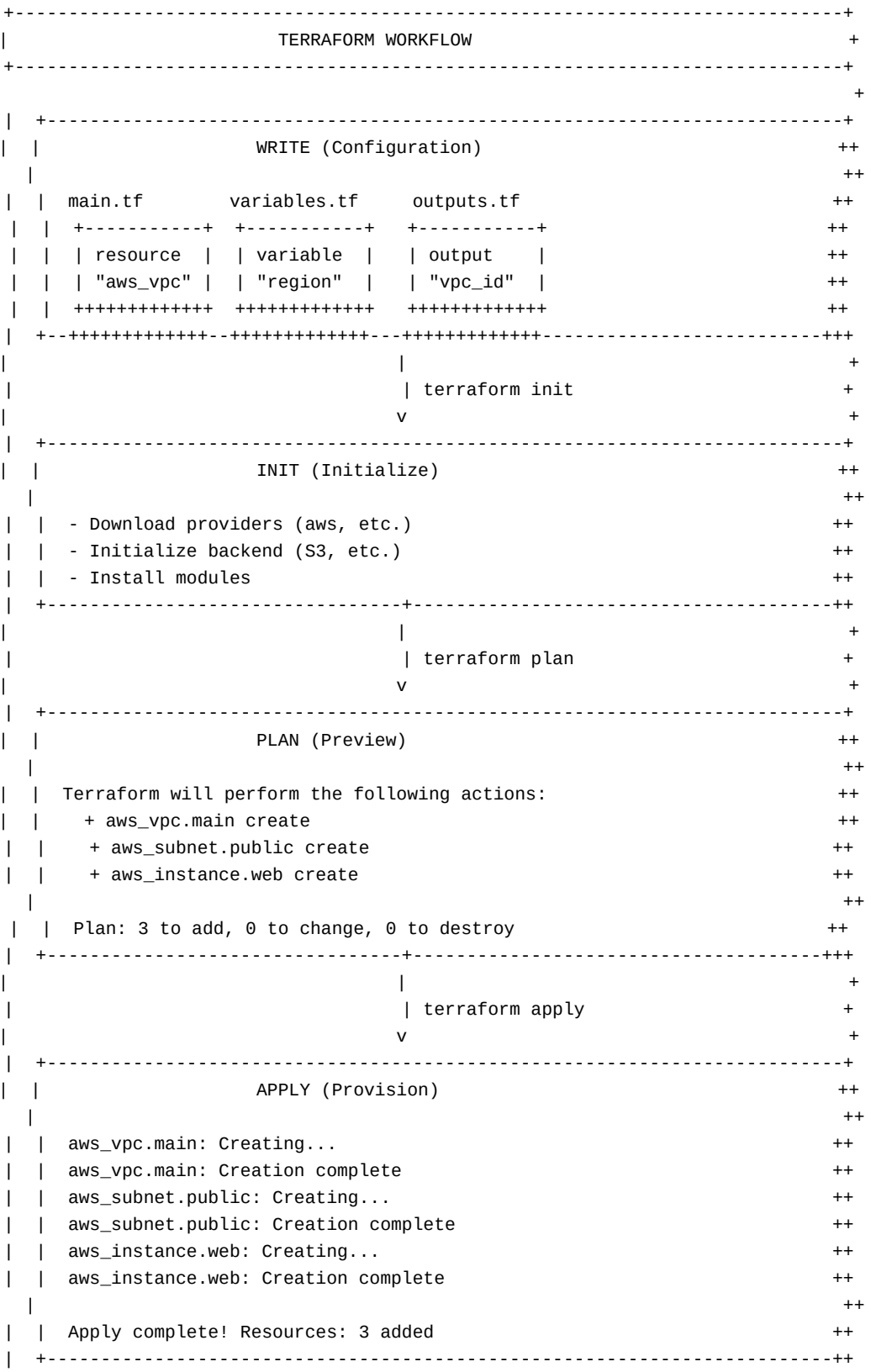
- No state management

- Difficult to track changes

With Terraform:

- Define infrastructure as code
- Version control (Git)
- Plan before apply
- State management
- Drift detection

Diagram: Terraform Workflow



```
|                                                                                +
 +-------------------------------------------------------------------------------+
```

Terraform Configuration Example (main.tf)

```
# Provider configuration
terraform {
required_providers {
aws = {
source  = "hashicorp/aws"
version = "~> 5.0"
}
}
}
# Configure AWS provider
provider "aws" {
region = var.aws_region
}
# Create VPC
resource "aws_vpc" "main" {
cidr_block           = var.vpc_cidr
enable_dns_hostnames = true
enable_dns_support   = true
tags = {
                            Name         = "${var.environment}-vpc"
Environment = var.environment
}
}
# Create public subnet
resource "aws_subnet" "public" {
count = length(var.public_subnet_cidrs)
vpc_id            = aws_vpc.main.id
cidr_block        = var.public_subnet_cidrs[count.index]
availability_zone = var.availability_zones[count.index]
map_public_ip_on_launch = true
tags = {
           Name         = "${var.environment}-public-subnet-${count.index + 1}"
Environment = var.environment
}
}
# Create security group
resource "aws_security_group" "web" {
name        = "${var.environment}-web-sg"
description = "Allow HTTP and HTTPS"
vpc_id      = aws_vpc.main.id
ingress {
                                      from_port   = 80
                                      to_port     = 80
protocol    = "tcp"
                                cidr_blocks = ["0.0.0.0/0"]
description = "HTTP from anywhere"
}
ingress {
```

```
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTPS from anywhere"
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name        = "${var.environment}-web-sg"
    Environment = var.environment
  }
}

# Create EC2 instance
resource "aws_instance" "web" {
  count         = var.instance_count
  ami           = var.ami_id
  instance_type = var.instance_type

  subnet_id                   = aws_subnet.public[count.index % length(aws_subnet.public)].id
  vpc_security_group_ids      = [aws_security_group.web.id]
  associate_public_ip_address = true

  user_data = <<-EOF
    #!/bin/bash
    yum update -y
    yum install -y httpd
    systemctl start httpd
    systemctl enable httpd
    echo "Hello from instance ${count.index + 1}" > /var/www/html/index.html
  EOF

  tags = {
    Name        = "${var.environment}-web-${count.index + 1}"
    Environment = var.environment
  }
}
```

Variables (variables.tf)

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "environment" {
  description = "Environment name (dev, staging, prod)"
  type        = string
  default     = "dev"
}

variable "vpc_cidr" {
  description = "VPC CIDR block"
  type        = string
  default     = "10.0.0.0/16"
```

```
}

variable "public_subnet_cidrs" {
  description = "Public subnet CIDR blocks"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24", "10.0.3.0/24"]
}

variable "availability_zones" {
  description = "Availability zones"
  type        = list(string)
  default     = ["us-east-1a", "us-east-1b", "us-east-1c"]
}

variable "instance_count" {
  description = "Number of EC2 instances"
  type        = number
  default     = 2
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}

variable "ami_id" {
  description = "AMI ID for EC2 instances"
  type        = string
  default     = "ami-0c55b159cbfafe1f0" # Amazon Linux 2 in us-east-1
}
```

Outputs (outputs.tf)

```
output "vpc_id" {
  description = "VPC ID"
  value       = aws_vpc.main.id
}

output "public_subnet_ids" {
  description = "Public subnet IDs"
  value       = aws_subnet.public[*].id
}

output "instance_public_ips" {
  description = "EC2 instance public IPs"
  value       = aws_instance.web[*].public_ip
}

output "instance_ids" {
  description = "EC2 instance IDs"
  value       = aws_instance.web[*].id
}
```

Basic Terraform Commands

Command	Purpose
`terraform init`	Initialize working directory (download providers)

Command	Purpose
`terraform plan`	Preview changes (what will be created/updated/deleted)
`terraform apply`	Apply changes (create/update resources)
`terraform destroy`	Destroy all resources
`terraform validate`	Validate configuration syntax
`terraform fmt`	Format configuration files
`terraform show`	Show current state
`terraform state list`	List resources in state

Terraform vs CloudFormation

Aspect	Terraform	CloudFormation
--------	----------	----------
Language	HCL (declarative)	JSON/YAML
Multi-cloud	Yes (AWS, Azure, GCP)	AWS only
State management	Terraform state file	Managed by AWS
Modules	Reusable modules	Nested stacks
Community	Large (open source)	AWS-specific
When to use	Multi-cloud, open source preference	AWS-only, managed service

---

[Page 8]

Chapter 35: State Management & Remote Backends (S3 + DynamoDB)

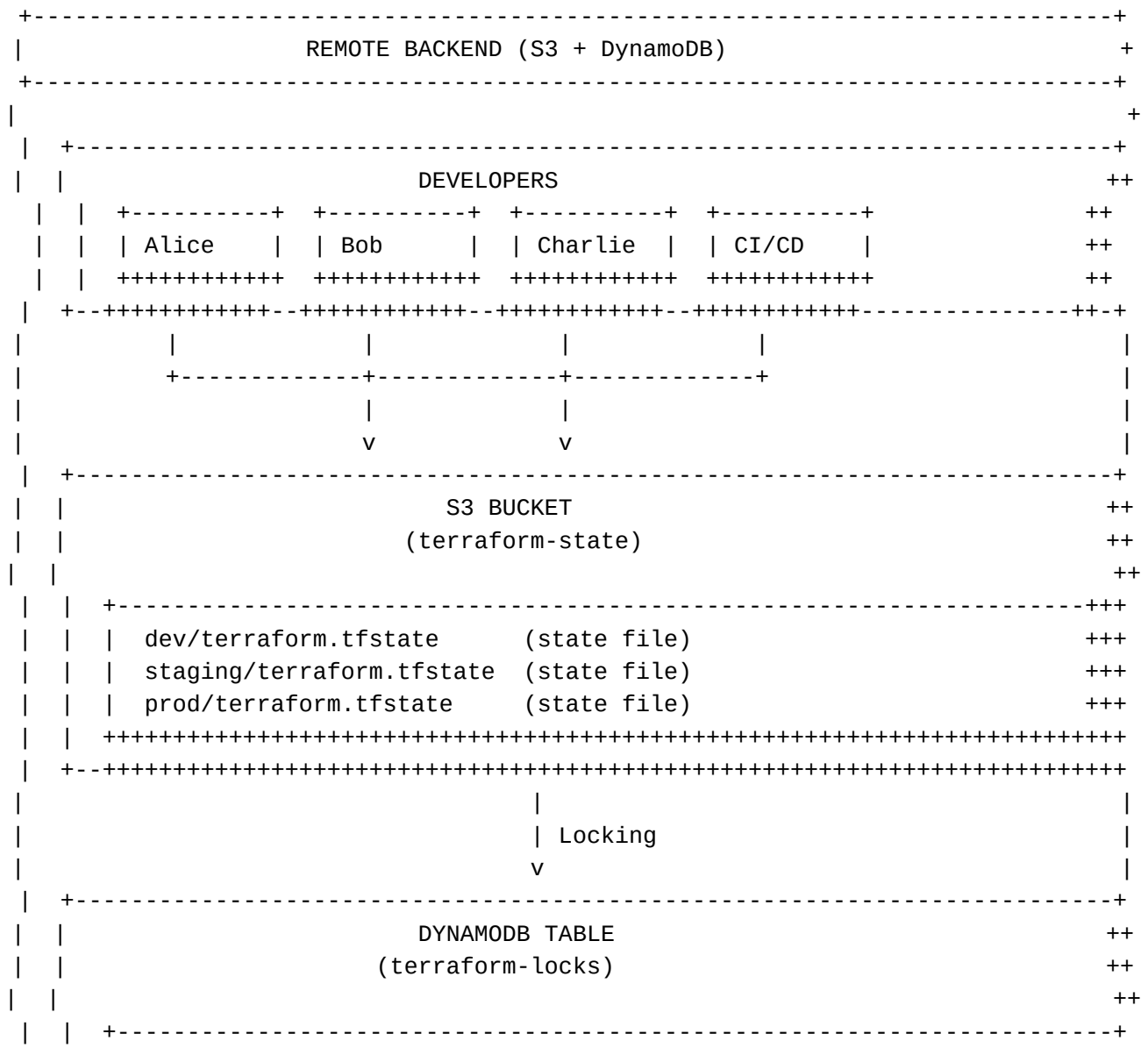
What is Terraform State?

Terraform state is a file that maps your configuration to real-world resources. It stores resource IDs, attributes, and dependencies.

Why State Matters

Aspect	Importance
--------	----------
Resource tracking	Knows what resources exist
Dependency management	Understands resource relationships

Aspect
Importance
Performance
Caches resource attributes
Drift detection
Identifies manual changes
Collaboration
Multiple team members need access
Problem with Local State
- **Local file** (terraform.tfstate) stored on your machine
- **Team collaboration** - state conflicts, overwrites
- **CI/CD** - state not accessible
- **Secrets** - may contain sensitive data
- **Accidental deletion** - losing state = losing infrastructure knowledge
Solution: Remote Backend
Store state remotely in a shared location with locking.
Diagram: Remote Backend Architecture



```
   |  |  |  LockID          | Info                                                  +++
   |  |  +------------------+-------------------------------------------------+++
   |  |  |  dev/terraform   | {"Operation":"apply","User":"alice","Timestamp":.}+++
   |  |  ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
   |  +--++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
  |                                                                               |
  |  Locking Process:                                                             |
   |  1. Alice runs 'terraform apply'                                            |
   |  2. Terraform acquires lock in DynamoDB                                     |
   |  3. Bob tries to run 'terraform plan'                                       |
  |  4. Terraform sees lock -> "state locked by Alice" error                      +
   |  5. Alice finishes, releases lock                                           |
   |  6. Bob can now run commands                                                |
  |                                                                               +
   +-------------------------------------------------------------------------------+
```

Backend Configuration (backend.tf)

```
terraform {
backend "s3" {
                        bucket         = "my-company-terraform-state"
key         = "dev/terraform.tfstate"
                                region         = "us-east-1"
encrypt      = true
                             dynamodb_table = "terraform-locks"
}
}
```

Complete Backend Setup with S3 and DynamoDB

```
# backend-setup.tf (run once to create backend infrastructure)
terraform {
required_providers {
aws = {
source = "hashicorp/aws"
}
}
}
# S3 bucket for state
resource "aws_s3_bucket" "terraform_state" {
bucket = "my-company-terraform-state"
lifecycle {
                                    prevent_destroy = true
}
}
# Enable versioning
resource "aws_s3_bucket_versioning" "terraform_state" {
bucket = aws_s3_bucket.terraform_state.id
versioning_configuration {
status = "Enabled"
}
}
# Enable encryption
resource "aws_s3_bucket_server_side_encryption_configuration" "terraform_state" {
bucket = aws_s3_bucket.terraform_state.id
rule {
```

```
                    apply_server_side_encryption_by_default {
                        sse_algorithm = "AES256"
}
}
}
# Block public access
resource "aws_s3_bucket_public_access_block" "terraform_state" {
bucket = aws_s3_bucket.terraform_state.id
block_public_acls       = true
block_public_policy     = true
ignore_public_acls      = true
restrict_public_buckets = true
}
# DynamoDB table for state locking
resource "aws_dynamodb_table" "terraform_locks" {
name         = "terraform-locks"
billing_mode = "PAY_PER_REQUEST"
hash_key     = "LockID"
attribute {
name = "LockID"
type = "S"
}
}
```

Environment-Specific State Files

```
s3://my-terraform-state/
                                    +-+ dev/
                              |   +-+ terraform.tfstate
                                  +-+ staging/
                              |   +-+ terraform.tfstate
                                    +-+ prod/
                              |   +-+ terraform.tfstate
                                  +-+ global/
                +-+ terraform.tfstate (shared resources like IAM roles)
```

State Commands

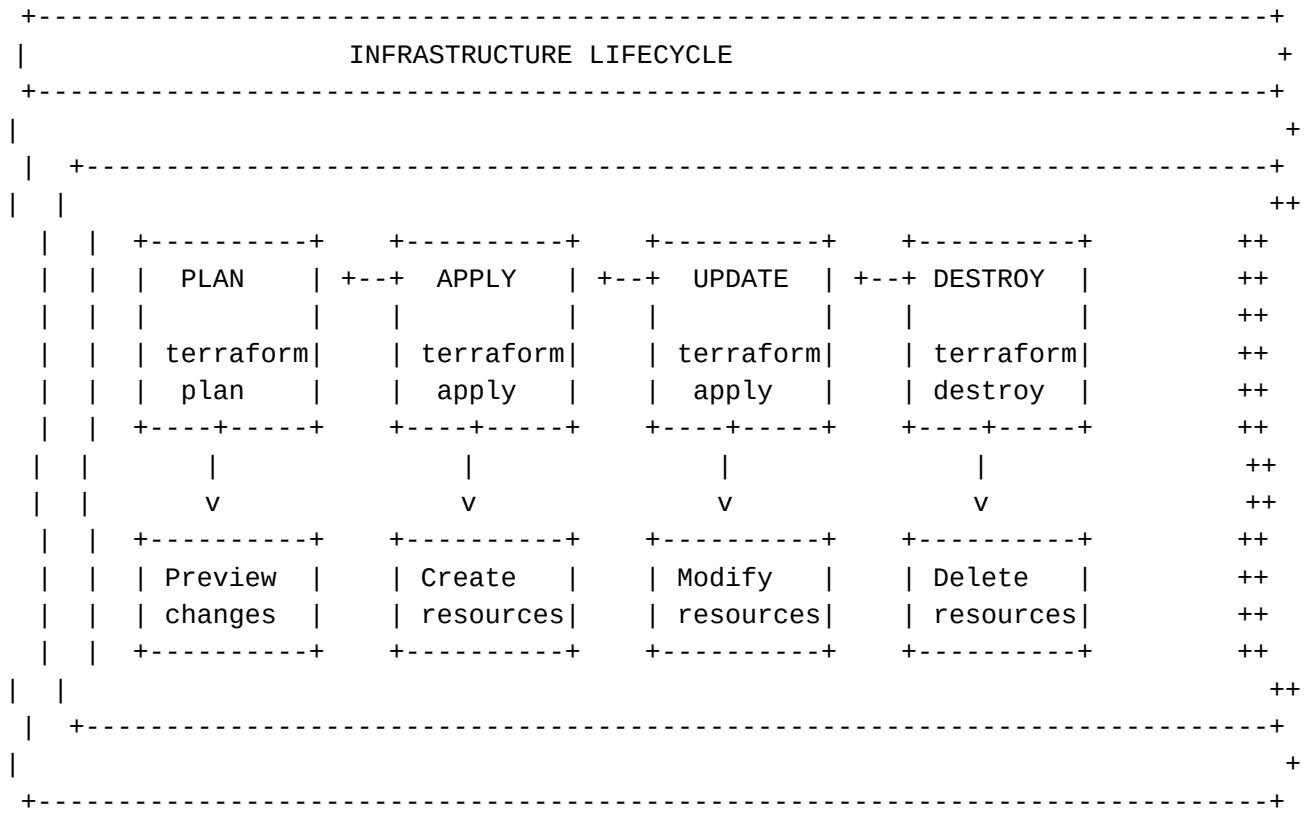
Command	Purpose
`terraform state list`	List all resources in state
`terraform state show <resource>`	Show resource details
`terraform state mv <src> <dst>`	Move resource in state
`terraform state rm <resource>`	Remove resource from state
`terraform import <resource> <id>`	Import existing resource

State Best Practices

Practice
Why

Use remote backend
Team collaboration, security
Enable versioning
Recover from mistakes
Enable encryption
Protect sensitive data
Use locking
Prevent concurrent modifications
Separate state per environment
Isolate changes
Never edit state manually
Corruption risk
Store in separate account
Security best practice
Backup state
Disaster recovery

[Page 9]
Chapter 36: Infrastructure Lifecycle Management
What is Infrastructure Lifecycle Management?
The process of managing infrastructure from creation through updates to eventual deletion, including provisioning, configuration, and decommissioning.
Infrastructure Lifecycle Phases



Terraform Lifecycle Rules

```
# Lifecycle rules for resources
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"

  lifecycle {
    create_before_destroy = true  # Create new before destroying old
    prevent_destroy       = true  # Prevent accidental deletion
    ignore_changes        = [     # Ignore changes to these attributes
      ami,
      user_data,
    ]
  }
}
```

Lifecycle Settings

Setting	Description	Use Case
---	---	---
`create_before_destroy`	Create replacement before destroying original	Zero-downtime updates
`prevent_destroy`	Block terraform destroy for this resource	Critical resources
`ignore_changes`	Ignore changes to specified attributes	Auto-scaling changes
`replace_triggered_by`	Trigger replacement when other resource changes	Dependency-based updates

Terraform Workspaces

Manage multiple environments with the same configuration.

```
# Create workspaces
terraform workspace new dev
terraform workspace new staging
terraform workspace new prod

# List workspaces
terraform workspace list

# Switch workspace
terraform workspace select prod

# Use workspace name in configuration
resource "aws_instance" "web" {
  tags = {
    Environment = terraform.workspace
  }
}
```

Terraform Modules

Reusable infrastructure components.

```
# modules/vpc/main.tf
resource "aws_vpc" "main" {
  cidr_block = var.cidr_block
}

# modules/ec2/main.tf
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = var.instance_type
}

# root/main.tf - using modules
module "vpc" {
  source = "./modules/vpc"
  cidr_block = "10.0.0.0/16"
}

module "web" {
  source = "./modules/ec2"
  ami_id = "ami-0c55b159cbfafe1f0"
  instance_type = "t3.micro"
}
```

CI/CD Integration (GitHub Actions)

```
# .github/workflows/terraform.yml
name: Terraform CI/CD

on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main

jobs:
  terraform:
    runs-on: ubuntu-latest
    environment: ${{ github.ref_name }}

    steps:
      - uses: actions/checkout@v3

      - uses: hashicorp/setup-terraform@v2
        with:
          terraform_version: 1.5.0

      - name: Terraform Init
        run: terraform init
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

      - name: Terraform Plan
        run: terraform plan
        env:
          AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
          AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}

      - name: Terraform Apply
        if: github.ref == 'refs/heads/main'
        run: terraform apply -auto-approve
```

```
      env:
        AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
        AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
```

Drift Detection

Detect when infrastructure has been modified outside of Terraform.

```
# Refresh state to detect drift
terraform refresh

# Plan to see differences
terraform plan

# Compare with actual infrastructure
terraform state list
terraform state show aws_instance.web
```

Lifecycle Best Practices

Practice	Why
----------	-----
Version control	Track infrastructure changes
Use workspaces	Separate environments
Use modules	Reuse and share infrastructure
Automate with CI/CD	Consistent, repeatable deployments
Prevent destroy for critical	Protect important resources
Regular drift detection	Catch manual changes
Plan before apply	Review changes first
Tag resources	Identify ownership, environment

[Page 10]

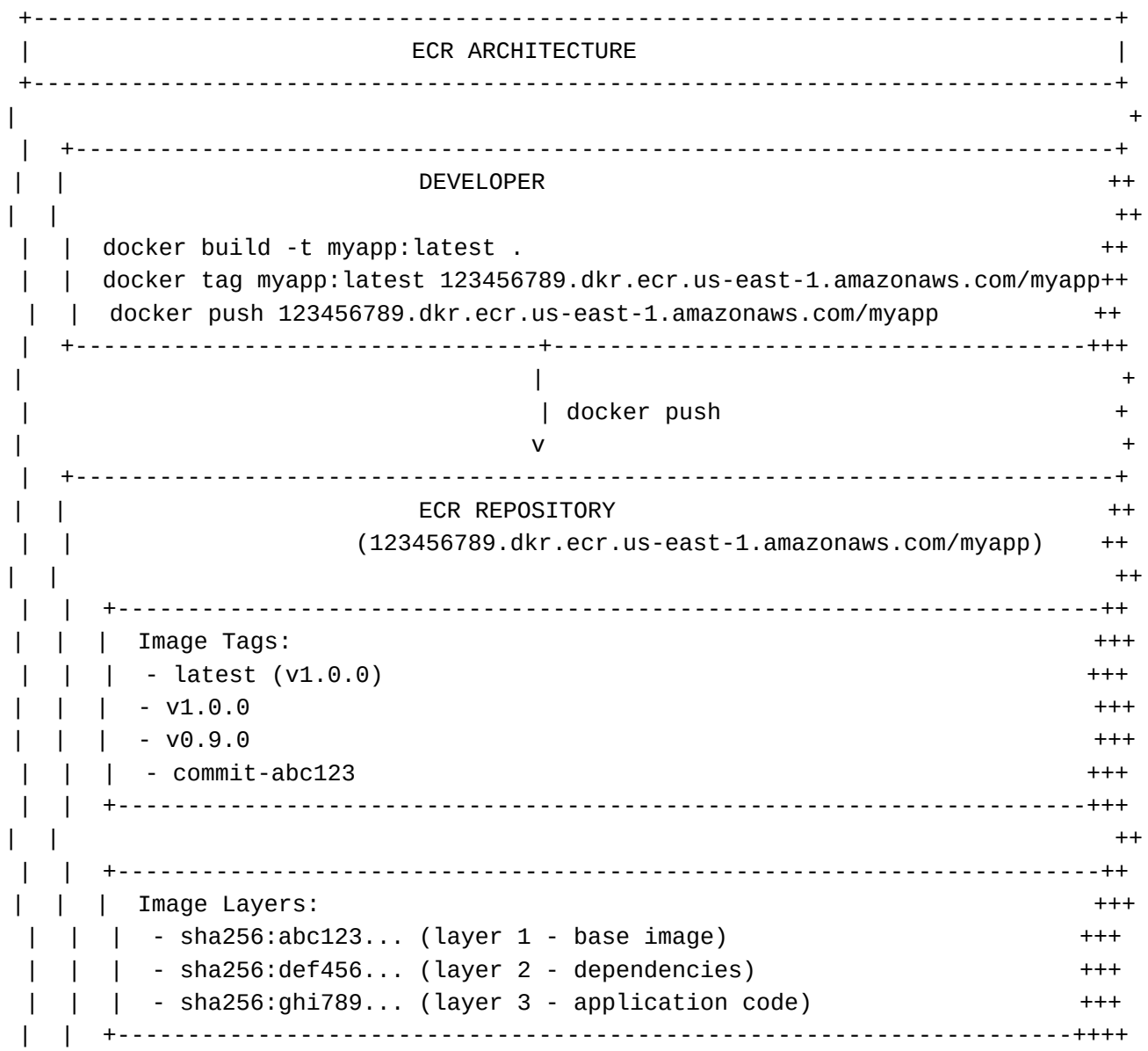
PART THIRTEEN: CONTAINER ECOSYSTEM

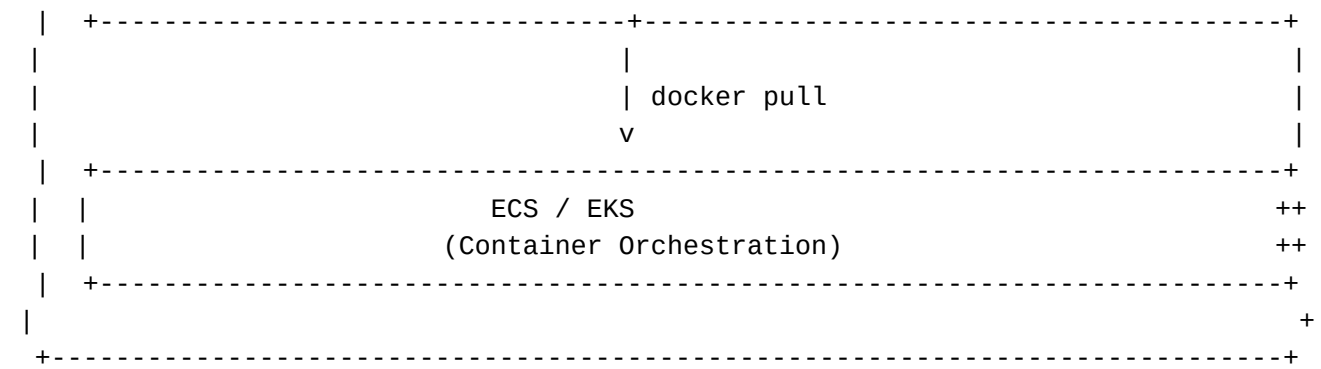
Chapter 37: ECR (Elastic Container Registry)

What is ECR?

Elastic Container Registry (ECR) is a fully managed Docker container registry that makes it easy to store, manage, and deploy container images.

Real Scenario
Your team builds Docker images for multiple microservices. You need a place to store these images and deploy them to ECS or EKS.
Without ECR:
- Docker Hub (public images, rate limits)
- Self-hosted registry (additional management)
- No integration with AWS IAM
With ECR:
- Fully managed, integrated with AWS
- IAM authentication
- Vulnerability scanning
- Lifecycle policies
Diagram: ECR Architecture





ECR Commands

```
# Authenticate Docker to ECR
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin 123456789.dkr.ecr.us-east-1.amazonaws.com
# Create repository
aws ecr create-repository \
--repository-name myapp \
--image-scanning-configuration scanOnPush=true \
--encryption-configuration encryptionType=AES256
# List repositories
aws ecr describe-repositories
# Tag local image
docker tag myapp:latest 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
# Push image
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
# Pull image
docker pull 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
# List images
aws ecr list-images --repository-name myapp
# Delete image
aws ecr batch-delete-image \
--repository-name myapp \
--image-ids imageTag=old-version
```

ECR Repository Policies (Resource-based)

```
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "AllowPull",
"Effect": "Allow",
"Principal": {
"AWS": "arn:aws:iam::123456789:role/ecsTaskRole"
},
"Action": [
"ecr:GetDownloadUrlForLayer",
"ecr:BatchGetImage",
"ecr:BatchCheckLayerAvailability"
]
},
{
```

```
        "Sid": "AllowPush",
        "Effect": "Allow",
        "Principal": {
          "AWS": "arn:aws:iam::123456789:role/ci-cd-role"
        },
        "Action": [
          "ecr:InitiateLayerUpload",
          "ecr:UploadLayerPart",
          "ecr:CompleteLayerUpload",
          "ecr:PutImage"
        ]
      }
    ]
}
```

When to Use ECR

Use Case	Why ECR
----------	---------
Store Docker images	Managed, secure container registry
CI/CD integration	Push/pull from build pipelines
ECS/EKS integration	Native integration with AWS compute
Vulnerability scanning	Built-in image scanning
Lifecycle management	Automatic image cleanup

Advantages of ECR

1. **Fully managed:** No servers to maintain
2. **Integrated with IAM:** Fine-grained access control
3. **High availability:** 99.9% availability SLA
4. **Vulnerability scanning:** Basic scanning included
5. **Lifecycle policies:** Automatic image cleanup
6. **Cross-region replication:** Copy images across regions

Disadvantages of ECR

1. **AWS only:** Not multi-cloud (though can be used with other clouds)
2. **Cost:** Storage and data transfer costs
3. **Learning curve:** Different from Docker Hub

ECR vs Docker Hub

Aspect	ECR	Docker Hub
--------	-----	----------
Integration	AWS native	Universal
Authentication	IAM	Docker ID
Private repos	Unlimited (pay for storage)	1 free private
Vulnerability scanning	Basic included	Enhanced with paid plan
Rate limits	Based on AWS account	Pull rate limits
Best for	AWS workloads	Open source, public images

[Page 11]

Chapter 38: Image Management & Versioning

What is Image Management?

The practice of organizing, tagging, storing, and maintaining container images throughout their lifecycle.

Image Tagging Strategies

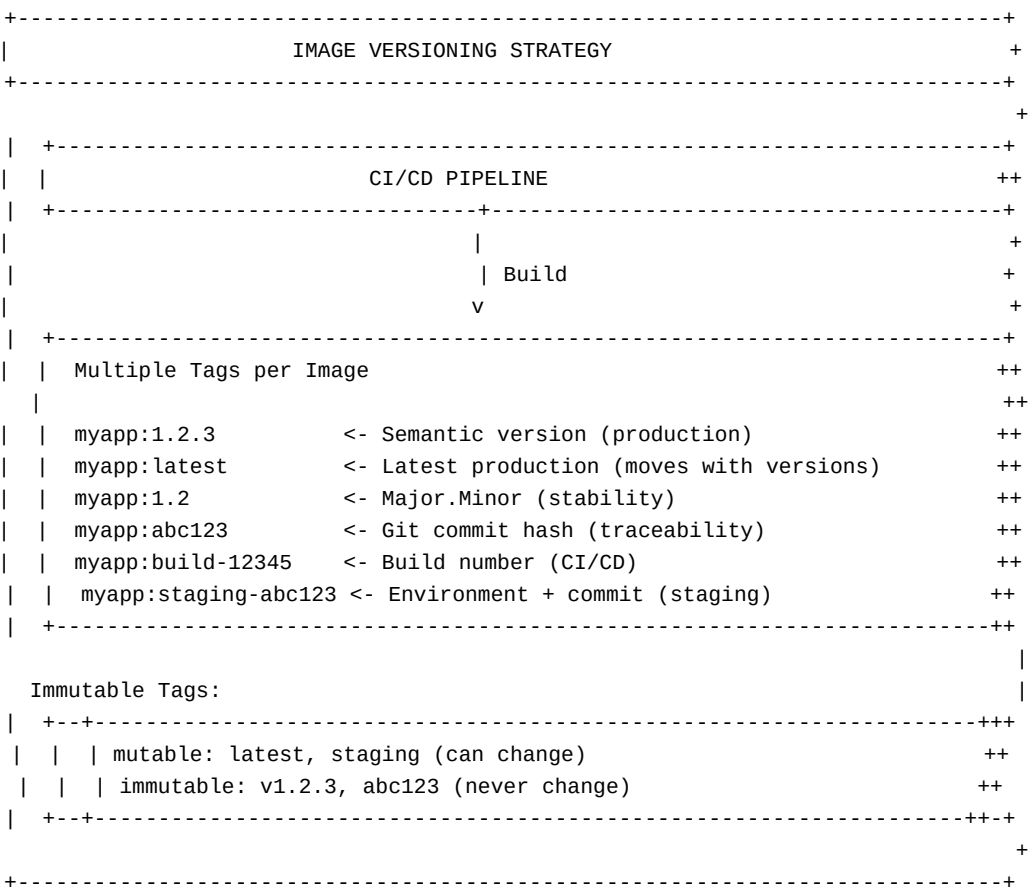
Strategy	Example	Pros	Cons
----------	---------	------	------
Latest	`myapp:latest`	Simple	No version history
Semantic version	`myapp:v1.2.3`	Clear versioning	Manual updates
Git commit hash	`myapp:abc123`	Traceable	Hard to compare
Build number	`myapp:build-123`	Unique, sequential	Not semantic
Date-based	`myapp:2024-01-15`	Temporal	Long names

Best Practice: Multiple Tags

```
# Tag with multiple identifiers
docker tag myapp:local 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
docker tag myapp:local 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:v1.2.3
docker tag myapp:local 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp:abc123

# Push all tags
docker push 123456789.dkr.ecr.us-east-1.amazonaws.com/myapp --all-tags
```

Image Versioning Strategy



ECR Lifecycle Policies

Automatically clean up old images.


```
{
"rules": [
{
"rulePriority": 1,
"description": "Keep last 10 production images",
"selection": {
"tagStatus": "tagged",
"tagPrefixList": ["v"],
"countType": "imageCountMoreThan",
"countNumber": 10
},
"action": {
"type": "expire"
}
},
{
"rulePriority": 2,
"description": "Expire untagged images after 14 days",
"selection": {
"tagStatus": "untagged",
"countType": "sinceImagePushed",
"countUnit": "days",
"countNumber": 14
},
"action": {
"type": "expire"
}
},
```

```
    {
      "rulePriority": 3,
      "description": "Expire staging images after 7 days",
      "selection": {
        "tagStatus": "tagged",
        "tagPrefixList": ["staging"],
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 7
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

Image Management Best Practices

Practice	Why
----------	-----
Use immutable tags	Reproducible deployments
Tag with commit hash	Traceability to source code
Automate versioning	CI/CD should tag images
Clean up old images	Reduce storage costs
Scan images regularly	Security vulnerabilities
Use minimal base images	Smaller images, faster pulls
Layer caching	Optimize build times

[Page 39]

Chapter 39: Image Scanning Basics

What is Image Scanning?

Analyzing container images for known vulnerabilities, malware, and security issues.

Real Scenario

You have hundreds of container images. Some may contain outdated libraries with known CVEs (Common Vulnerabilities and Exposures).

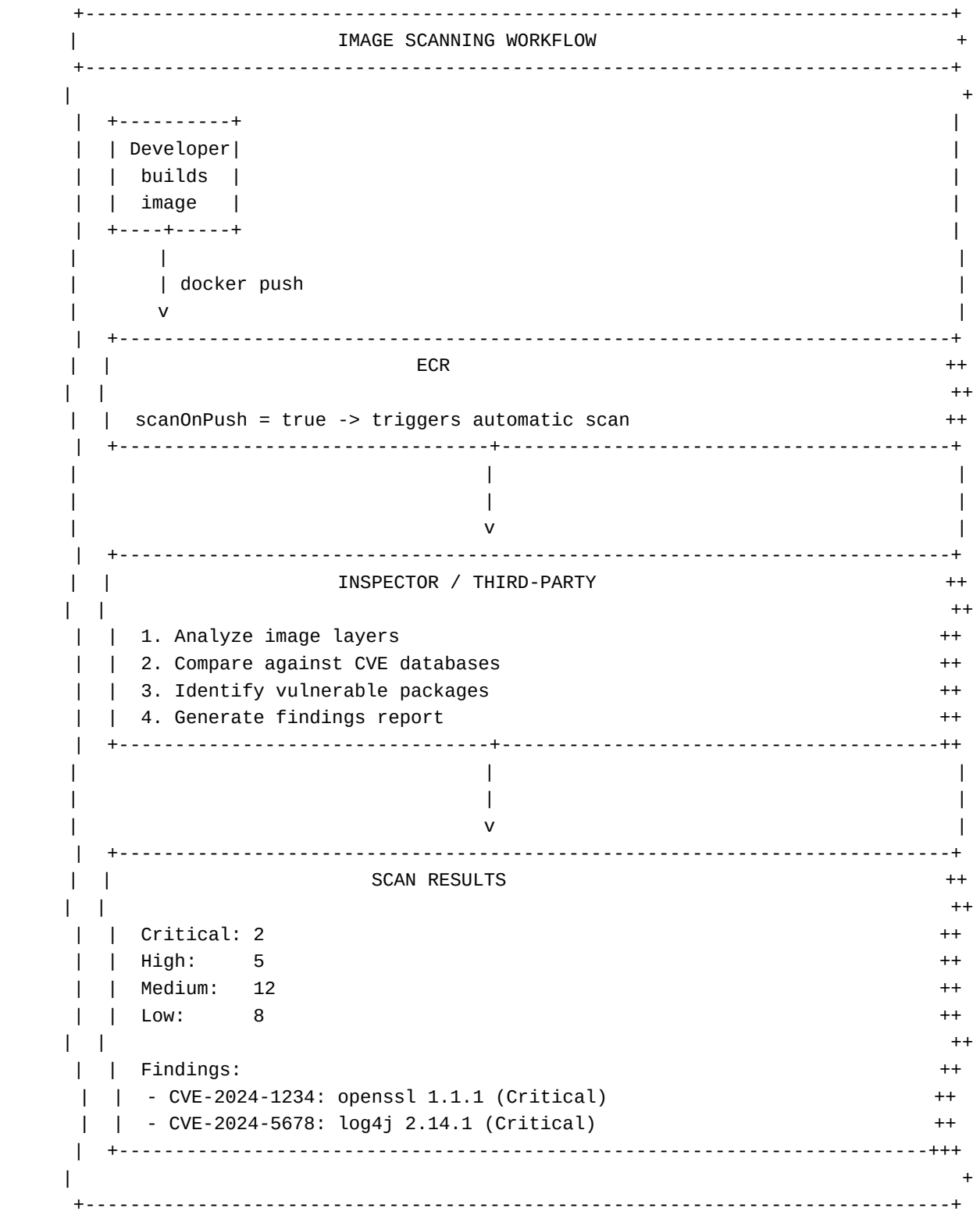
Without Scanning:

- Vulnerable images deployed to production
- Security breaches from known exploits
- Compliance violations

With Scanning:

- Identify vulnerabilities before deployment
- Block vulnerable images
- Meet compliance requirements

Diagram: Image Scanning Flow



ECR Enhanced Scanning (with Inspector)

```
{
  "repositoryName": "myapp",
  "imageScanningConfiguration": {
    "scanOnPush": true
  },
  "encryptionConfiguration": {
    "encryptionType": "AES256"
  }
}
```

Scan Results Interpretation

Severity	Action	Time to fix
----------	--------	------------
Critical	Block deployment, fix immediately	Hours
High	Block deployment (or with approval)	Days
Medium	Review, schedule fix	Week
Low	Monitor, fix in next release	Next sprint

CI/CD Integration (Block Vulnerable Images)

```
# GitHub Actions workflow
name: Image Scan

jobs:
  scan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v3

      - name: Build image
        run: docker build -t myapp:latest .

      - name: Scan image with Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: myapp:latest
          format: 'sarif'
          output: 'trivy-results.sarif'
          severity: 'CRITICAL,HIGH'

      - name: Fail if critical vulnerabilities found
        run: |
          if grep -q '"severity":"CRITICAL"' trivy-results.sarif; then
            echo "Critical vulnerabilities found!"
            exit 1
          fi
```

Image Scanning Best Practices

Practice	Why
----------	-----
Scan on every push	Catch vulnerabilities early
Block critical/high	Prevent vulnerable images
Use minimal base images	Fewer vulnerabilities
Regular base image updates	Patch known CVEs
Pin base image versions	Reproducible builds
Monitor vulnerability databases	New CVEs discovered daily
Automated remediation	Rebuild images automatically

[Page 12]

PART FOURTEEN: DEPLOYMENT STRATEGIES

Chapter 40: Blue-Green Deployment

What is Blue-Green Deployment?

A deployment strategy that uses two identical environments (blue and green). Only one environment serves live traffic at a time, enabling instant rollbacks.

Real Scenario

You need to deploy a new version of your application with zero downtime and instant rollback capability.

Without Blue-Green:

- Rolling deployment (some users see old version, some new)

- Slow rollback (requires redeployment)

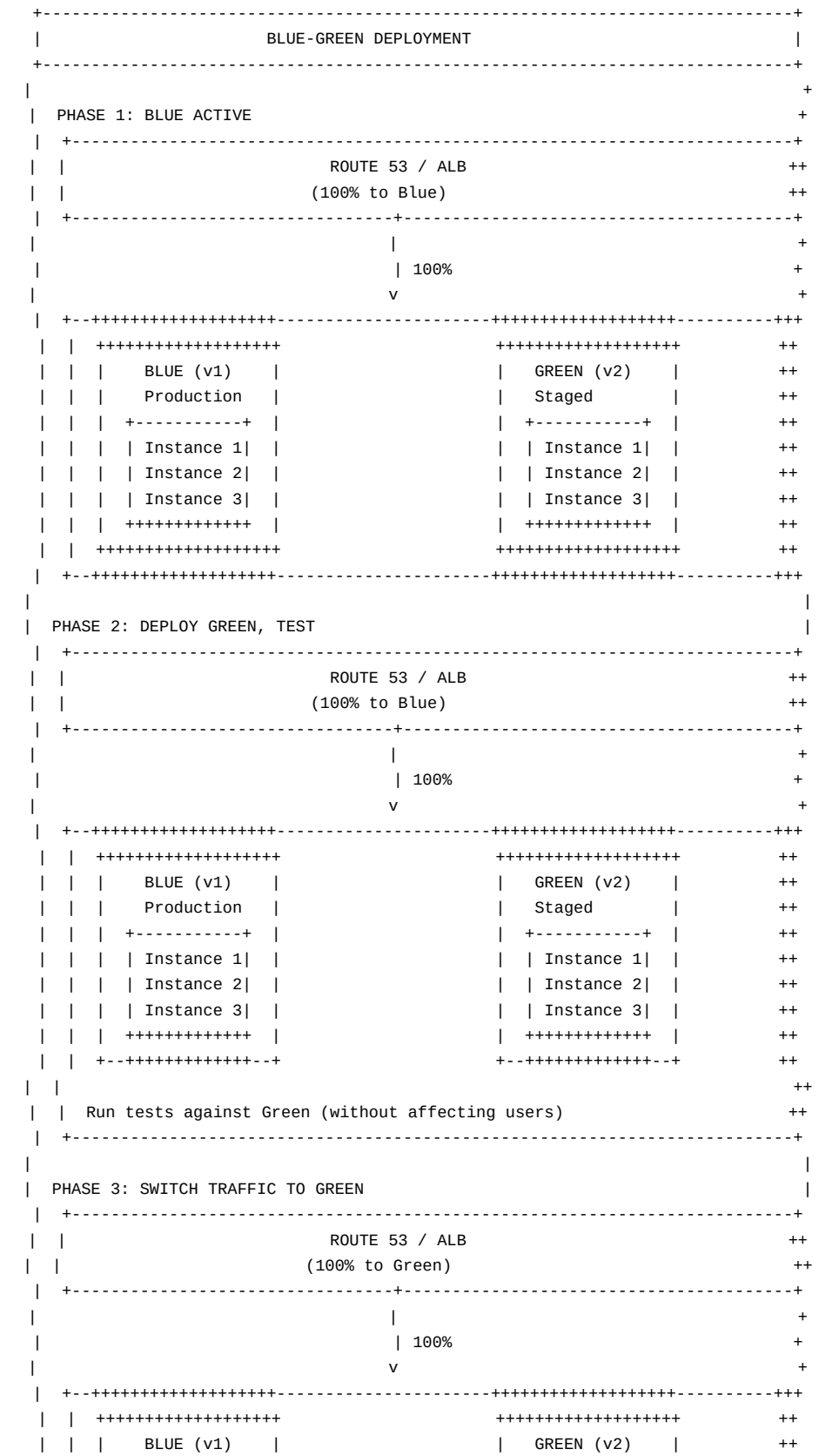
- Risk of downtime

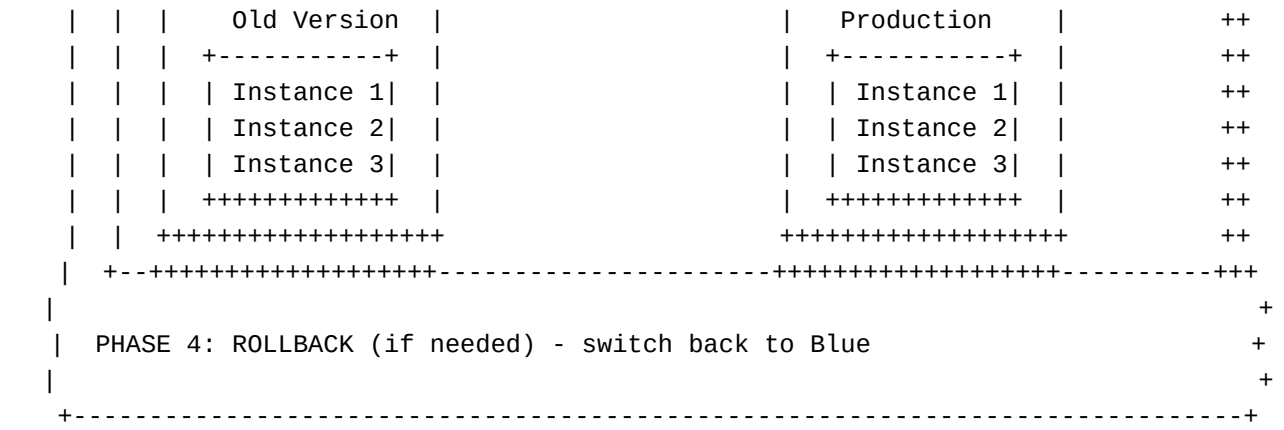
With Blue-Green:

- Zero downtime (instant switch)

- Instant rollback (switch back)
- Full testing in production environment

Diagram: Blue-Green Deployment





Blue-Green with Route 53 (Weighted Records)

```
# Route 53 weighted records
resource "aws_route53_record" "blue" {
zone_id = data.aws_route53_zone.main.zone_id
name    = "api.example.com"
type    = "A"
weighted_routing_policy {
weight = 100
}
set_identifier = "blue"
alias {
                        name                   = aws_lb.blue.dns_name
                        zone_id                = aws_lb.blue.zone_id
                                evaluate_target_health = true
}
}
resource "aws_route53_record" "green" {
zone_id = data.aws_route53_zone.main.zone_id
name    = "api.example.com"
type    = "A"
weighted_routing_policy {
weight = 0
}
set_identifier = "green"
alias {
                       name                   = aws_lb.green.dns_name
                        zone_id                = aws_lb.green.zone_id
                                evaluate_target_health = true
}
}
```

Blue-Green Deployment Script

```
#!/bin/bash
# Deploy to Green environment
aws ecs update-service --cluster my-cluster --service green-service --force-new-deployment
# Wait for deployment to complete
aws ecs wait services-stable --cluster my-cluster --services green-service
# Run smoke tests against Green
curl -f https://green-api.example.com/health || exit 1
# Switch traffic to Green
aws route53 change-resource-record-sets --hosted-zone-id Z123456 --change-batch '{
```

```
    "Changes": [
      {
        "Action": "UPSERT",
        "ResourceRecordSet": {
          "Name": "api.example.com",
          "Type": "A",
          "Weight": 100,
          "SetIdentifier": "green",
          "AliasTarget": {
            "HostedZoneId": "Z123456",
            "DNSName": "green-lb.example.com",
            "EvaluateTargetHealth": true
          }
        }
      },
      {
        "Action": "UPSERT",
        "ResourceRecordSet": {
          "Name": "api.example.com",
          "Type": "A",
          "Weight": 0,
          "SetIdentifier": "blue",
          "AliasTarget": {
            "HostedZoneId": "Z123456",
            "DNSName": "blue-lb.example.com",
            "EvaluateTargetHealth": true
          }
        }
      }
    ]
  }'
```

When to Use Blue-Green

Use Case	Why Blue-Green
----------	----------
Zero downtime required	Instant switch, no interruption
Quick rollback needed	Instant switch back
Full testing in production	Test in Green before switch
High-risk changes	Minimize blast radius

Advantages of Blue-Green

1. **Zero downtime:** Instant switch
2. **Instant rollback:** Switch back to previous version
3. **Full testing:** Test in production environment
4. **Simple mental model:** Easy to understand
5. **Reduced risk:** Canary not needed

Disadvantages of Blue-Green

1. **Double infrastructure:** Two complete environments
2. **Database challenges:** Schema must be compatible
3. **Stateful services:** Harder to handle state
4. **Cost:** Double the resources during deployment

[Page 13]

Chapter 41: Rolling Deployment

What is Rolling Deployment?

Gradually replaces instances of the old version with new version, one batch at a time, while keeping the application available.

Real Scenario

You need to deploy a new version with zero downtime, but you don't want to double your infrastructure costs.

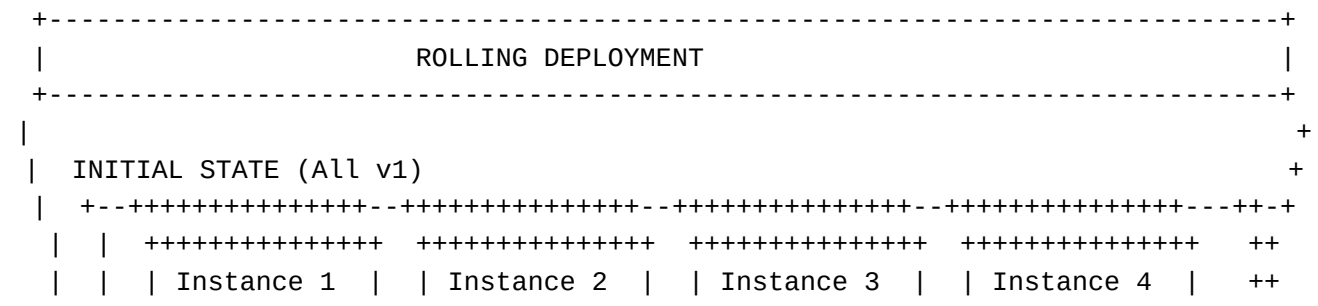
Without Rolling:

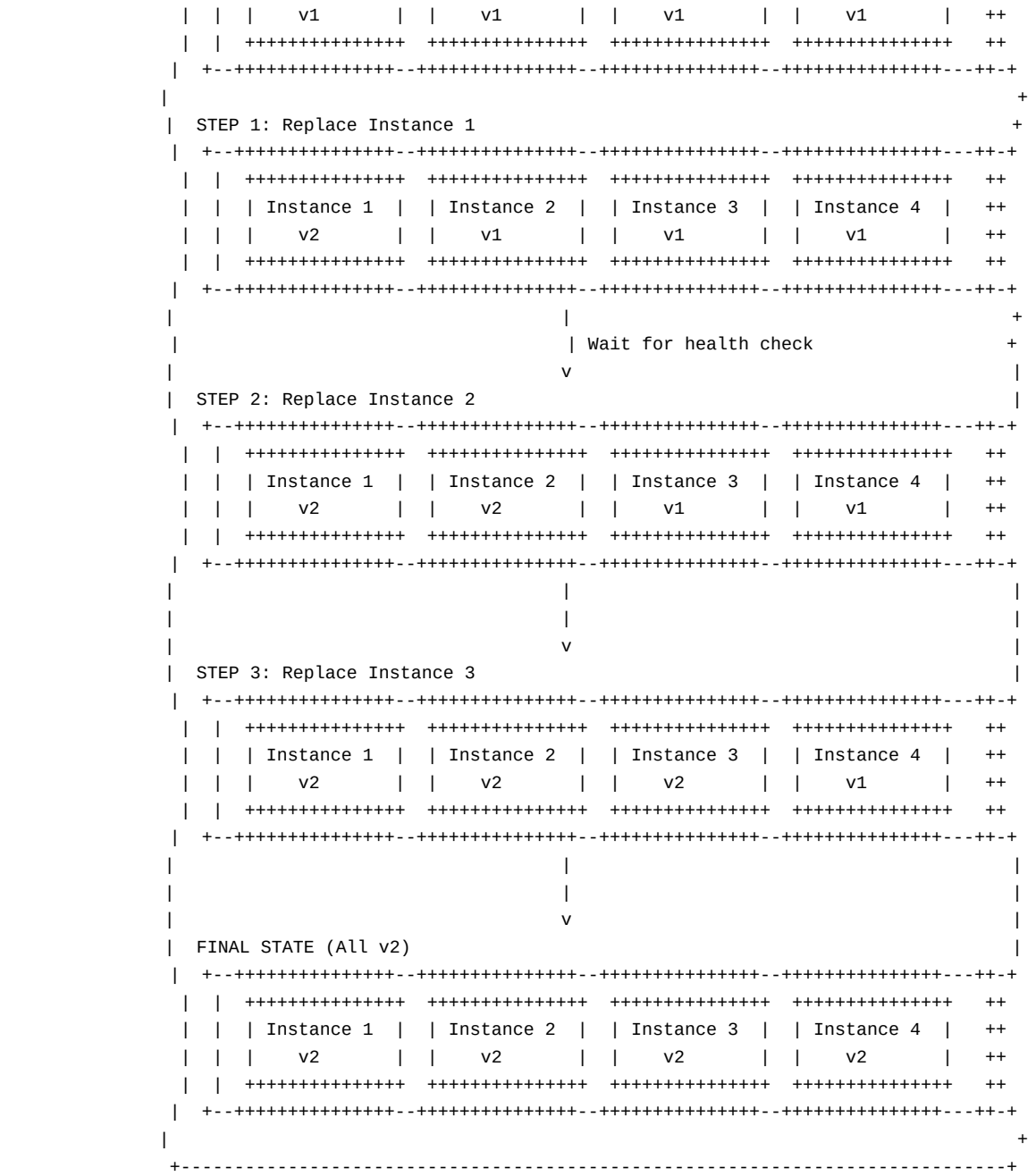
- Blue-green requires double infrastructure
- In-place deployment causes downtime

With Rolling:

- Gradual replacement (batch size: 1-2 instances)
- Zero downtime (if configured correctly)
- No additional infrastructure cost

Diagram: Rolling Deployment





CodeDeploy Rolling Configuration

```
# appspec.yaml
version: 0.0
os: linux
files:
- source: /
destination: /var/www/html
hooks:
ApplicationStop:
    - location: scripts/stop_server.sh
timeout: 300
BeforeInstall:
  - location: scripts/install_dependencies.sh
timeout: 300
AfterInstall:
    - location: scripts/start_server.sh
timeout: 300
```

```
  ApplicationStart:
    - location: scripts/validate_service.sh
      timeout: 300
```

CodeDeploy Deployment Group Configuration

```
resource "aws_codedeploy_deployment_group" "rolling" {
  app_name              = aws_codedeploy_app.myapp.name
  deployment_group_name = "rolling-group"
  service_role_arn      = aws_iam_role.codedeploy_role.arn

  deployment_style {
    deployment_type = "IN_PLACE"
    deployment_option = "WITH_TRAFFIC_CONTROL"
  }

  load_balancer_info {
    target_group_info {
      name = aws_lb_target_group.web.name
    }
  }

  auto_rollback_configuration {
    enabled = true
    events  = ["DEPLOYMENT_FAILURE"]
  }

  deployment_config_name = "CodeDeployDefault.HalfAtATime"

  ec2_tag_set {
    ec2_tag_filter {
      key   = "Name"
      type  = "KEY_AND_VALUE"
      value = "web-server"
    }
  }
}
```

Rolling Deployment Batch Sizes

Batch Size	Description	Use Case
----------	----------	----------
1 instance	Slowest, safest	Critical applications
50%	Balanced	Standard applications
100%	Fastest, riskiest	Non-critical

When to Use Rolling Deployment

Use Case	Why Rolling
----------	----------
Cost-sensitive	No extra infrastructure
Stateful applications	Instances replaced one by one

Use Case	Why Rolling
Slow ramp-up	Gradual exposure to new version
Limited capacity	Can't double infrastructure

Advantages of Rolling Deployment

1. **No extra infrastructure:** Uses existing capacity
2. **Zero downtime:** Gradual replacement
3. **Controlled rollout:** Batch size controls speed
4. **Simple to implement:** Native to many deployment tools

Disadvantages of Rolling Deployment

1. **Mixed versions during deployment:** May cause issues
2. **Slow rollback:** Must roll back batches
3. **No full testing before release:** Some users get new version immediately
4. **Stateful challenges:** State must be compatible

Rolling vs Blue-Green

Aspect	Rolling	Blue-Green
--------	---------	----------
Infrastructure cost	Normal	Double during deployment
Rollback speed	Slow (per batch)	Instant (switch traffic)
Testing before release	Limited	Full testing
Mixed versions	Yes	No

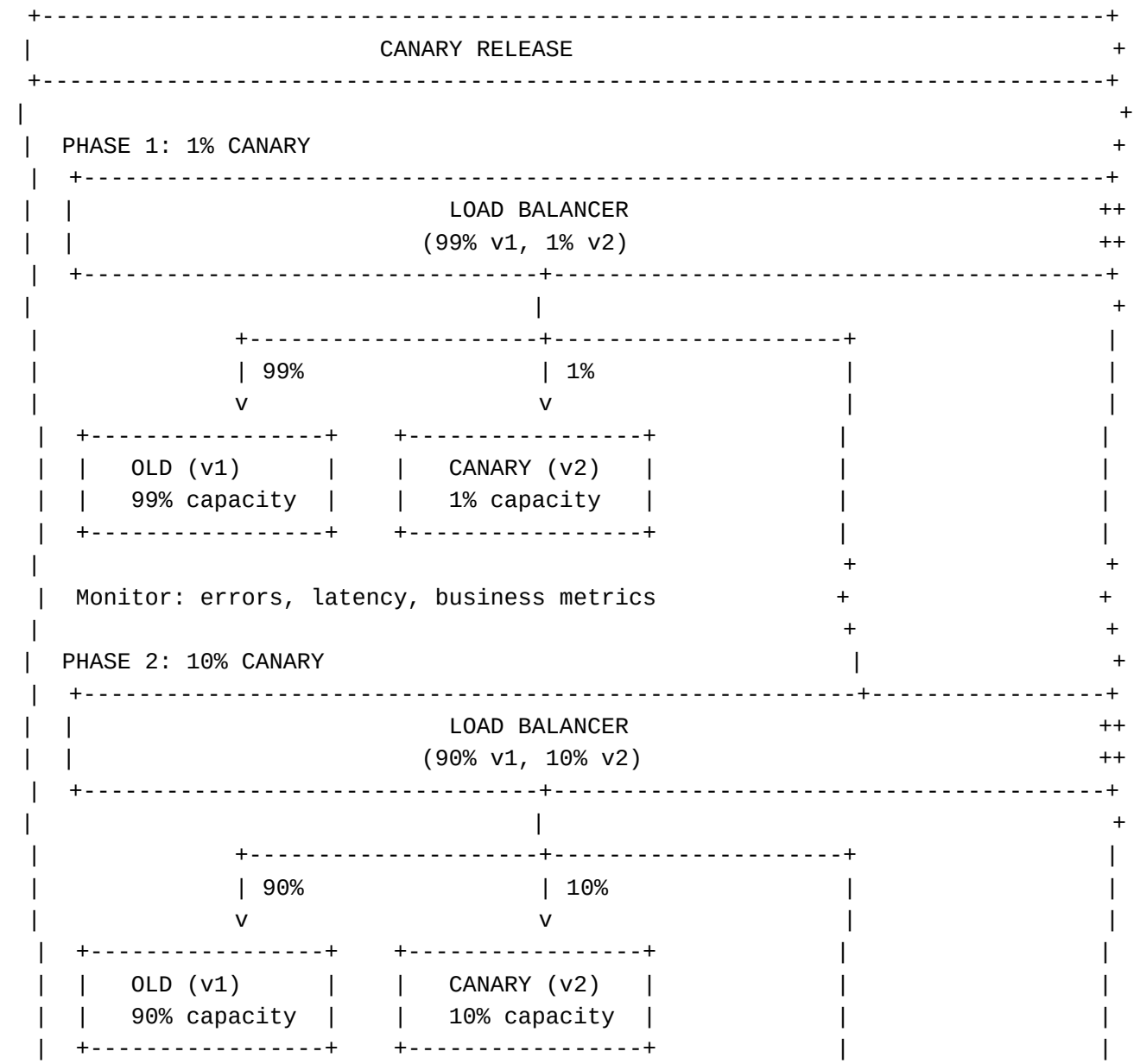
[Page 14]

Chapter 42: Canary Releases

What is Canary Release?

Gradually roll out a new version to a small subset of users first, monitor for issues, then gradually increase the percentage.

Real Scenario
You want to test a new feature with real production traffic before rolling out to all users.
Without Canary:
- Blue-green: all users at once
- Rolling: all users eventually get new version
- No gradual testing with real traffic
With Canary:
- Start with 1% of users
- Monitor metrics (errors, latency)
- Increase to 10%, 25%, 50%, 100%
- Rollback instantly if issues detected
Diagram: Canary Release



```
    |                                                          |                  |
    |  Continue monitoring                                     |                  |
    |                                                          |                  |
   |  PHASE 3: 50% CANARY                                      |                   |
   |  PHASE 4: 100% CANARY                                     |                   |
    |                                                          |                  |
    +----------------------------------------------------------+------------------+
```

Canary with Application Load Balancer (Weighted Target Groups)

```
# Target groups
resource "aws_lb_target_group" "v1" {
name = "app-v1"
port = 80
protocol = "HTTP"
vpc_id = aws_vpc.main.id
}
resource "aws_lb_target_group" "v2" {
name = "app-v2"
port = 80
protocol = "HTTP"
vpc_id = aws_vpc.main.id
}
# Listener with weighted forwarding
resource "aws_lb_listener" "main" {
load_balancer_arn = aws_lb.main.arn
port = 80
protocol = "HTTP"
default_action {
type = "forward"
forward {
                                        target_group {
                                    arn = aws_lb_target_group.v1.arn
weight = 99
}
                                        target_group {
                                    arn = aws_lb_target_group.v2.arn
weight = 1
}
}
}
}
```

Canary Deployment Script

```
#!/bin/bash
# Initial canary (1%)
update_weight() {
local v1_weight=$1
local v2_weight=$2

aws elbv2 modify-listener --listener-arn $LISTENER_ARN \
                                        --default-actions "[
{\"Type\":\"forward\",\"ForwardConfig\":{
\"TargetGroups\":[
                    {\"TargetGroupArn\":\"$V1_TG_ARN\",\"Weight\":$v1_weight},
                     {\"TargetGroupArn\":\"$V2_TG_ARN\",\"Weight\":$v2_weight}
```

```
        ]
      }}
    ]"
}

# Deploy v2 to canary target group
aws ecs update-service --cluster my-cluster --service v2-service --force-new-deployment
wait_for_service_stable "v2-service"

# Start canary with 1%
update_weight 99 1

# Monitor for 5 minutes
sleep 300

# Check metrics (error rate, latency)
ERROR_RATE=$(get_error_rate)
if [ $ERROR_RATE -gt 1 ]; then
  echo "Error rate too high, rolling back"
  update_weight 100 0
  exit 1
fi

# Increase to 10%
update_weight 90 10
sleep 300

# Increase to 50%
update_weight 50 50
sleep 300

# Full rollout
update_weight 0 100
```

Canary with AWS CodeDeploy

```
resource "aws_codedeploy_deployment_group" "canary" {
  app_name              = aws_codedeploy_app.myapp.name
  deployment_group_name = "canary-group"
  service_role_arn      = aws_iam_role.codedeploy_role.arn

  deployment_style {
    deployment_type   = "BLUE_GREEN"
    deployment_option = "WITH_TRAFFIC_CONTROL"
  }

  blue_green_deployment_config {
    deployment_ready_option {
      action_on_timeout = "CONTINUE_DEPLOYMENT"
      wait_time_in_minutes = 5
    }

    terminate_blue_instances_on_deployment_success {
      action                           = "TERMINATE"
      termination_wait_time_in_minutes = 5
    }
  }

  deployment_config_name = "CodeDeployDefault.LambdaCanary10Percent5Minutes"
}
```

When to Use Canary Release

Use Case	Why Canary
----------	----------
High-risk changes	Test with real traffic, small blast radius
Performance-sensitive	Validate performance before full rollout
A/B testing	Compare versions
Gradual feature rollout	Release to increasing user segments

Advantages of Canary Release

1. **Reduced risk:** Small blast radius
2. **Real-world testing:** Test with actual production traffic
3. **Performance validation:** See real performance before full rollout
4. **Gradual rollout:** Can stop at any time
5. **Business metrics:** Track business impact (conversion, revenue)

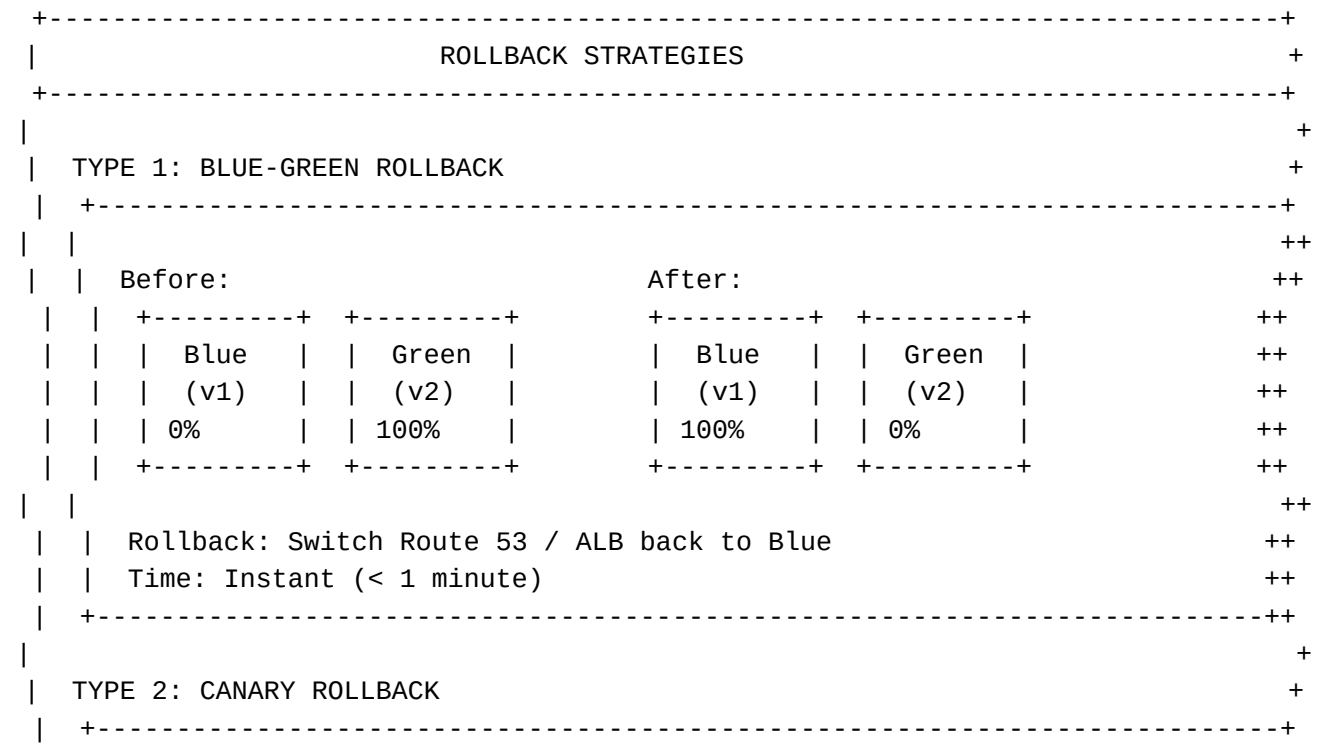
Disadvantages of Canary Release

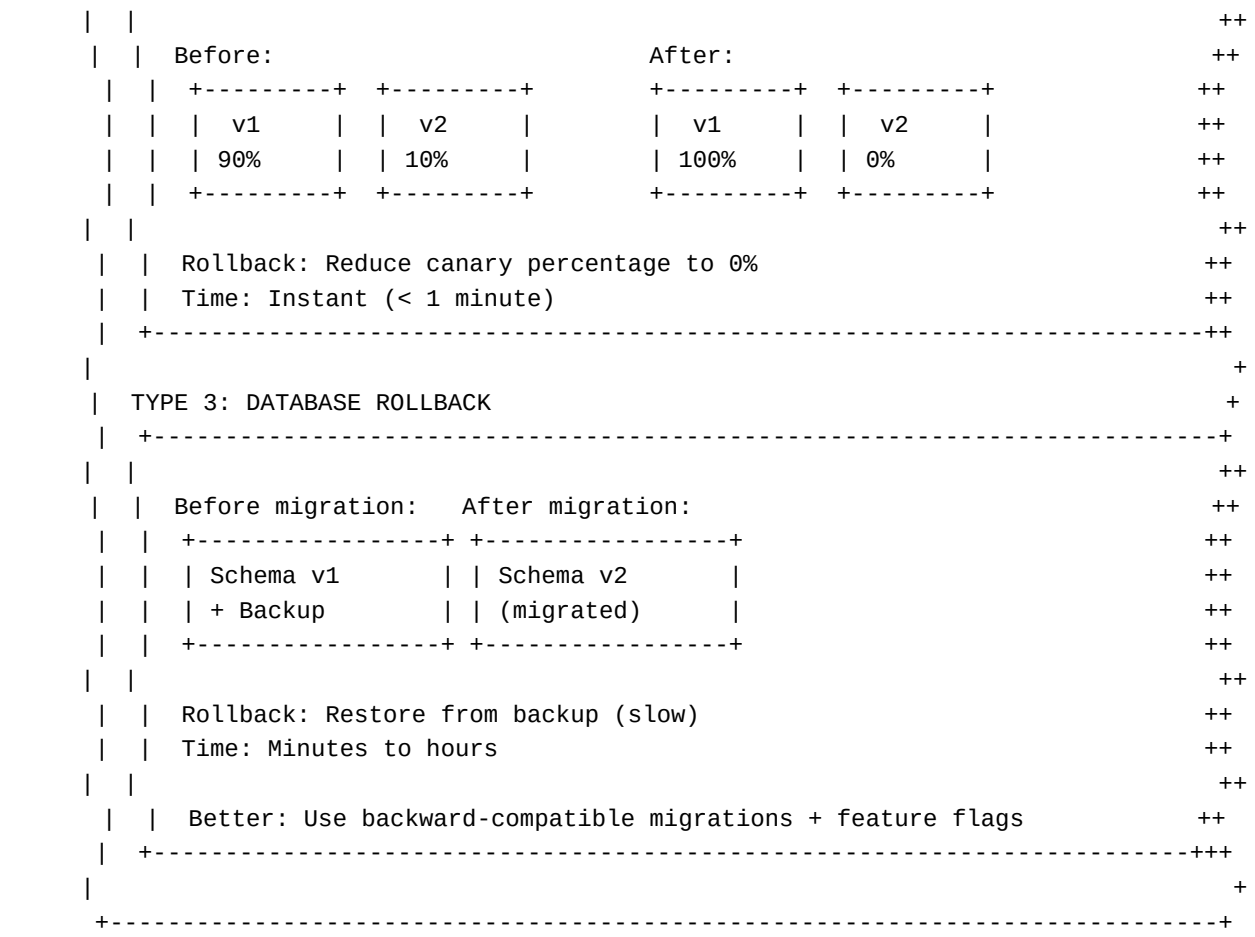
1. **Complexity:** More complex than blue-green
2. **Longer deployment time:** Takes time to increase percentages
3. **Monitoring required:** Need good observability
4. **State management:** Harder with stateful services
5. **User experience:** Some users see new version, some old

Canary vs Blue-Green vs Rolling

Aspect	Canary	Blue-Green	Rolling
--------	--------	----------	---------
Risk	Lowest	Medium	Medium
Rollback speed	Instant	Instant	Slow
Testing before release	Real traffic	Full testing	Limited
Infrastructure cost	Slightly higher	Double	Normal
Deployment time	Long (gradual)	Fast	Medium

Chapter 43: Rollback Strategies
What are Rollback Strategies?
Plans and mechanisms to quickly revert to a known good state when a deployment causes issues.
Real Scenario
You deployed version 2.0, but immediately see error rates spiking. You need to revert to version 1.0 as fast as possible.
Without Rollback Strategy:
- Manual re-deployment (slow, error-prone)
- Extended outage
- Customer impact
With Rollback Strategy:
- Instant or automated rollback
- Minimal customer impact
- Fast recovery
Diagram: Rollback Strategies





Automated Rollback with CodeDeploy

```
resource "aws_codedeploy_deployment_group" "rollback" {
app_name          = aws_codedeploy_app.myapp.name
deployment_group_name = "rollback-group"
auto_rollback_configuration {
enabled = true
events  = [
                                    "DEPLOYMENT_FAILURE",
                                  "DEPLOYMENT_STOP_ON_ALARM",
                                  "DEPLOYMENT_STOP_ON_REQUEST"
]
}
alarm_configuration {
alarms  = ["HighErrorRate", "HighLatency"]
enabled = true
}
}
```

CloudWatch Alarm for Rollback

```
{
"AlarmName": "HighErrorRate",
"MetricName": "Errors",
"Namespace": "AWS/ApplicationELB",
"Statistic": "Sum",
"Period": 60,
"EvaluationPeriods": 3,
"Threshold": 10,
"ComparisonOperator": "GreaterThanThreshold",
"AlarmActions": [
```

```
      "arn:aws:codedeploy:us-east-1:123456789012:deploymentgroup:myapp/rollback-group"
    ]
}
```

Manual Rollback Commands

```
# ECS rollback (deploy previous task definition)
aws ecs update-service \
  --cluster my-cluster \
  --service my-service \
  --task-definition myapp:v1 \
  --force-new-deployment

# Kubernetes rollback
kubectl rollout undo deployment/myapp

# Lambda rollback (publish previous version)
aws lambda update-alias \
  --function-name my-function \
  --name prod \
  --function-version 1

# EC2 (ASG) rollback (update launch template)
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name my-asg \
  --launch-template "LaunchTemplateName=myapp-v1,Version=1"
```

Rollback Best Practices

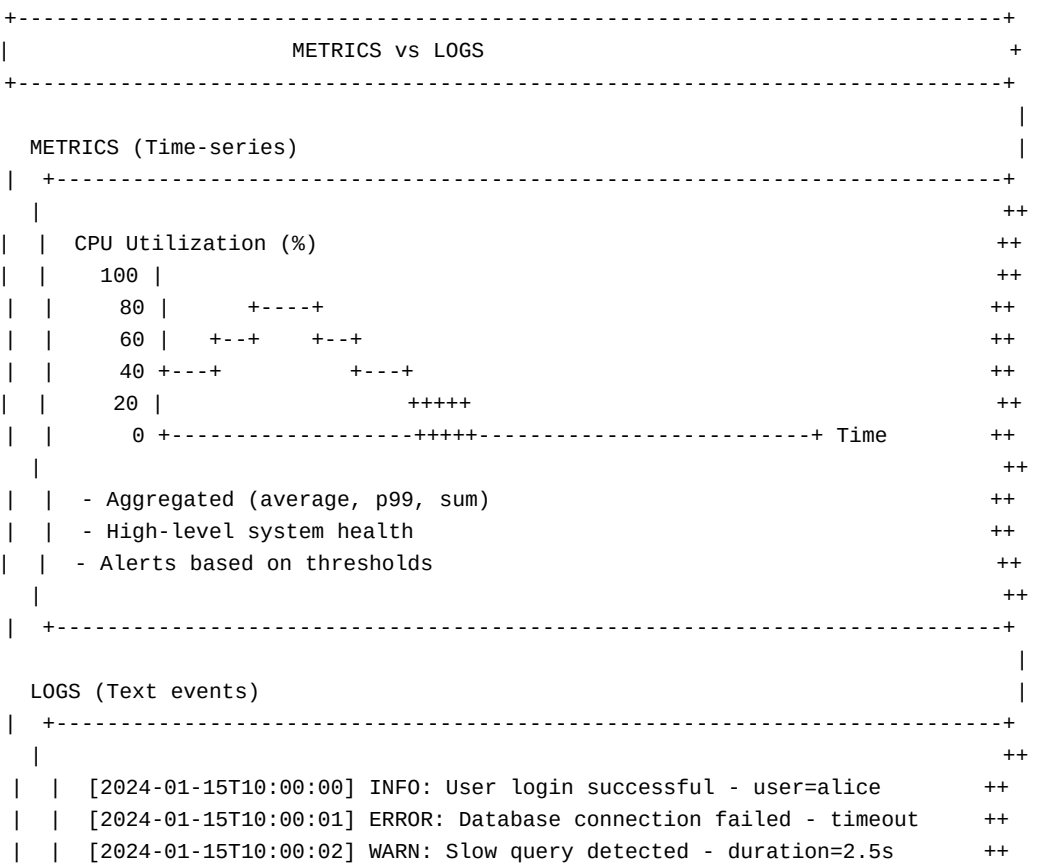
Practice	**Why**
----------	-----
Automate rollback	Faster recovery, less human error
Test rollback regularly	Ensure it works when needed
Database backward compatibility	Enable application rollback
Feature flags	Disable features without rollback
Monitor after rollback	Verify system returns to normal
Document rollback procedures	Enable on-call engineers
Practice disaster recovery	Regularly simulate failures

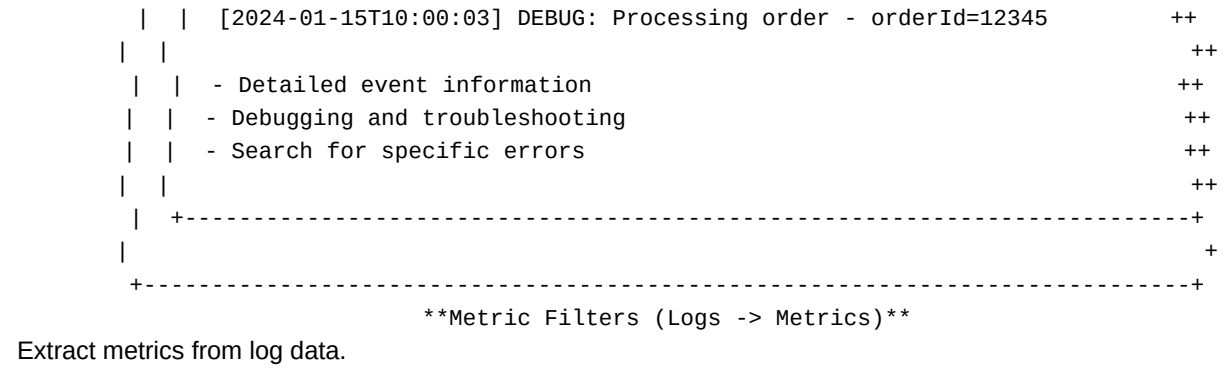
Rollback Strategy Comparison

Strategy	**Speed**	**Complexity**	**Data Risk**	**Use Case**
----------	-------	------------	-----------	----------
Blue-Green	Instant	Low	Low	Stateless apps
Canary	Instant	Medium	Low	High-risk changes
Rolling	Slow (per batch)	Medium	Medium	Cost-sensitive
Database restore	Slow (minutes/hours)	High	High	Last resort

[Page 16]
PART FIFTEEN: OBSERVABILITY &
MONITORING (EXPANDED)
Chapter 44: CloudWatch Logs vs Metrics
What's the Difference?
Aspect
Metrics
Logs

Data type
Numerical (time-series)
Text (unstructured/semi-structured)
Purpose
Performance, capacity
Debugging, audit, troubleshooting
Query
Statistical (avg, sum, max)
Text search, pattern matching
Storage cost
Lower
Higher
Retention
15 months default
Configurable (1 day to 10 years)
Alerts
Yes (metric alarms)
Yes (metric filters -> alarm)
Diagram: Metrics vs Logs





Metric Filters (Logs -> Metrics)

Extract metrics from log data.

```
# Create metric filter for error count
aws logs put-metric-filter \
--log-group-name /aws/lambda/my-function \
--filter-name ErrorCount \
--filter-pattern "ERROR" \
--metric-transformations \
metricName=ErrorCount,metricNamespace=MyApp,metricValue=1
# Create alarm from metric filter
aws cloudwatch put-metric-alarm \
--alarm-name HighErrorRate \
--alarm-description "Error rate > 1%" \
--metric-name ErrorCount \
--namespace MyApp \
--statistic Sum \
--period 60 \
--evaluation-periods 5 \
--threshold 10 \
--comparison-operator GreaterThanThreshold
```

When to Use Metrics vs Logs

Use Case
Metrics
Logs

Performance monitoring

|
|

Capacity planning

|
|

Error debugging

|
|

Security audit

|
|

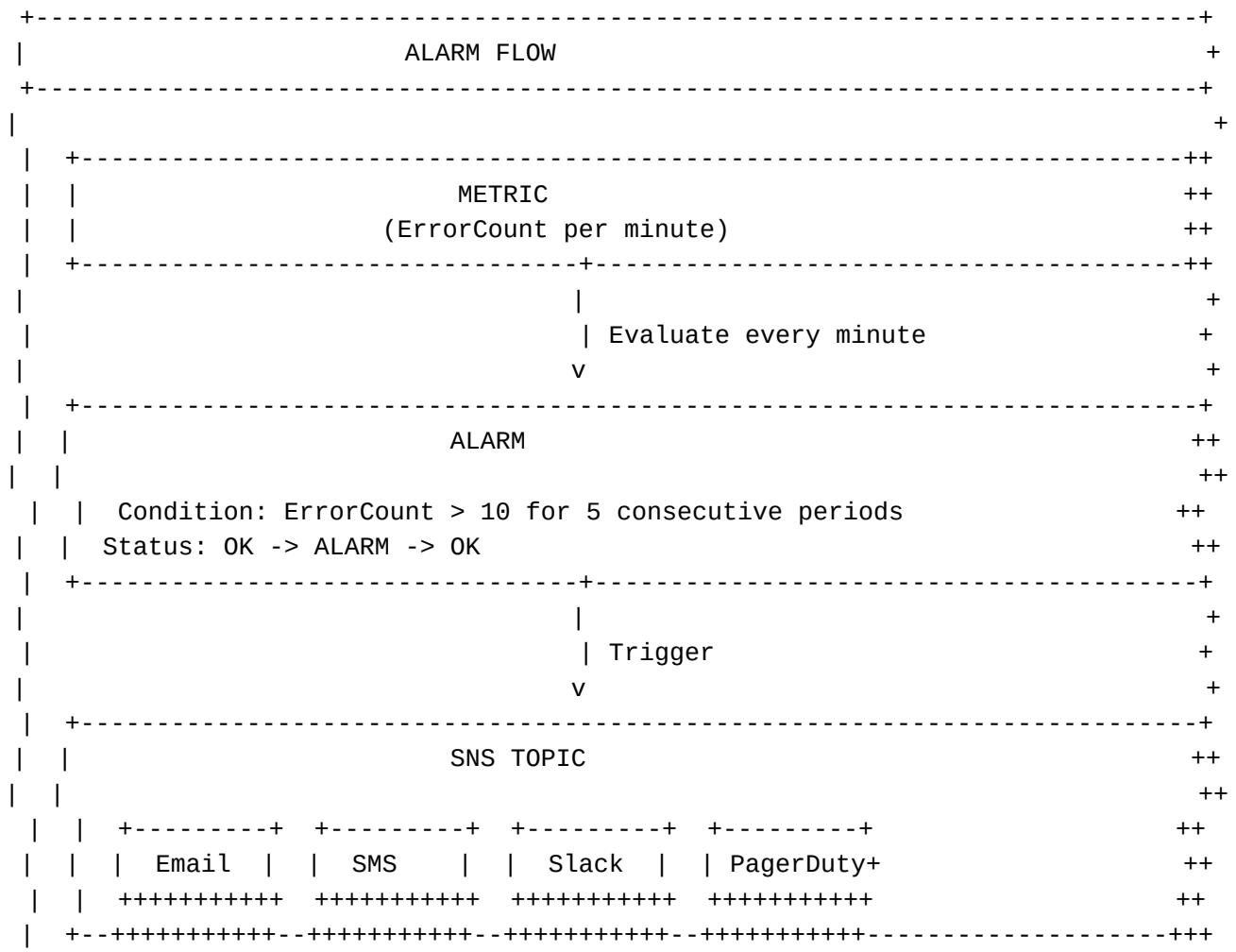
Alerting

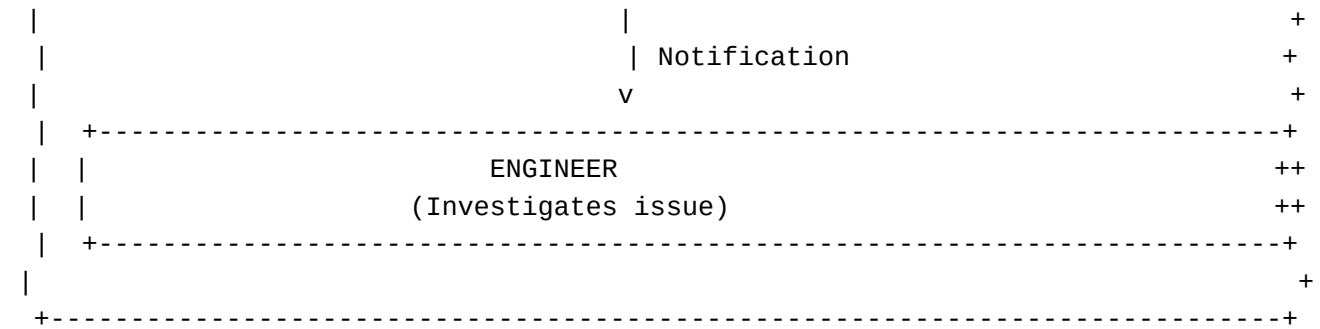
|
| (via filters)

Trend analysis

|
|

Chapter 45: Alarms & Alerting
What are CloudWatch Alarms?
Alarms monitor metrics and trigger actions when thresholds are crossed.
Real Scenario
Your application's error rate suddenly spikes at 3 AM. You want to be notified immediately.
Without Alarms:
- Users report issues
- Manual detection
- Delayed response
With Alarms:
- Automatic detection
- Page on-call engineer
- Fast response
Diagram: Alarm Flow





Alarm Configuration Example

```
{
"AlarmName": "HighCPUUtilization",
"AlarmDescription": "CPU > 80% for 10 minutes",
"MetricName": "CPUUtilization",
"Namespace": "AWS/EC2",
"Statistic": "Average",
"Period": 300,
"EvaluationPeriods": 2,
"Threshold": 80,
"ComparisonOperator": "GreaterThanThreshold",
"AlarmActions": ["arn:aws:sns:us-east-1:123456789012:alerts"],
"OKActions": ["arn:aws:sns:us-east-1:123456789012:alerts"],
"InsufficientDataActions": [],
"Dimensions": [
{
"Name": "InstanceId",
                                "Value": "i-1234567890abcdef0"
}
]
}
```

Composite Alarms (AND/OR logic)

Combine multiple alarms into a single alarm.

```
resource "aws_cloudwatch_composite_alarm" "system_health" {
alarm_name        = "SystemHealth"
alarm_description = "System is degraded"
alarm_rule = "(ALARM(HighCPU) AND ALARM(HighMemory)) OR ALARM(ServiceDown)"
}
```

Alarm States

State
Description

OK
Metric within threshold
ALARM
Metric crossed threshold
INSUFFICIENT_DATA
Not enough data to evaluate

Alerting Best Practices

Practice	Why
----------	-----
Actionable alerts	Every alert should lead to action
Avoid alert fatigue	Only alert on important issues
Use multiple evaluation periods	Reduce false alarms
Set appropriate thresholds	Based on normal behavior
Test alarms	Simulate failures
Document runbooks	What to do when alarmed
On-call rotation	Ensure coverage

[Page 18]

Chapter 46: AWS X-Ray (Distributed Tracing)

What is AWS X-Ray?

Distributed tracing service that helps debug and analyze microservices applications by tracking requests as they travel through your system.

Real Scenario

Your microservices application has 10+ services. A user request is slow, but you don't know which service is causing the delay.

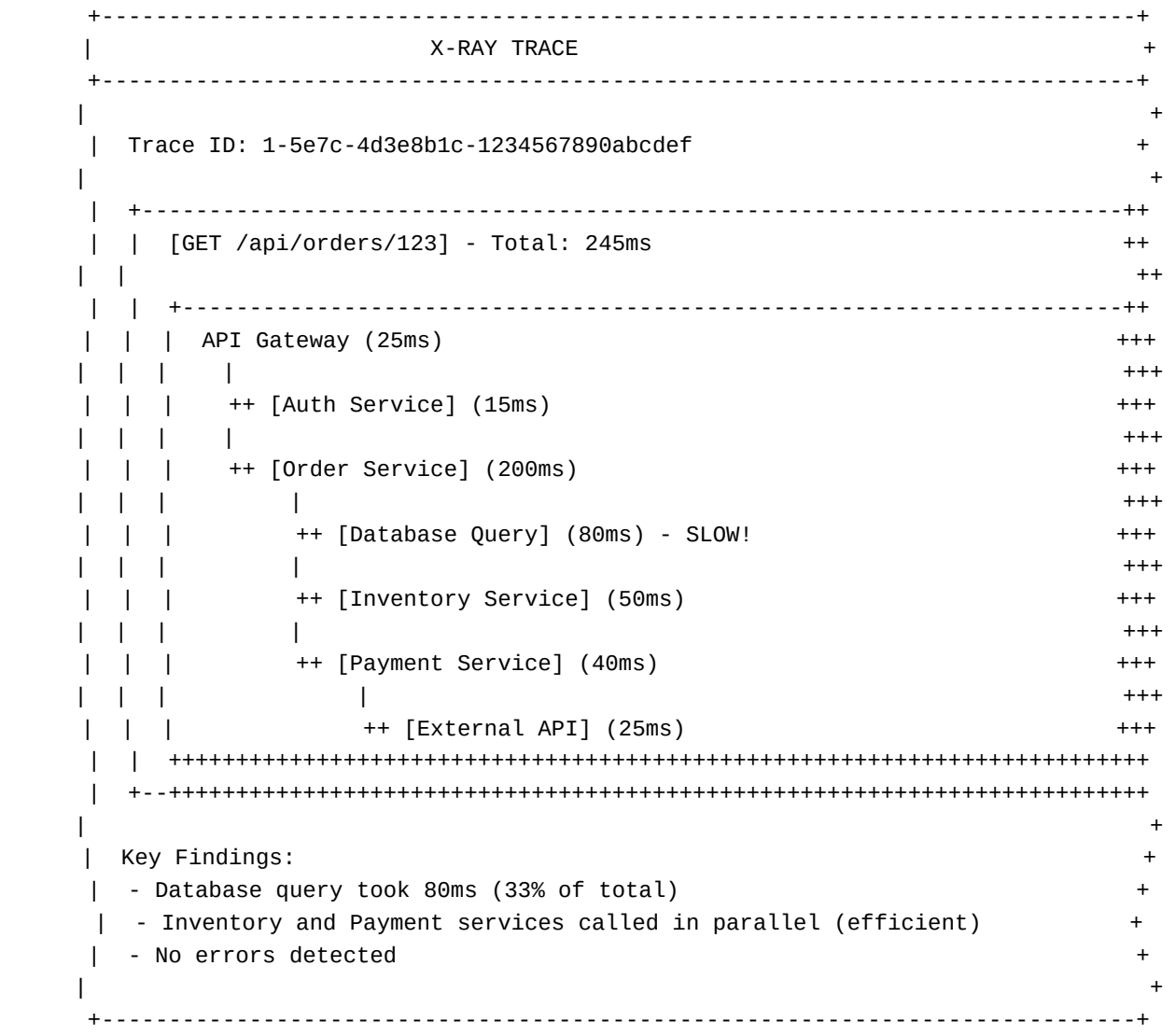
Without X-Ray:

- Check logs from each service (time-consuming)
- Hard to correlate across services
- Can't see full request path

With X-Ray:

- Single trace shows entire request path
- Identify slowest service (bottleneck)
- See where errors occur

Diagram: X-Ray Trace



X-Ray Integration (Lambda + API Gateway)

```
# CloudFormation for X-Ray enabled Lambda
MyFunction:
Type: AWS::Lambda::Function
Properties:
Handler: index.handler
Runtime: nodejs18.x
TracingConfig:
Mode: Active
# API Gateway with X-Ray tracing
MyApi:
Type: AWS::ApiGateway::RestApi
Properties:
                                        Name: my-api
TracingEnabled: true
```

X-Ray SDK Instrumentation (Node.js)

```
const AWSXRay = require('aws-xray-sdk-core');
const AWS = AWSXRay.captureAWS(require('aws-sdk'));
const express = require('express');
const app = express();
// Capture all incoming requests
```

```
app.use(AWSXRay.express.openSegment('my-app'));

app.get('/api/orders/:id', async (req, res) => {
  // Subsegment for database query
  const subsegment = AWSXRay.getSegment().addNewSubsegment('DatabaseQuery');
  try {
    const result = await db.query('SELECT * FROM orders WHERE id = ?', [req.params.id]);
    subsegment.close();
    res.json(result);
  } catch (err) {
    subsegment.close(err);
    res.status(500).json({ error: err.message });
  }
});

app.use(AWSXRay.express.closeSegment());
```

X-Ray Sampling Rules

Control how many traces are recorded.

```
{
  "SamplingRule": {
    "RuleName": "HighTrafficRule",
    "ResourceARN": "*",
    "Priority": 10,
    "FixedRate": 0.05,
    "ReservoirSize": 50,
    "ServiceName": "my-service",
    "ServiceType": "AWS::EC2::Instance",
    "Host": "*",
    "HTTPMethod": "*",
    "URLPath": "*"
  }
}
```

When to Use X-Ray

Use Case	**Why X-Ray**
----------	----------
Microservices debugging	Trace across service boundaries
Performance bottlenecks	Identify slowest component
Error localization	Find where errors occur
Dependency mapping	Understand service dependencies
Latency analysis	Breakdown of request time

Advantages of X-Ray

1. **AWS integrated:** Works with API Gateway, Lambda, ELB, SQS
2. **Low overhead:** Sampling reduces cost

3. **Service maps:** Visualize dependencies

4. **Error analysis:** See which service failed

5. **Performance insights:** Identify bottlenecks

Disadvantages of X-Ray

1. **AWS only:** Not multi-cloud

2. **Sampling:** Not all requests traced

3. **Storage:** Limited retention (30 days)

4. **Cost:** Can be expensive at high volume

[Page 19]

Chapter 47: Centralized Logging Architecture

What is Centralized Logging?

Collecting logs from all services and instances into a single location for searching, analysis, and alerting.

Real Scenario

Your application runs on 50 EC2 instances, 20 Lambda functions, and 10 containers. When debugging an issue, you need to search across all of them.

Without Centralized Logging:

- SSH into each instance

- grep log files manually

- Hard to correlate across services

- No historical analysis

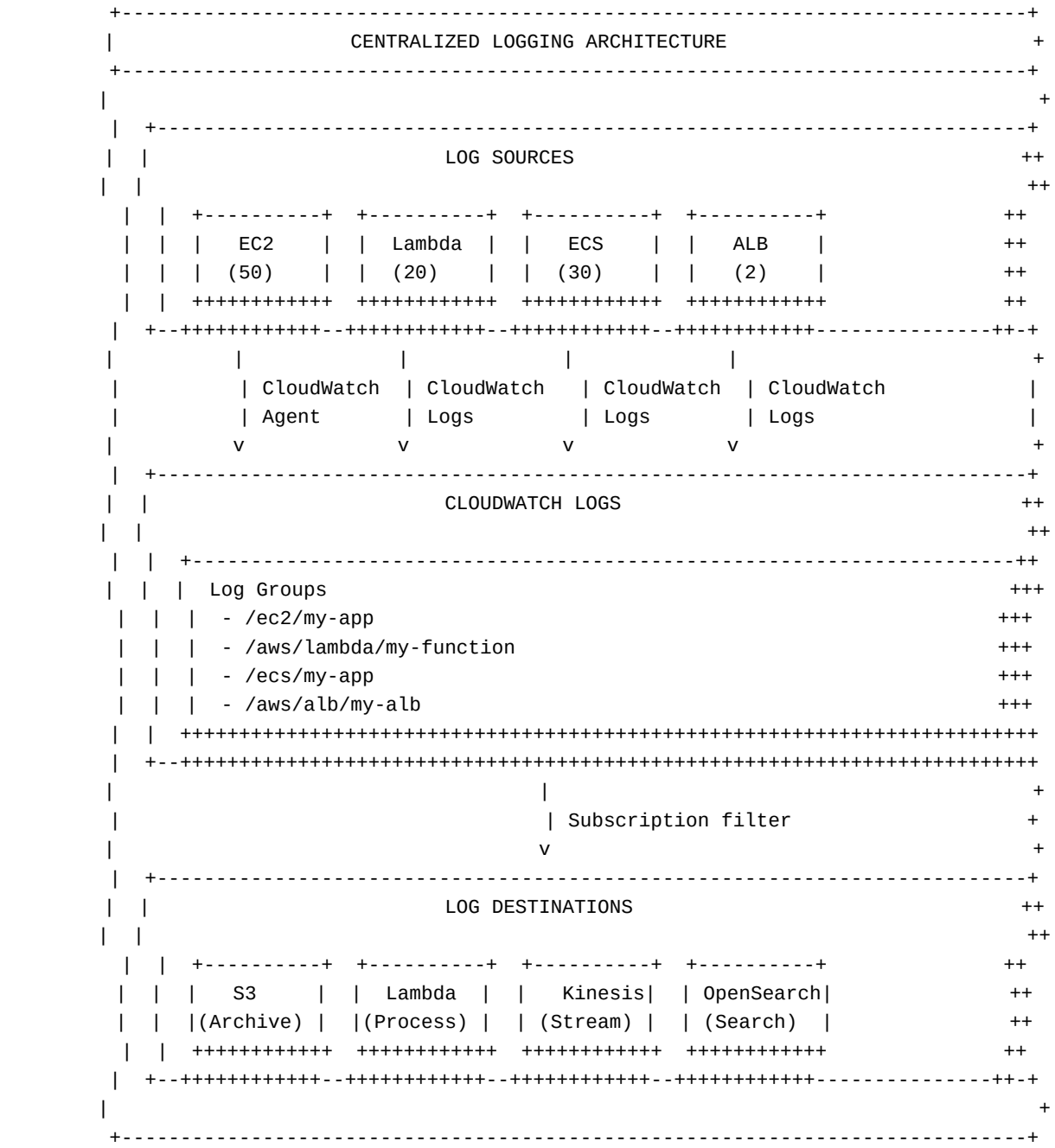
With Centralized Logging:

- All logs in one place

- Search across all services

- Correlate by trace ID
- Retain for months

Diagram: Centralized Logging Architecture



CloudWatch Agent Configuration (/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json)

```
{
"logs": {
                                        "logs_collected": {
"files": {
                                           "collect_list": [
{
                                    "file_path": "/var/log/nginx/access.log",
                                     "log_group_name": "/ec2/my-app/nginx",
                                      "log_stream_name": "{instance_id}",
"timezone": "UTC"
},
```

```
            {
              "file_path": "/var/log/application/app.log",
              "log_group_name": "/ec2/my-app/application",
              "log_stream_name": "{instance_id}"
            }
          ]
        }
      },
      "log_stream_name": "my-app-logs"
    },
    "metrics": {
      "metrics_collected": {
        "cpu": {
          "measurement": ["cpu_usage_idle", "cpu_usage_user", "cpu_usage_system"]
        },
        "disk": {
          "measurement": ["used_percent", "inodes_free"],
          "resources": ["/"]
        },
        "mem": {
          "measurement": ["mem_used_percent"]
        }
      }
    }
  }
```

Centralized Logging with OpenSearch (ELK Stack)

```
# CloudFormation for OpenSearch (Elasticsearch)
OpenSearchDomain:
  Type: AWS::OpenSearchService::Domain
  Properties:
    DomainName: my-logs-domain
    EngineVersion: OpenSearch_2.11
    ClusterConfig:
      InstanceType: r6g.large.search
      InstanceCount: 3
    EBSOptions:
      EBSEnabled: true
      VolumeSize: 100
      VolumeType: gp3

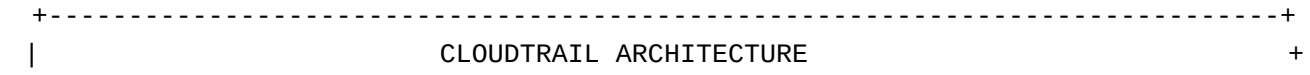
# Subscription filter to send logs to OpenSearch
LogSubscription:
  Type: AWS::Logs::SubscriptionFilter
  Properties:
    LogGroupName: /aws/lambda/my-function
    FilterPattern: ""
    DestinationArn: !GetAtt OpenSearchDomain.Arn
```

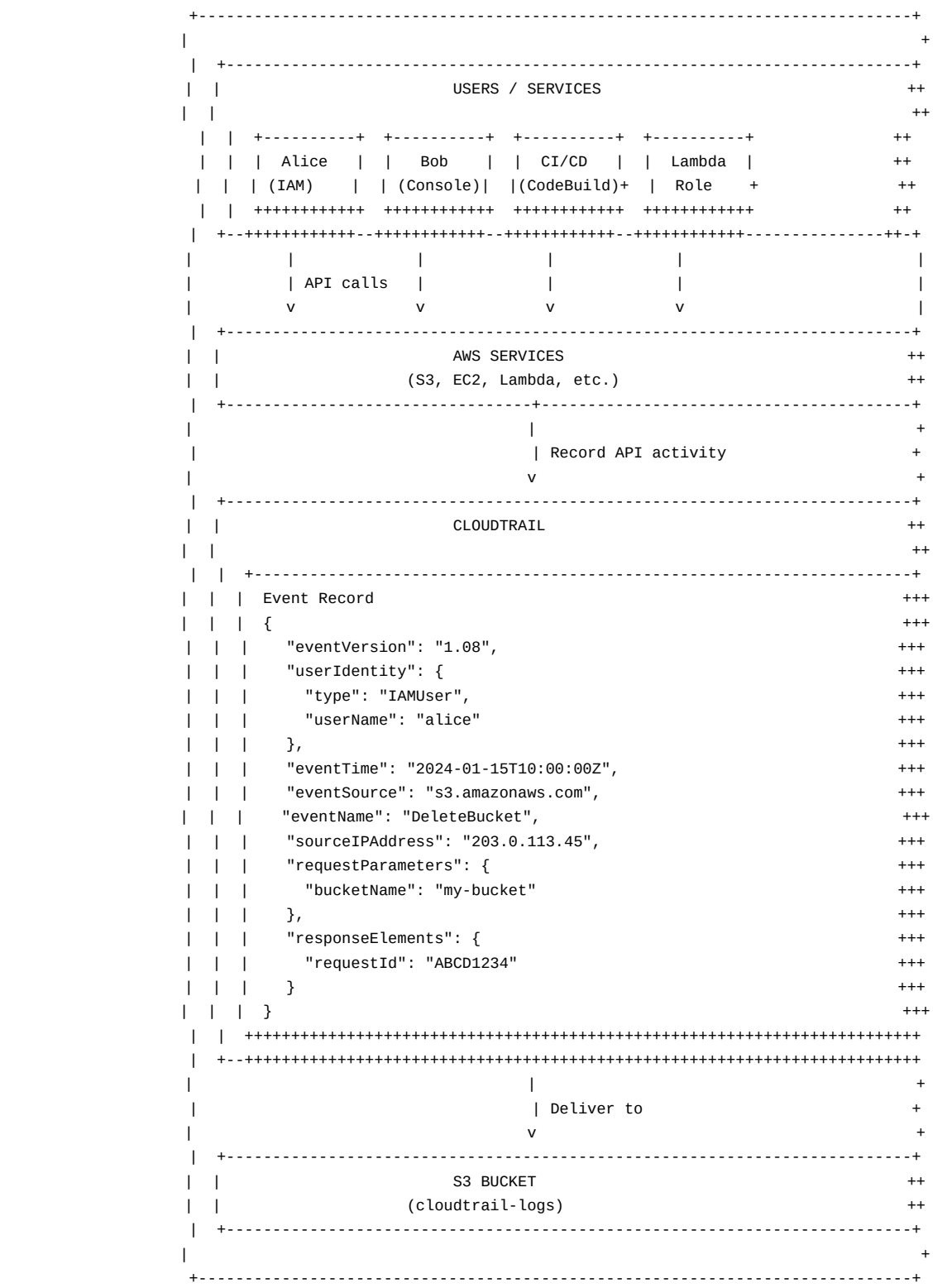
Centralized Logging Best Practices

Practice	**Why**
----------	-----
Use structured logging (JSON)	Easier to search and parse
Include correlation IDs	Trace requests across services
Set appropriate retention	Balance cost and compliance

Practice
Why
Use log levels
DEBUG, INFO, WARN, ERROR
Centralize all logs
One place to search
Monitor log volume
Avoid cost surprises
Archive old logs to S3
Reduce CloudWatch costs

[Page 20]
PART SIXTEEN: AUDITING & COMPLIANCE
Chapter 48: CloudTrail (API Auditing)
What is CloudTrail?
Records API activity in your AWS account, providing an audit trail of who did what, when, and from where.
Real Scenario
You need to know who deleted an S3 bucket and when. Security team asks for an audit log.
Without CloudTrail:
- No record of API calls
- Can't determine who made changes
- Security investigations impossible
With CloudTrail:
- Every API call recorded
- User identity, source IP, timestamp
- Full audit trail
Diagram: CloudTrail Architecture





CloudTrail Event Example

{
"eventVersion": "1.08",
"userIdentity": {
"type": "AssumedRole",
"principalId": "ARO1234567890:my-role",
"arn": "arn:aws:sts::123456789012:assumed-role/MyRole/session",
"accountId": "123456789012",

```
      "accessKeyId": "AKIAIOSFODNN7EXAMPLE",
      "sessionContext": {
        "sessionIssuer": {
          "type": "Role",
          "userName": "MyRole"
        }
      }
    },
    "eventTime": "2024-01-15T10:00:00Z",
    "eventSource": "ec2.amazonaws.com",
    "eventName": "RunInstances",
    "awsRegion": "us-east-1",
    "sourceIPAddress": "203.0.113.45",
    "userAgent": "aws-cli/2.0",
    "requestParameters": {
      "instanceType": "t3.micro",
      "imageId": "ami-0c55b159cbfafe1f0",
      "minCount": 1,
      "maxCount": 1
    },
    "responseElements": {
      "instancesSet": {
        "items": [
          {
            "instanceId": "i-1234567890abcdef0"
          }
        ]
      }
    }
  }
```

CloudTrail Trail Configuration

```
resource "aws_cloudtrail" "main" {
  name                          = "my-trail"
  s3_bucket_name                = aws_s3_bucket.cloudtrail.id
  enable_log_file_validation    = true
  include_global_service_events = true
  is_multi_region_trail         = true

  event_selector {
    read_write_type           = "All"
    include_management_events = true

    data_resource {
      type   = "AWS::S3::Object"
      values = ["arn:aws:s3:::my-bucket/"]
    }
  }
}
```

CloudTrail vs CloudWatch

Aspect	**CloudTrail**	**CloudWatch**
--------	----------	----------
What it logs	API calls (who did what)	Application logs, metrics
Purpose	Auditing, security	Monitoring, debugging
Data retention	Configurable (S3)	Configurable (1 day - 10 years)

Aspect	CloudTrail	CloudWatch
Alerting	Via EventBridge	Native alarms

When to Use CloudTrail

Use Case	Why CloudTrail
----------	----------
Security audit	Track who made changes
Compliance (SOC2, HIPAA)	Required for audit trails
Incident investigation	Determine what changed
Troubleshooting	See API errors
Cost tracking	Identify who launched resources

CloudTrail Best Practices

Practice	Why
----------	-----
Enable multi-region	Capture global events (IAM, Route53)
Send to S3	Long-term retention
Enable log validation	Detect tampering
Set up CloudWatch Logs	Real-time monitoring
Create EventBridge rules	Alert on suspicious activity
Encrypt logs	Protect sensitive data

[Page 21]

Chapter 49: AWS Config (Resource Compliance & Drift Detection)

What is AWS Config?

Service that tracks configuration changes to AWS resources and evaluates them against desired policies.

Real Scenario

You need to ensure all S3 buckets are private and all EC2 instances are using approved AMIs.

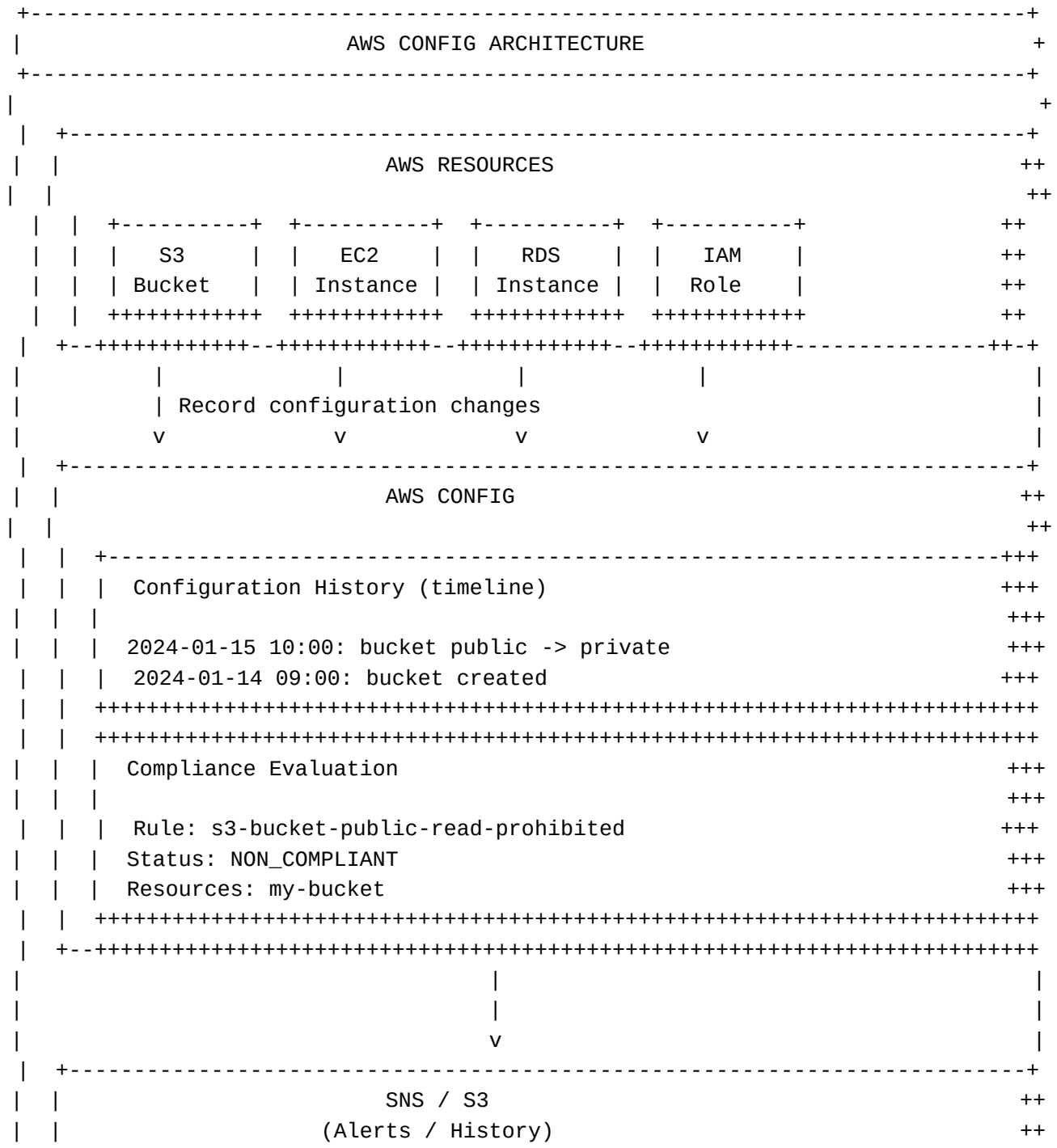
Without AWS Config:

- Manual checks (error-prone)
- Drift goes undetected
- Compliance violations missed

With AWS Config:

- Automatic resource tracking
- Continuous compliance evaluation
- Drift detection and alerts

Diagram: AWS Config Architecture



```
 |  +-------------------------------------------------------------------------+
|                                                                             +
 +-----------------------------------------------------------------------------+
```

AWS Config Rules (Managed)

```
# S3 bucket public access prohibited
resource "aws_config_config_rule" "s3_public_read" {
name = "s3-bucket-public-read-prohibited"
source {
owner           = "AWS"
                source_identifier = "S3_BUCKET_PUBLIC_READ_PROHIBITED"
}
scope {
                    compliance_resource_types = ["AWS::S3::Bucket"]
}
}
# EC2 instance no public IP
resource "aws_config_config_rule" "ec2_no_public_ip" {
name = "ec2-instance-no-public-ip"
source {
owner           = "AWS"
                    source_identifier = "EC2_INSTANCE_NO_PUBLIC_IP"
}
}
# RDS backup retention (custom rule)
resource "aws_config_config_rule" "rds_backup_retention" {
name = "rds-backup-retention-check"
source {
owner           = "AWS"
                   source_identifier = "RDS_BACKUP_RETENTION_CHECK"
}
}
```

Custom Config Rule (Lambda)

```
exports.handler = async (event, context) => {
const evaluatingControls = event.configRuleInvokingEvent;
const invokingEvent = JSON.parse(event.invokingEvent);
const configurationItem = invokingEvent.configurationItem;

// Check if resource is compliant
let complianceType = 'COMPLIANT';

if (configurationItem.resourceType === 'AWS::EC2::Instance') {
const instanceType = configurationItem.configuration.instanceType;

// Only allow approved instance types
const approvedTypes = ['t3.micro', 't3.small', 'm5.large'];

if (!approvedTypes.includes(instanceType)) {
                             complianceType = 'NON_COMPLIANT';
}
}

// Return evaluation result
await putEvaluations({
                            EvaluationType: 'CUSTOM_POLICY',
```

```
      ResultToken: event.resultToken,
      Evaluations: [{
        ComplianceResourceType: configurationItem.resourceType,
        ComplianceResourceId: configurationItem.resourceId,
        ComplianceType: complianceType,
        OrderingTimestamp: configurationItem.configurationItemCaptureTime
      }]
    });
};
```

Config Aggregator (Multi-account, Multi-region)

```
resource "aws_config_configuration_aggregator" "organization" {
  name = "organization-aggregator"

  organization_aggregation_source {
    all_regions = true
    role_arn    = aws_iam_role.config_aggregator.arn
  }
}
```

When to Use AWS Config

Use Case	**Why Config**
----------	----------
Compliance monitoring	Detect violations automatically
Drift detection	Identify manual changes
Security baseline	Enforce security policies
Troubleshooting	See configuration history
Audit preparation	Export compliance reports

Advantages of AWS Config

1. **Automatic discovery:** Discovers resources automatically
2. **Historical tracking:** See configuration timeline
3. **Compliance rules:** 100+ managed rules
4. **Custom rules:** Lambda-based rules
5. **Multi-account:** Aggregate across accounts

Disadvantages of AWS Config

1. **Cost:** Per resource evaluation (adds up)
2. **Latency:** Not real-time (15-60 minutes)
3. **Regional:** Must enable per region

4. **Complexity:** Custom rules require Lambda

Config vs CloudTrail

Aspect	Config	CloudTrail
--------	--------	----------
Purpose	Resource state	API activity
What it tracks	Configuration changes	Who made changes
History	Configuration timeline	API call timeline
Compliance	Rules evaluation	Not applicable
Use together	Config shows state, CloudTrail shows who changed it	

[Page 22]

PART SEVENTEEN: THREAT DETECTION & SECURITY MONITORING

Chapter 50: GuardDuty

What is GuardDuty?

Intelligent threat detection service that continuously monitors for malicious activity and unauthorized behavior using machine learning and anomaly detection.

Real Scenario

You need to detect if an EC2 instance has been compromised and is communicating with a known malicious IP address.

Without GuardDuty:

- Manual log analysis (too slow)

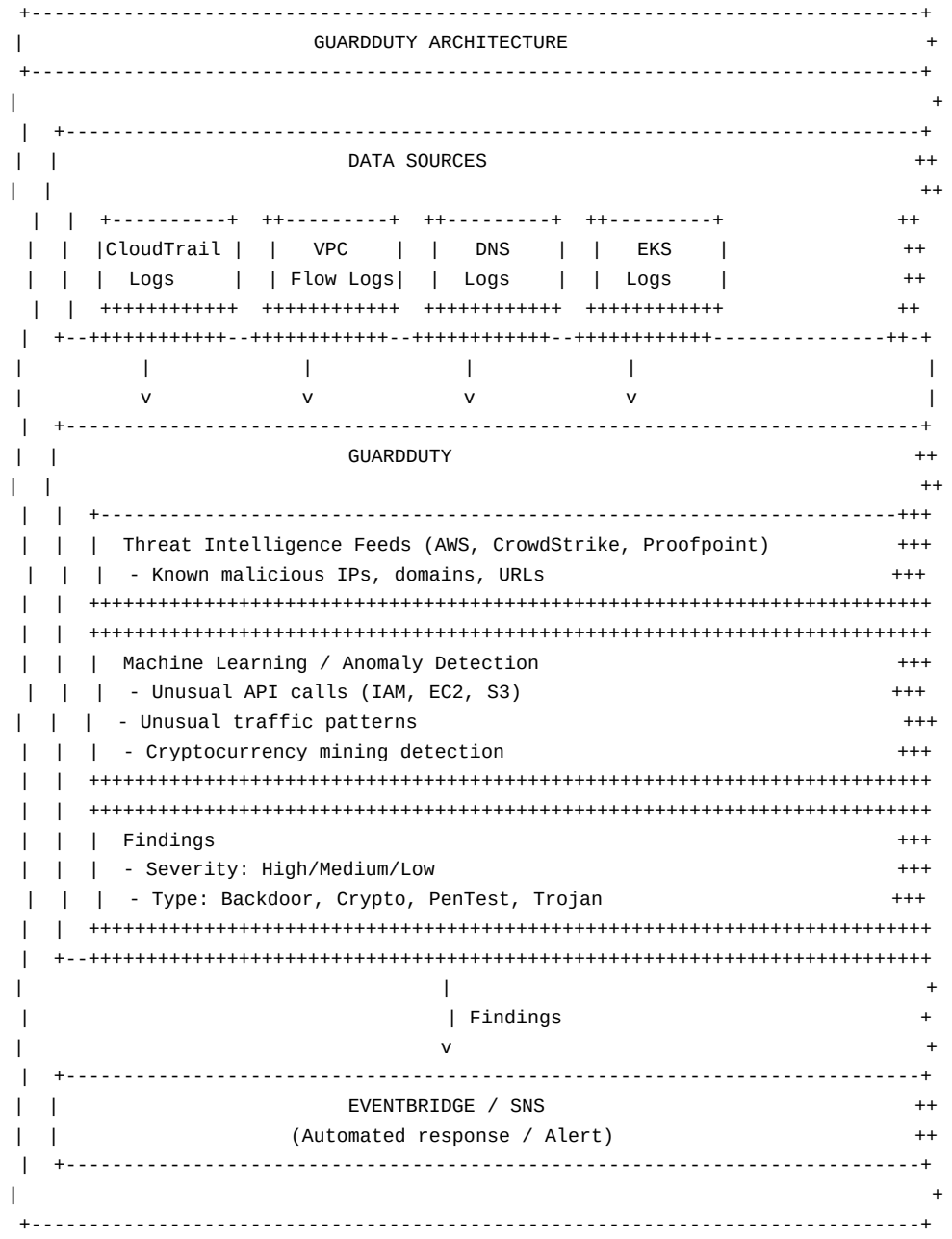
- No automated detection

- Breach goes unnoticed for days/weeks

With GuardDuty:

- Continuous monitoring
- ML-based detection
- Automated alerts

Diagram: GuardDuty Architecture



GuardDuty Finding Types

Category
Example Finding
Severity

Backdoor
EC2 instance communicating with known C2 server
High
CryptoCurrency
Instance mining cryptocurrency
High

Category	Example Finding	Severity
PenTest	Port scanning, brute force attempts	Medium
Trojan	Malicious software detected	High
IAM	Unusual API calls from new IP	Medium
S3	Unusual data access pattern	Medium

GuardDuty Finding Example

```
{
  "type": "Backdoor:EC2/C&CActivity.B",
  "severity": 8,
  "title": "EC2 instance i-1234567890abcdef0 is communicating with a known command and control
  "description": "Your EC2 instance is communicating with an IP address that is associated with
  "resource": {
    "instanceDetails": {
      "instanceId": "i-1234567890abcdef0",
      "instanceType": "t3.micro",
      "launchTime": "2024-01-15T10:00:00Z",
      "tags": [
        { "key": "Name", "value": "web-server" }
      ]
    }
  },
  "service": {
    "count": 5,
    "eventFirstSeen": "2024-01-15T10:00:00Z",
    "eventLastSeen": "2024-01-15T10:05:00Z",
    "detectorId": "detector-1234567890"
  }
}
```

Automated Response (EventBridge)

```
resource "aws_cloudwatch_event_rule" "guardduty_high_severity" {
  name        = "guardduty-high-severity"
  description = "Capture high severity GuardDuty findings"

  event_pattern = jsonencode({
    source = ["aws.guardduty"]
    detail-type = ["GuardDuty Finding"]
    detail = {
      severity = [{ numeric: [">=", 7] }]
    }
  })
}

resource "aws_cloudwatch_event_target" "isolate_instance" {
  rule      = aws_cloudwatch_event_rule.guardduty_high_severity.name
  target_id = "IsolateInstance"
  arn       = aws_lambda_function.isolate_instance.arn
}
```

GuardDuty vs Security Hub

Aspect	GuardDuty	Security Hub
--------	----------	----------
Purpose	Threat detection	Centralized security view
Sources	AWS logs (CloudTrail, VPC)	Multiple (GuardDuty, Inspector, Config)
Findings	Threats, anomalies	Compliance, vulnerabilities, threats
Automation	EventBridge for response	Custom actions

When to Use GuardDuty

Use Case	Why GuardDuty
----------	----------
Threat detection	ML-based detection
Compromised EC2	Detect C2 communication
Unauthorized access	Unusual API calls
Crypto mining	Detect mining activity
Security monitoring	24/7 automated

Advantages of GuardDuty

1. **Automated:** No manual configuration
2. **ML-based:** Detects unknown threats
3. **Integrated:** Works with EventBridge, Security Hub
4. **Low cost:** $1-5 per month for small accounts
5. **No performance impact:** Passive monitoring

Disadvantages of GuardDuty

1. **AWS only:** Not multi-cloud
2. **False positives:** Can generate noise
3. **30-day free trial:** Then paid
4. **No remediation:** Only detection

Chapter 51: Security Hub

What is Security Hub?

Centralized security and compliance dashboard that aggregates findings from multiple AWS services and partner tools.

Real Scenario

You use GuardDuty, Inspector, and Config for security. You need a single place to see all security findings and compliance status.

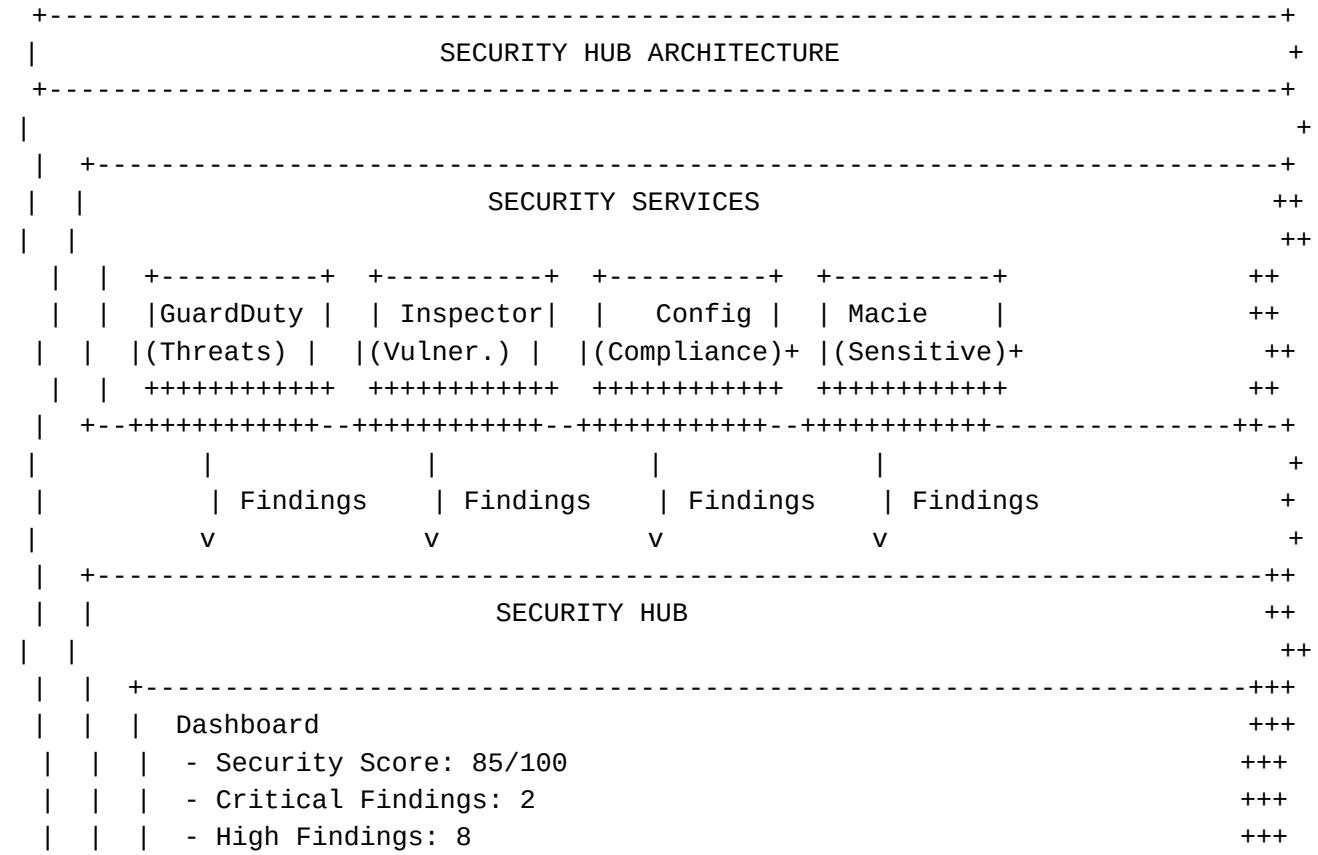
Without Security Hub:

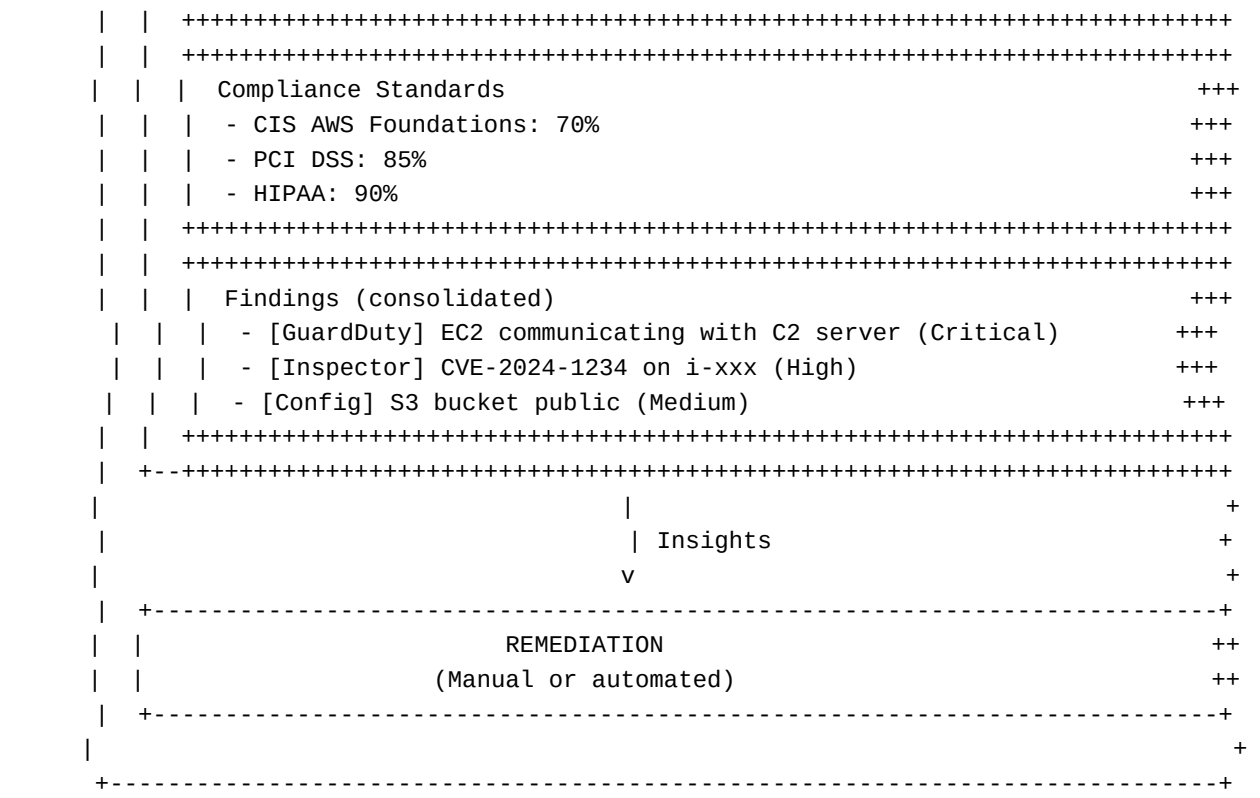
- Check multiple consoles
- Missed findings
- No centralized view

With Security Hub:

- Single dashboard
- Consolidated findings
- Compliance scores

Diagram: Security Hub Architecture





Security Hub Standards
Standard
Description

CIS AWS Foundations
Security best practices (100+ controls)
PCI DSS
Payment Card Industry compliance
HIPAA
Healthcare compliance
AWS Foundational
AWS-recommended security practices
NIST
Government security standards
Enabling Security Hub

```
resource "aws_securityhub_account" "default" {}
resource "aws_securityhub_standards_subscription" "cis" {
standards_arn = "arn:aws:securityhub:us-east-1::standards/cis-aws-foundations-benchmark/v/1.2.0"
}
resource "aws_securityhub_standards_subscription" "aws_foundational" {
standards_arn = "arn:aws:securityhub:us-east-1::standards/aws-foundational-security-best-practices/v/1.0.0"
}
```

Insights (Custom Views)

```
resource "aws_securityhub_insight" "critical_findings" {
group_by_attribute = "ResourceId"
filters {
                                        severity_label {
comparison = "EQUALS"
value      = "CRITICAL"
```

```
      }
      record_state {
        comparison = "EQUALS"
        value      = "ACTIVE"
      }
    }
  }
}
```

Security Hub vs GuardDuty

Aspect	Security Hub	GuardDuty
--------	----------	----------
Purpose	Centralized view	Threat detection
Data sources	Multiple services	CloudTrail, VPC logs
Compliance	Yes (CIS, PCI)	No
Remediation	Custom actions	EventBridge integration

When to Use Security Hub

Use Case	Why Security Hub
----------	----------
Security dashboard	Single pane of glass
Compliance monitoring	Track against standards
Findings consolidation	Aggregate from multiple services
Security score	Measure security posture
Audit preparation	Export compliance reports

Advantages of Security Hub

1. **Centralized:** One dashboard for all security findings
2. **Compliance:** Built-in standards (CIS, PCI, HIPAA)
3. **Integration:** Works with 30+ AWS services
4. **Insights:** Custom views of findings
5. **Automation:** Custom actions for remediation

Disadvantages of Security Hub

1. **Cost:** Additional cost per finding
2. **No remediation:** Only detection and aggregation
3. **Regional:** Must enable in each region

4. **Delay:** Findings take 15-30 minutes to appear

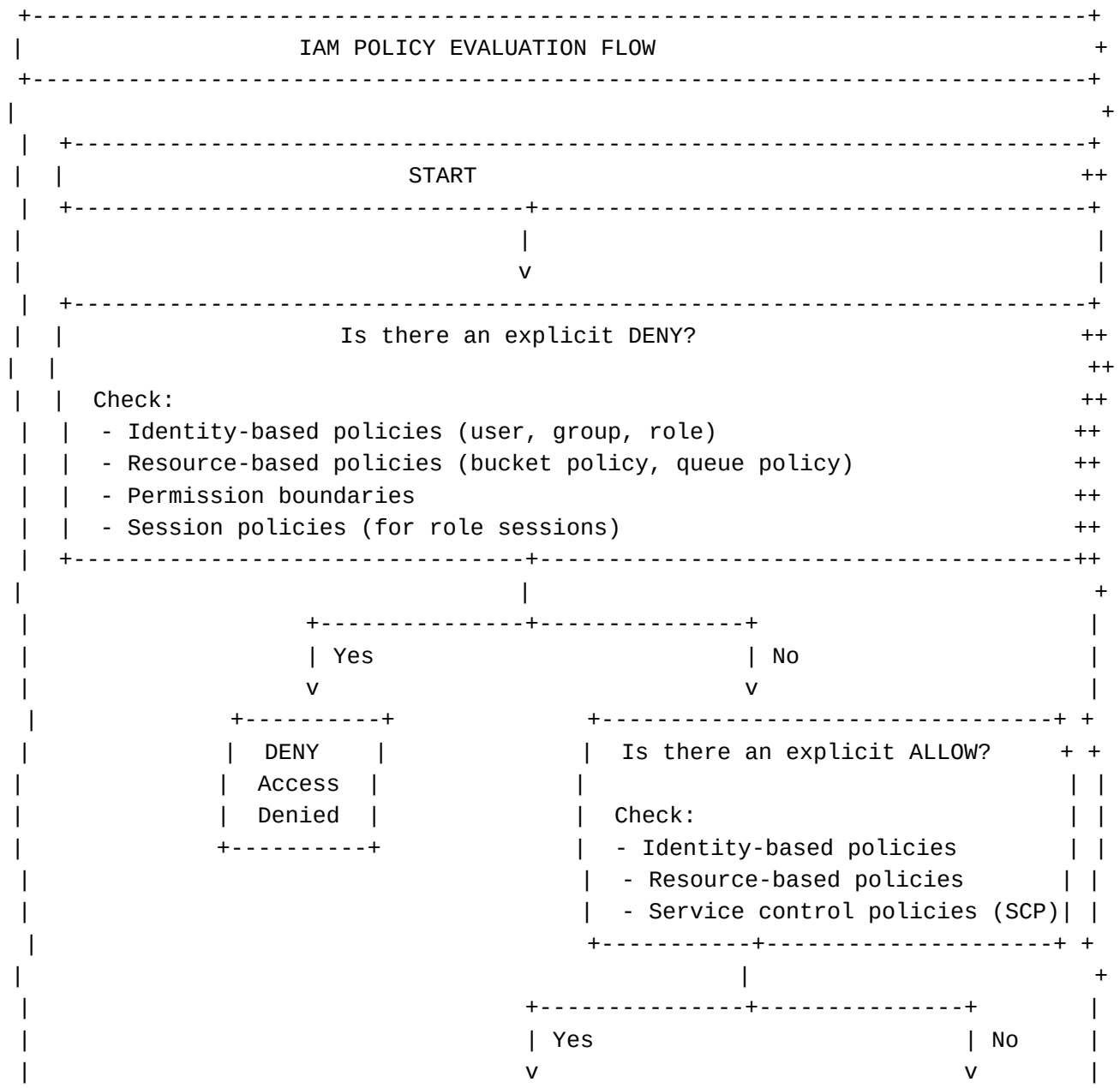
PART EIGHTEEN: ADVANCED IAM & SECURITY PRACTICES

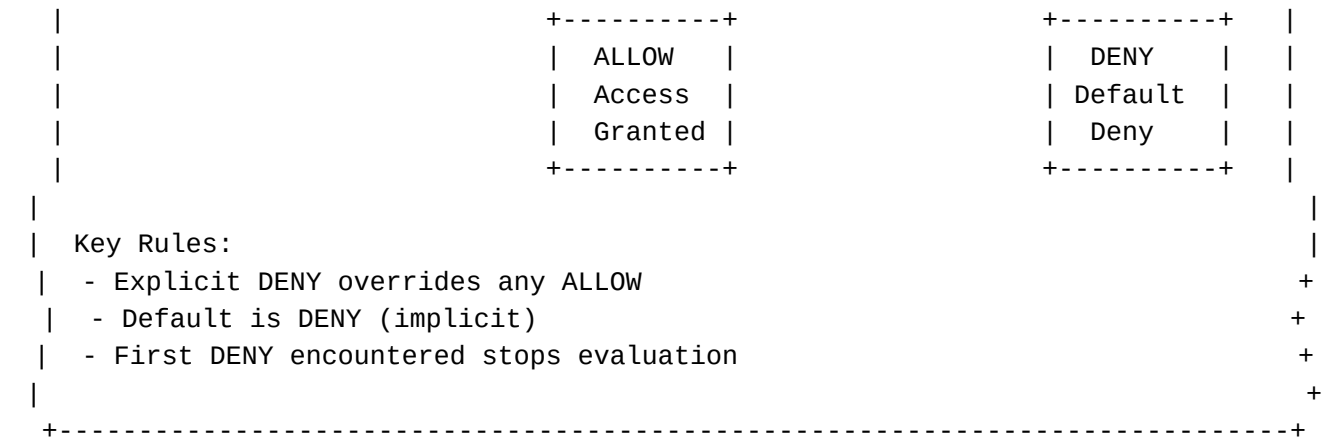
Chapter 52: IAM Policy Evaluation Logic

How IAM Evaluates Policies

IAM policies are evaluated in a specific order to determine whether to allow or deny access.

Diagram: IAM Policy Evaluation Flow





Policy Evaluation Example

```
{
"Effect": "DENY",
"Action": "s3:DeleteBucket",
"Resource": "*"
}
```

vs

```
{
"Effect": "ALLOW",
"Action": "s3:*",
"Resource": "*"
}
```

Result: DENY wins (explicit deny overrides allow)

Multiple Policies (Union of Permissions)

When multiple policies apply, permissions are combined (union). If any policy allows an action and no policy denies it, the action is allowed.

Condition Keys (Contextual Evaluation)

```
{
"Effect": "Allow",
"Action": "ec2:RunInstances",
"Resource": "*",
"Condition": {
"StringEquals": {
                              "aws:RequestedRegion": "us-east-1"
},
"StringLike": {
"ec2:InstanceType": ["t3.*", "m5.*"]
},
"IpAddress": {
"aws:SourceIp": "192.168.0.0/24"
},
"Bool": {
"aws:MultiFactorAuthPresent": "true"
}
}
```

```
        }
```

Common Condition Keys

Condition	Purpose
----------	---------
`aws:SourceIp`	Restrict by source IP
`aws:RequestedRegion`	Restrict by region
`aws:MultiFactorAuthPresent`	Require MFA
`aws:PrincipalTag`	Based on user tags
`aws:ResourceTag`	Based on resource tags
`ec2:InstanceType`	Restrict instance types
`s3:prefix`	Restrict S3 path

[Page 25]

Chapter 53: Least Privilege Design

What is Least Privilege?

The principle of granting only the minimum permissions necessary to perform a task.

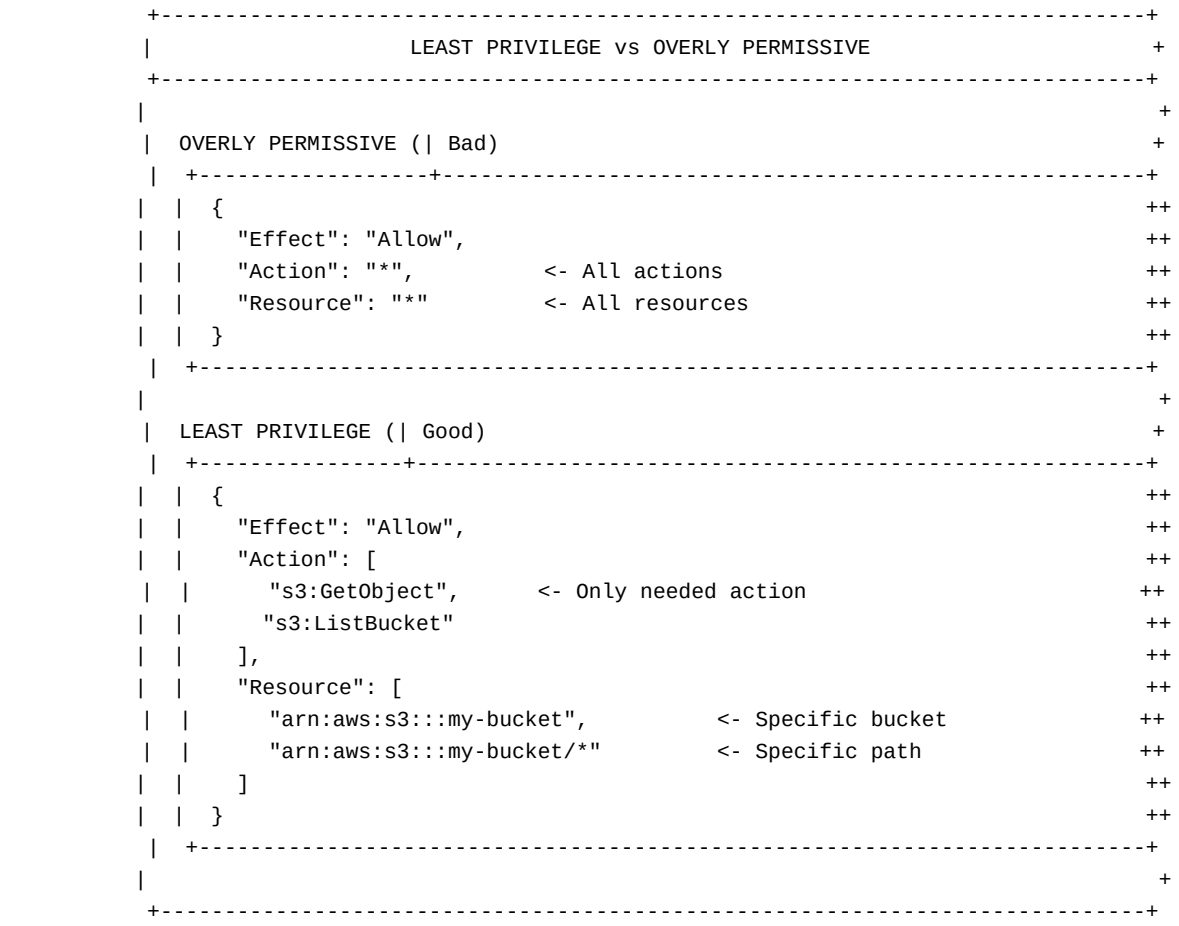
Real Scenario

Your application needs to read from an S3 bucket. It doesn't need to write or delete.

Without Least Privilege:
- Grant `s3:*` (all S3 permissions)
- Security risk (could delete data)
- Compliance violation

With Least Privilege:
- Grant only `s3:GetObject`
- Scoped to specific bucket and prefix
- Reduced risk

Diagram: Least Privilege vs Overly Permissive



Least Privilege Steps

Step
Action

1
Identify the task (e.g., "read from S3")
2
Determine required actions (e.g., `s3:GetObject`)
3
Scope to specific resources (e.g., `my-bucket/logs/*`)
4
Add conditions if needed (e.g., IP restriction)
5
Test with least privilege
6
Monitor and refine

Tools for Least Privilege

Tool
Purpose

IAM Access Analyzer
Generate least privilege policies
IAM Policy Simulator
Test policies before applying
CloudTrail
See what actions are actually used

Tool	Purpose
IAM Access Advisor	Show last accessed information

IAM Access Analyzer (Generate Least Privilege Policy)

```
# Generate policy based on CloudTrail logs
aws accessanalyzer generate-finding \
  --analyzer-arn arn:aws:access-analyzer:us-east-1:123456789012:analyzer/my-analyzer

# Generate policy from activity
aws accessanalyzer generate-policy \
  --principal-arn arn:aws:iam::123456789012:role/my-role \
  --days 30
```

Least Privilege Best Practices

Practice	Why
----------	-----
Start with no permissions	Add only what's needed
Use managed policies when possible	AWS-managed, regularly updated
Scope resources	Don't use `Resource: "*"`
Use conditions	IP, MFA, tags
Review regularly	Remove unused permissions
Use IAM Access Analyzer	Generate least privilege policies
Test with policy simulator	Validate before applying

[Page 26]

Chapter 54: IAM Roles vs Users (Best Practices)

What's the Difference?

Aspect	IAM User	IAM Role
--------	----------	----------
Purpose	Individual human	Service or temporary access
Credentials	Long-term (password + access keys)	Temporary (session-based)
Rotation	Manual	Automatic (AWS STS)
Shared access	Not recommended	Yes (assumed by multiple)
Use case	Human users	EC2, Lambda, cross-account

Best Practices

Practice	Why
---	---
No root user access	Root has full permissions, use only for account setup
Use roles for services	Never store credentials on EC2
Use roles for cross-account	Trust relationships instead of access keys
Use groups for users	Manage permissions by role
Enable MFA	Adds security layer
Rotate access keys	Every 90 days
Delete unused keys	Reduce attack surface

Role vs User Example

```
# ■ Bad: Store credentials on EC2
ENV:
  AWS_ACCESS_KEY_ID: AKIAIOSFODNN7EXAMPLE
  AWS_SECRET_ACCESS_KEY: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY

# ■ Good: EC2 assumes role
ec2_role:
  Type: AWS::IAM::Role
  Properties:
    AssumeRolePolicyDocument:
      Statement:
        - Effect: Allow
          Principal:
            Service: ec2.amazonaws.com
          Action: sts:AssumeRole
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

[Page 27]

Chapter 55: Cross-Account Access

What is Cross-Account Access?

Allowing principals (users or roles) in one AWS account to access resources in another AWS account.

Real Scenario

Your organization has separate AWS accounts for development, staging, and production. Developers need access to all accounts.

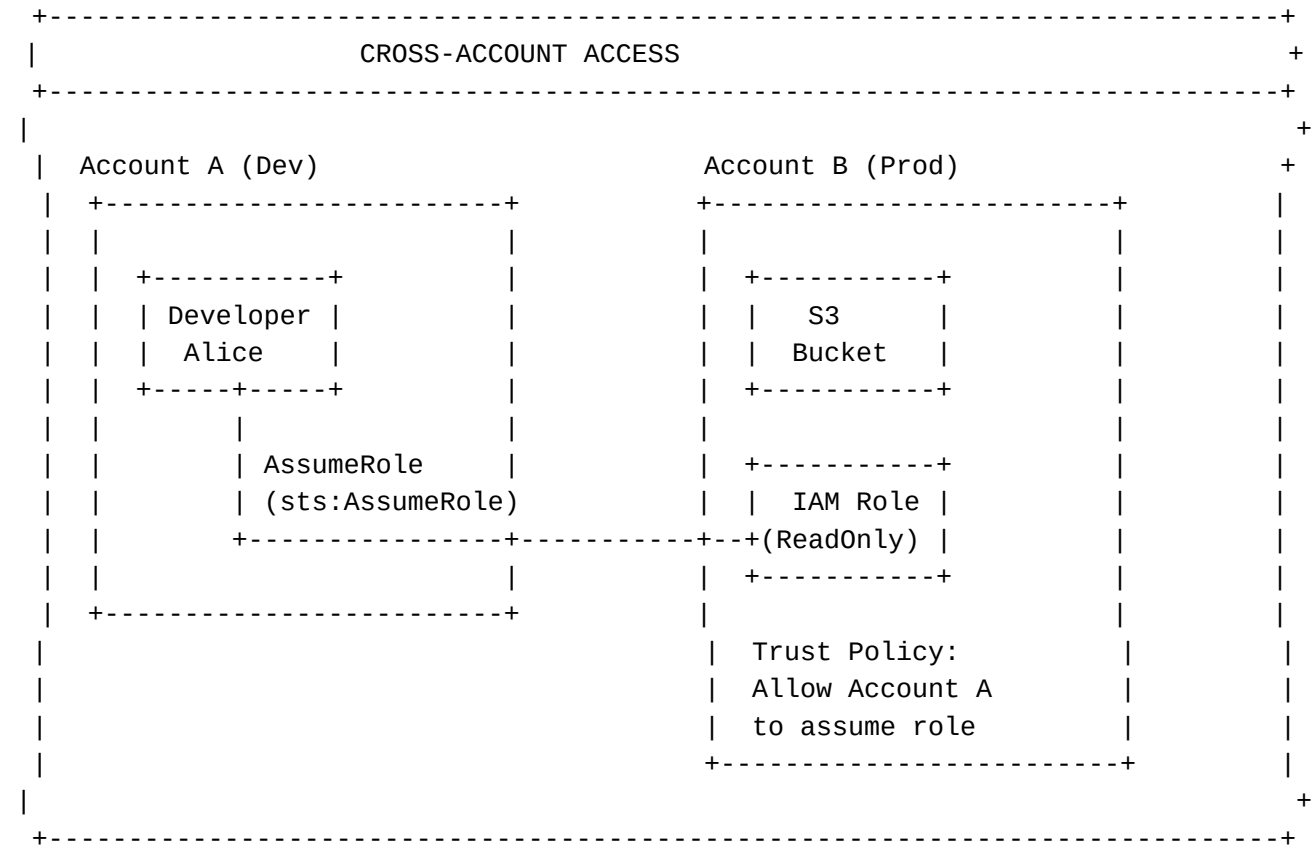
Without Cross-Account Access:

- Separate credentials for each account
- Manual role switching
- Security risk (multiple sets of keys)

With Cross-Account Access:

- Assume role from dev account to prod
- Single set of credentials
- Audit trail via CloudTrail

Diagram: Cross-Account Access



Role Trust Policy (Account B - Prod)

```
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Principal": {
"AWS": "arn:aws:iam::123456789012:root"
},
"Action": "sts:AssumeRole",
"Condition": {
"Bool": {
```

```
            "aws:MultiFactorAuthPresent": "true"
          }
        }
      }
    ]
  }
```

Role Permissions Policy (Account B - Prod)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::prod-bucket/*"
    }
  ]
}
```

Assume Role from Account A (Dev)

```
# Assume role
aws sts assume-role \
  --role-arn "arn:aws:iam::567890123456:role/ProdReadOnlyRole" \
  --role-session-name "AliceSession"

# Use temporary credentials
export AWS_ACCESS_KEY_ID="ASIA..."
export AWS_SECRET_ACCESS_KEY="..."
export AWS_SESSION_TOKEN="..."

# Access resource in Account B
aws s3 ls s3://prod-bucket
```

Cross-Account Access Best Practices

Practice	Why
----------	-----
Use roles, not users	Roles for cross-account access
Require MFA	Extra security for sensitive accounts
Use permission boundaries	Limit max permissions
Audit with CloudTrail	Track cross-account activity
Use AWS Organizations	Simplify cross-account management
Least privilege	Grant minimum required permissions

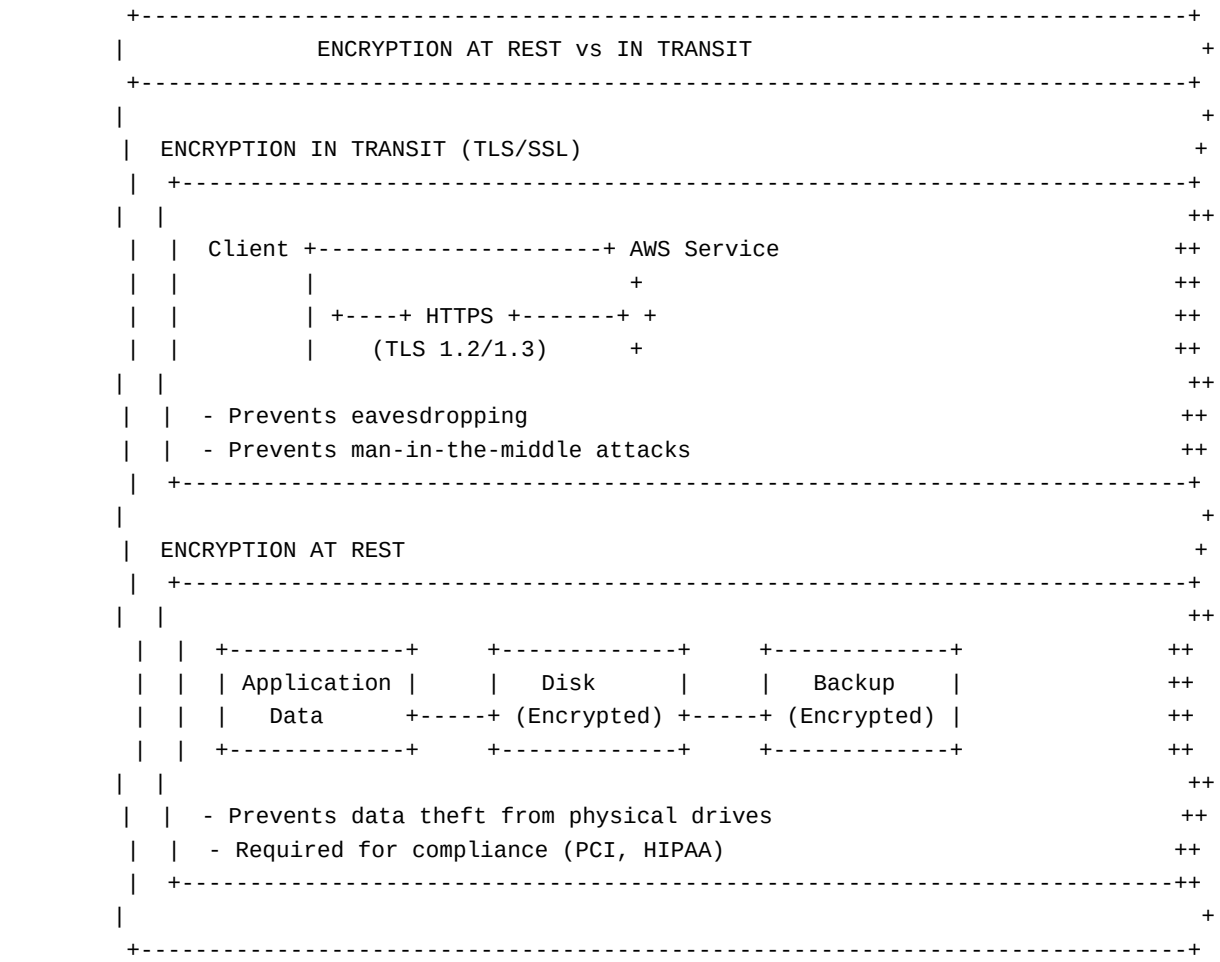
PART NINETEEN: ENCRYPTION & SECRETS MANAGEMENT (EXPANDED)

Chapter 56: Encryption at Rest vs In Transit

What's the Difference?

Aspect	Encryption at Rest	Encryption in Transit
When	Data stored on disk	Data moving over network
Purpose	Protect from physical theft	Protect from interception
Examples	EBS encryption, S3 SSE	TLS/SSL, HTTPS

Diagram: Encryption at Rest vs In Transit

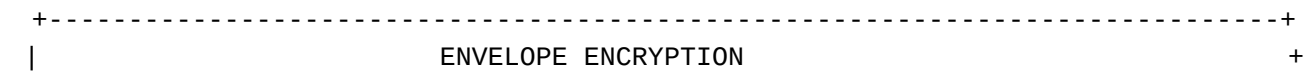


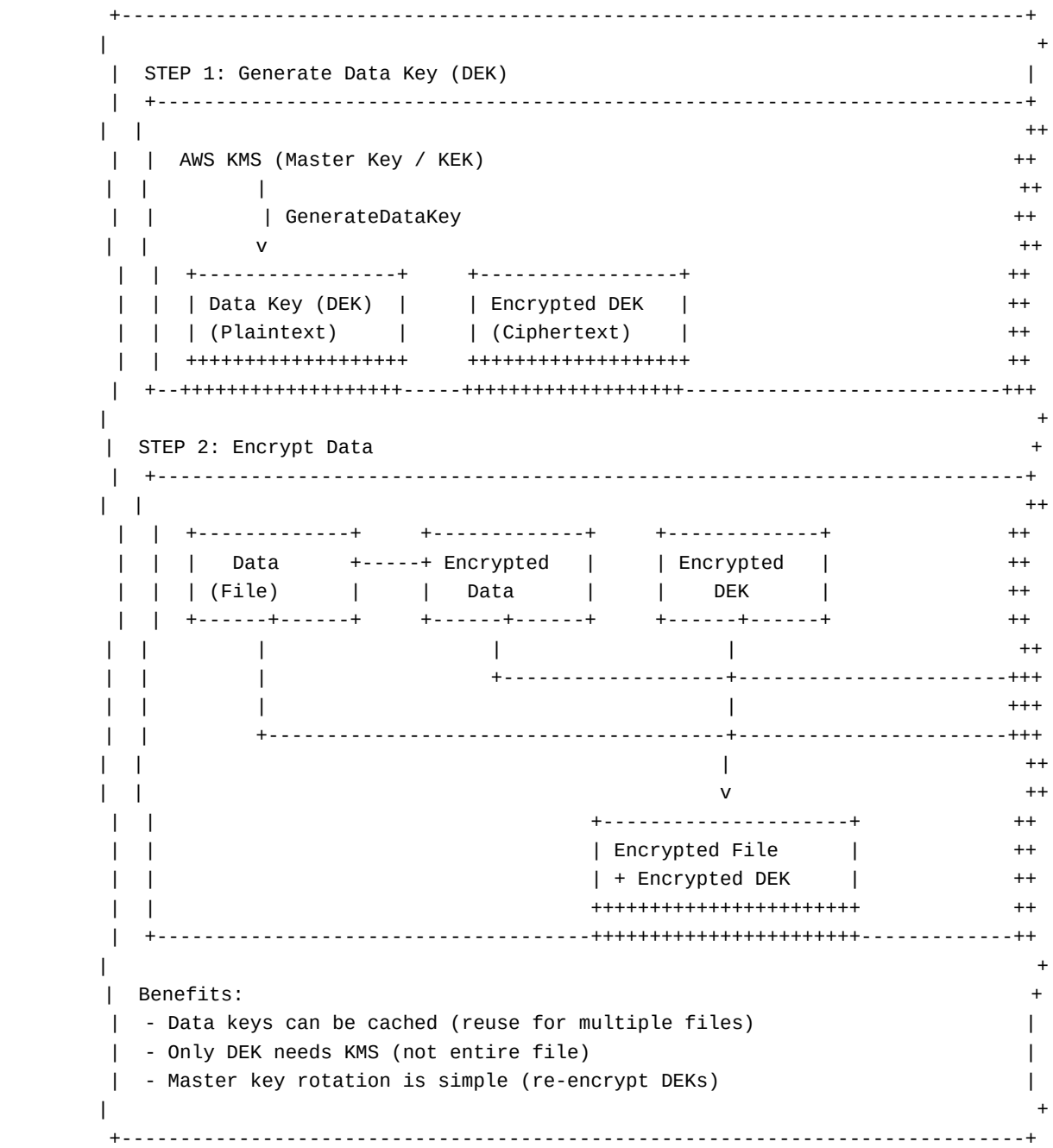
AWS Encryption Options

Service	At Rest	In Transit

Service
At Rest
In Transit
S3
SSE-S3, SSE-KMS, SSE-C
HTTPS
EBS
EBS encryption (KMS)
N/A (attached to EC2)
RDS
RDS encryption (KMS)
TLS
DynamoDB
Default encryption
HTTPS
Lambda
Environment variables (KMS)
HTTPS

[Page 29]
Chapter 57: Envelope Encryption
What is Envelope Encryption?
Using a master key (KEK) to encrypt data keys (DEK), which then encrypt data. This allows efficient key rotation and reduces KMS costs.
Real Scenario
You need to encrypt thousands of files in S3. Using KMS directly for each file would be expensive (KMS API calls cost money).
Without Envelope Encryption:
- Call KMS for every file (costly)
- KMS has rate limits
- Each file = one API call
With Envelope Encryption:
- Generate one data key per file
- Encrypt file with data key (local, fast)
- Encrypt data key with KMS (only once per file)
- Pay only for KMS API calls (one per file)
Diagram: Envelope Encryption





S3 Server-Side Encryption with KMS (SSE-KMS) - Uses envelope encryption

```
resource "aws_s3_bucket" "encrypted_bucket" {
bucket = "my-encrypted-bucket"
}
resource "aws_s3_bucket_server_side_encryption_configuration" "example" {
bucket = aws_s3_bucket.encrypted_bucket.id
rule {
                            apply_server_side_encryption_by_default {
                             kms_master_key_id = aws_kms_key.mykey.arn
                                   sse_algorithm     = "aws:kms"
}
}
}
```

Custom Envelope Encryption (AWS Encryption SDK)

```
const AWS = require('aws-sdk');
```

```
const { KmsKeyringNode } = require('@aws-crypto/client-node');
const { encrypt, decrypt } = require('@aws-crypto/client-node');

const kms = new AWS.KMS({ region: 'us-east-1' });

async function encryptFile(data, keyArn) {
  // Create keyring with KMS master key
  const keyring = new KmsKeyringNode({ generatorKeyId: keyArn });

  // Encrypt data (envelope encryption automatically)
  const { result } = await encrypt(keyring, data);

  // result contains encrypted data + encrypted data keys
  return result;
}

async function decryptFile(encryptedData, keyArn) {
  const keyring = new KmsKeyringNode({ generatorKeyId: keyArn });
  const { plaintext } = await decrypt(keyring, encryptedData);
  return plaintext;
}
```

[Page 30]

Chapter 58: Key Rotation Strategies

What is Key Rotation?

Replacing old encryption keys with new ones to limit exposure if a key is compromised.

Key Rotation Strategies

Strategy	Description	Use Case
---	---	---
Automatic (AWS-managed)	AWS rotates keys automatically	Default, low effort
Manual (Customer-managed)	You rotate keys manually	Compliance, custom schedule
Alias-based	Update alias to point to new key	Zero-downtime rotation
Re-encrypt	Re-encrypt data with new key	Strong security

KMS Key Rotation

```
resource "aws_kms_key" "rotating_key" {
  description             = "Key with automatic rotation"
  enable_key_rotation     = true  # Automatic rotation every year
  deletion_window_in_days = 7
}

# Custom rotation schedule (Lambda)
```

```
resource "aws_lambda_function" "rotate_key" {
  filename         = "rotate.zip"
  function_name    = "rotate-kms-key"
  role             = aws_iam_role.rotation_role.arn
  handler          = "index.handler"
  runtime          = "nodejs18.x"
}

# Schedule rotation monthly
resource "aws_cloudwatch_event_rule" "key_rotation" {
  name                = "key-rotation-schedule"
  schedule_expression = "cron(0 0 1 * ? *)"  # 1st of every month
}

resource "aws_cloudwatch_event_target" "rotate" {
  rule = aws_cloudwatch_event_rule.key_rotation.name
  arn  = aws_lambda_function.rotate_key.arn
}
```

Data Key Rotation (Re-encryption)

```
async function rotateDataKeys(bucket, prefix, newKeyArn) {
  const s3 = new AWS.S3();
  const kms = new AWS.KMS();

  // List all objects
  const objects = await s3.listObjectsV2({ Bucket: bucket, Prefix: prefix }).promise();

  for (const object of objects.Contents) {
    // Download encrypted object
    const data = await s3.getObject({ Bucket: bucket, Key: object.Key }).promise();

    // Decrypt with old key (KMS tracks which key was used)
    const decrypted = await kms.decrypt({ CiphertextBlob: data.Body }).promise();

    // Re-encrypt with new key
    const reencrypted = await kms.encrypt({
      KeyId: newKeyArn,
      Plaintext: decrypted.Plaintext
    }).promise();

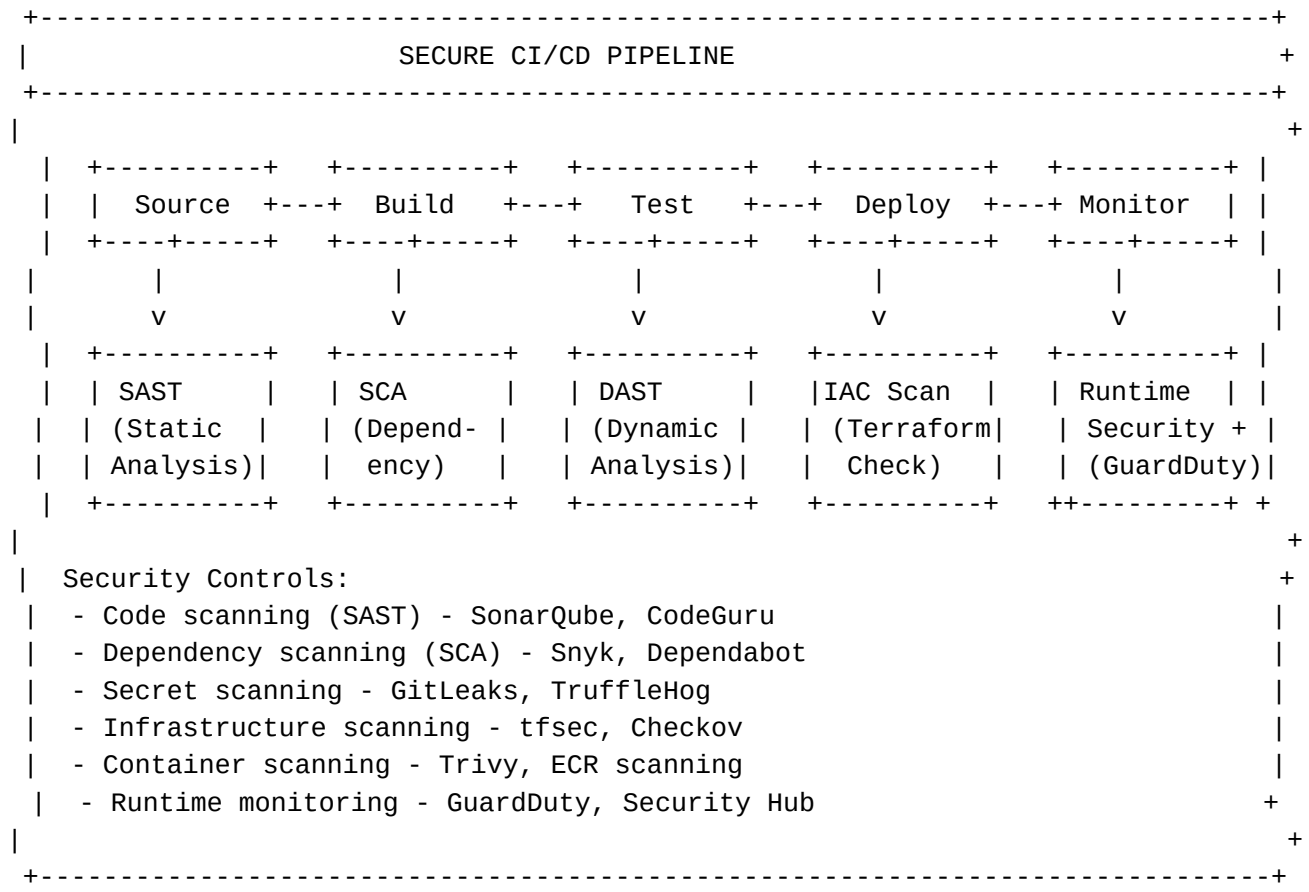
    // Upload re-encrypted object
    await s3.putObject({
      Bucket: bucket,
      Key: object.Key,
      Body: reencrypted.CiphertextBlob
    }).promise();
  }
}
```

Key Rotation Best Practices

Practice	**Why**
----------	-----
Enable automatic rotation	AWS-managed, no effort
Rotate data keys regularly	Limit exposure window
Use aliases for customer-managed	Abstract key IDs

Practice
Why
Monitor key usage
Detect unusual patterns
Test rotation process
Ensure it works
Keep old keys for decryption
Don't delete immediately

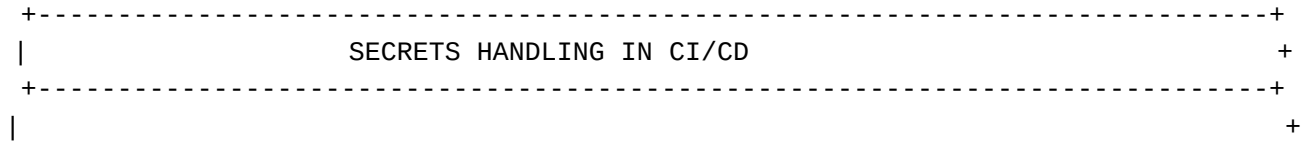
[Page 31]
PART TWENTY: SECURE CI/CD PIPELINES
Chapter 59: Pipeline Security Best Practices
What is Secure CI/CD?
Integrating security controls into every stage of the CI/CD pipeline (DevSecOps).
Diagram: Secure CI/CD Pipeline

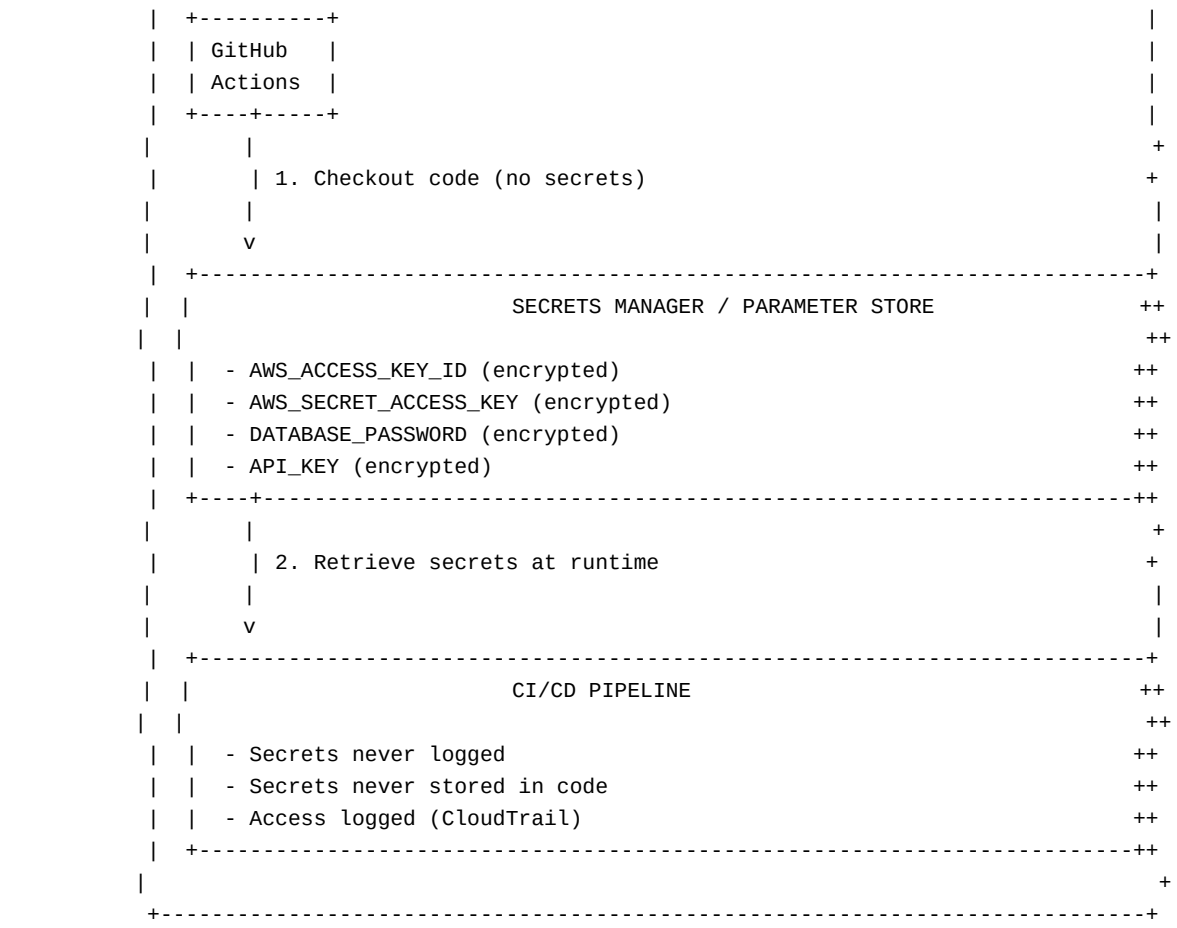


Security Controls by Stage
Stage
Security Control
Tools

Stage
Security Control
Tools
Source
Secret scanning, SAST
GitLeaks, SonarQube, CodeGuru
Build
SCA, Container scan
Snyk, Trivy, ECR scanning
Test
DAST, API security
OWASP ZAP, Burp Suite
Deploy
IaC scan, Policy as Code
tfsec, Checkov, OPA
Monitor
Runtime security
GuardDuty, Security Hub

[Page 32]
Chapter 60: Secrets Handling in Pipelines
What is Secrets Handling?
Securely managing and accessing secrets (API keys, passwords, tokens) in CI/CD pipelines.
Real Scenario
Your pipeline needs to access AWS credentials to deploy infrastructure. You can't hardcode them in the pipeline configuration.
Without Secrets Handling:
- Hardcoded secrets in code (exposed in Git)
- Environment variables (visible in logs)
- Security breach
With Secrets Handling:
- Secrets stored in Secrets Manager
- Retrieved at runtime
- Never exposed in logs
Diagram: Secrets Handling in CI/CD





GitHub Actions Secrets Example

```
# .github/workflows/deploy.yml
name: Deploy
on:
  push:
    branches: [main]
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v1
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      - name: Retrieve database password
        run: |
          DB_PASSWORD=$(aws secretsmanager get-secret-value \
            --secret-id prod/db/password \
            --query SecretString \
            --output text)

      # Use password (without logging)
      echo "::add-mask::$DB_PASSWORD"
      ./deploy.sh --db-password "$DB_PASSWORD"
```

Secrets Handling Best Practices

Practice	Why
----------	-----
Never hardcode secrets	Exposed in Git history
Use secrets manager	Centralized, audited
Mask secrets in logs	Prevent accidental exposure
Rotate regularly	Limit exposure window
Least privilege	Pipeline only needs deploy permissions
Audit secret access	Detect unauthorized access
Use OIDC instead of long-term keys	Temporary credentials

[Page 33]

Chapter 61: Image & Dependency Scanning

What is Image & Dependency Scanning?

Analyzing container images and application dependencies for known vulnerabilities.

Real Scenario

Your application uses third-party libraries. A new CVE is discovered in one of them. You need to detect and fix it.

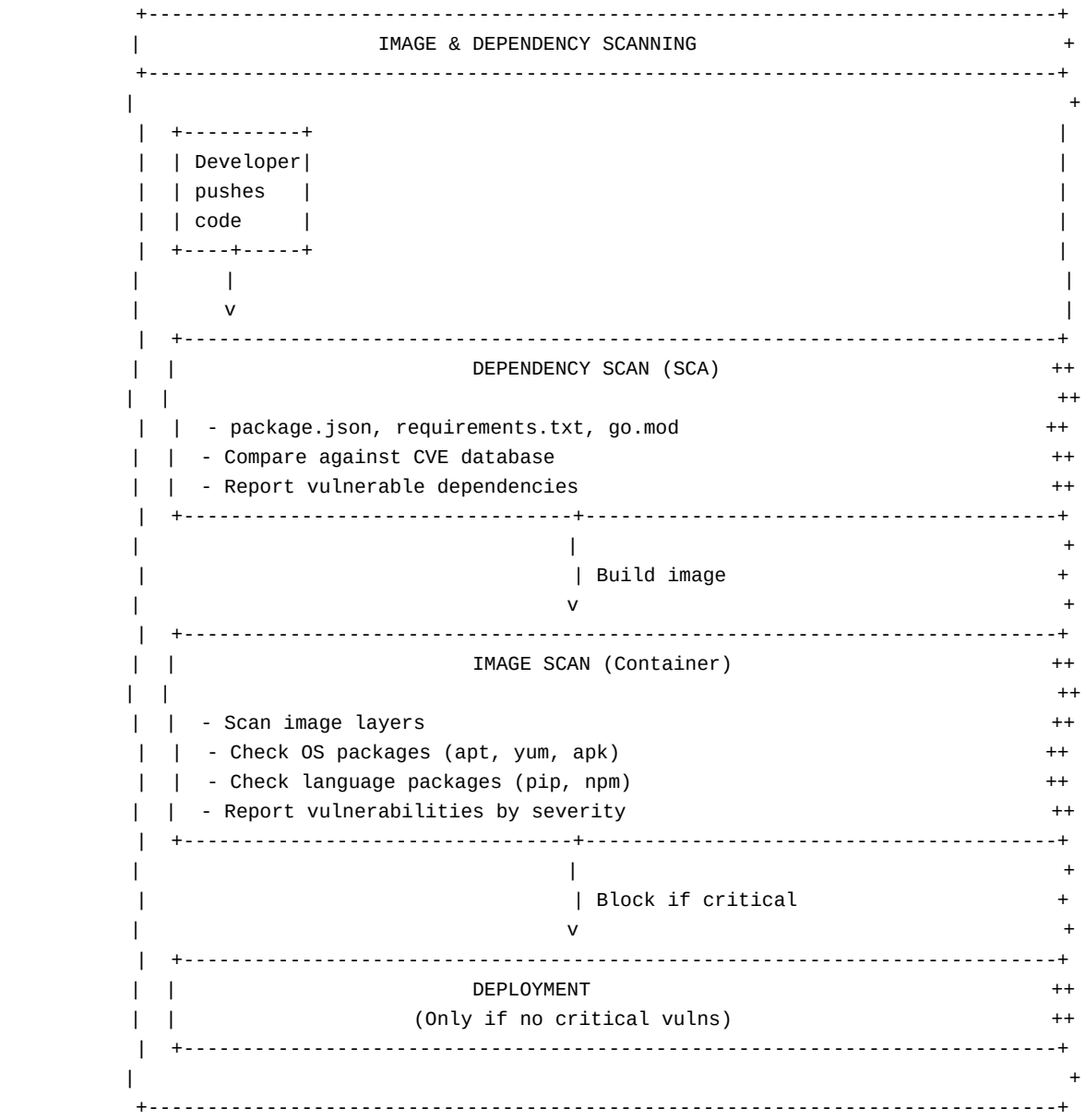
Without Scanning:

- Vulnerable code deployed to production
- Security breach
- Compliance violation

With Scanning:

- Detect vulnerabilities before deployment
- Block vulnerable images
- Meet compliance requirements

Diagram: Image & Dependency Scanning



Trivy Scan Example

```
# GitHub Actions workflow
name: Container Scan
jobs:
  scan:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3

      - name: Build image
        run: docker build -t myapp:latest .

      - name: Run Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: myapp:latest
          format: 'sarif'
          output: 'trivy-results.sarif'
          severity: 'CRITICAL,HIGH'
```

```
      - name: Upload scan results
        uses: github/codeql-action/upload-sarif@v2
        with:
          sarif_file: trivy-results.sarif

      - name: Fail if critical vulnerabilities
        run: |
          if grep -q '"severity":"CRITICAL"' trivy-results.sarif; then
            echo "Critical vulnerabilities found!"
            exit 1
          fi
```

Dependency Scanning (Dependabot)

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 10
    labels:
      - "dependencies"
      - "security"

  - package-ecosystem: "docker"
    directory: "/"
    schedule:
      interval: "weekly"

  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
```

Chapter 62: Access Control in CI/CD

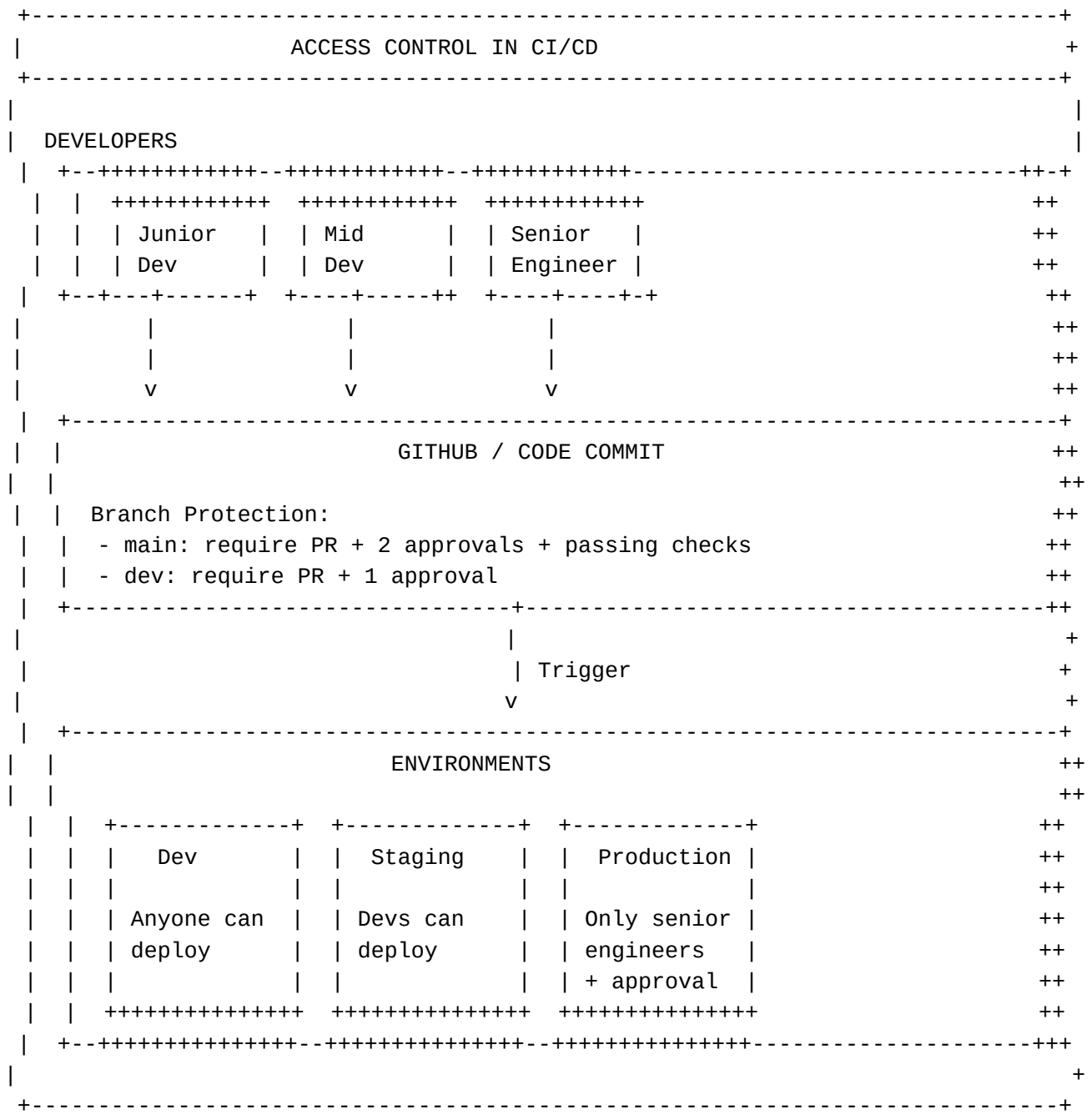
What is Access Control in CI/CD?

Managing who and what can modify CI/CD pipelines, access secrets, and deploy to environments.

Real Scenario

Only senior engineers should be able to deploy to production. Developers can deploy to dev/staging.

Without Access Control:
- Anyone can modify pipeline
- Anyone can deploy to prod
- Security risk
With Access Control:
- Branch protection rules
- Environment approvals
- Role-based access
Diagram: CI/CD Access Control



GitHub Environment Protection Rules

```
# .github/workflows/deploy.yml
name: Deploy
```

```
on:
  push:
    branches: [main]

jobs:
  deploy-prod:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://myapp.com
    steps:
      - uses: actions/checkout@v3

      - name: Deploy to production
        run: ./deploy.sh
```

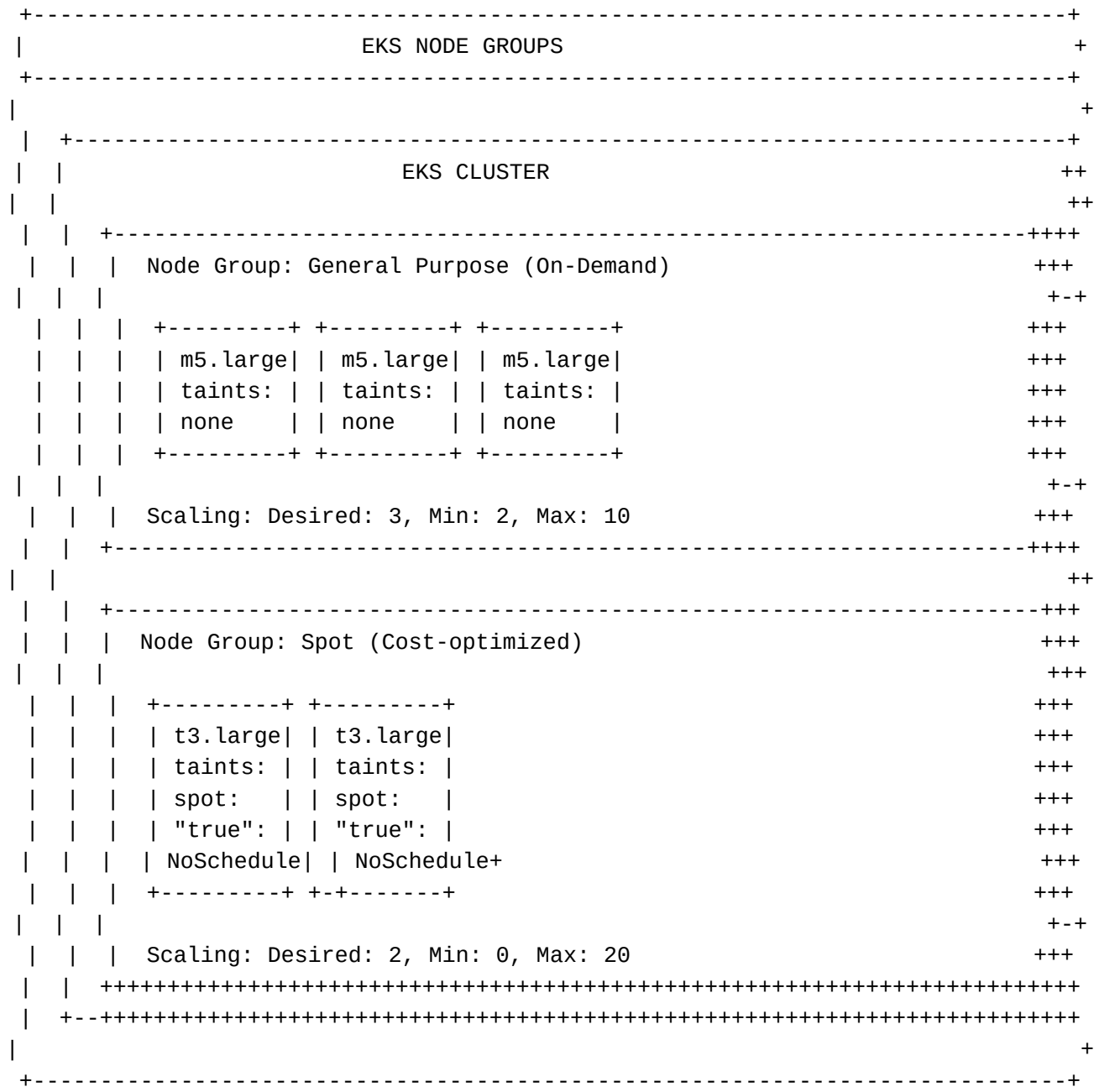
IAM Role for CI/CD (Least Privilege)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "ecs:UpdateService",
        "ecs:DescribeServices"
      ],
      "Resource": "arn:aws:ecs:us-east-1:123456789012:service/prod/*"
    },
    {
      "Effect": "Deny",
      "Action": [
        "s3:DeleteBucket",
        "rds:DeleteDBInstance"
      ],
      "Resource": "*"
    }
  ]
}
```

Access Control Best Practices

Practice	**Why**
----------	-----
Branch protection	Prevent direct pushes to main
Required reviews	Code quality, security
Status checks	Automated tests must pass
Environment approvals	Manual approval for prod
Least privilege for CI/CD	Pipeline only gets deploy permissions
Audit pipeline changes	Track who modified pipeline
Separate CI/CD from application	Different IAM roles

PART TWENTY-ONE: KUBERNETES (EKS)
DEEP DIVE
Chapter 63: Node Groups & Scaling
What are Node Groups?
A set of EC2 instances (nodes) in an EKS cluster that have the same configuration (instance type, AMI, scaling settings).
Diagram: EKS Node Groups



EKS Node Group Terraform

```
resource "aws_eks_node_group" "general" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "general-purpose"
  node_role_arn   = aws_iam_role.eks_node.arn
  subnet_ids      = var.private_subnet_ids

  scaling_config {
    desired_size = 3
    max_size     = 10
    min_size     = 2
  }

  instance_types = ["m5.large"]

  labels = {
    "node-group" = "general"
    "spot"       = "false"
  }

  taint {
    key    = "dedicated"
    value  = "general"
    effect = "NO_SCHEDULE"
  }

  update_config {
    max_unavailable = 1
  }
}

resource "aws_eks_node_group" "spot" {
  cluster_name    = aws_eks_cluster.main.name
  node_group_name = "spot-workers"
  node_role_arn   = aws_iam_role.eks_node.arn
  subnet_ids      = var.private_subnet_ids

  scaling_config {
    desired_size = 2
    max_size     = 20
    min_size     = 0
  }

  instance_types = ["t3.large", "t3.xlarge"]

  capacity_type = "SPOT"

  labels = {
    "node-group" = "spot"
    "spot"       = "true"
  }

  taint {
    key    = "spot"
    value  = "true"
    effect = "NO_SCHEDULE"
  }
}
```

Cluster Autoscaler (for node scaling)

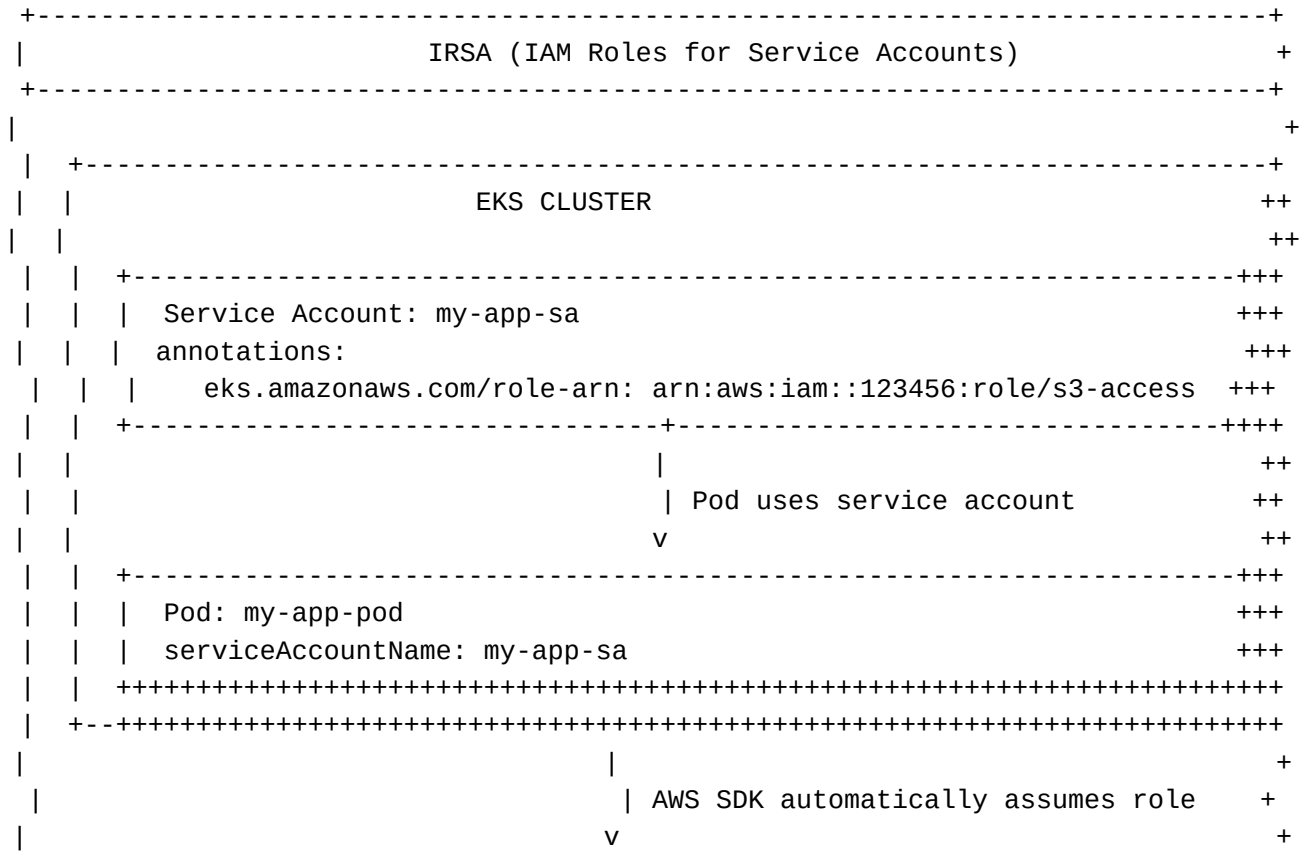
```
# cluster-autoscaler.yaml
apiVersion: apps/v1
```

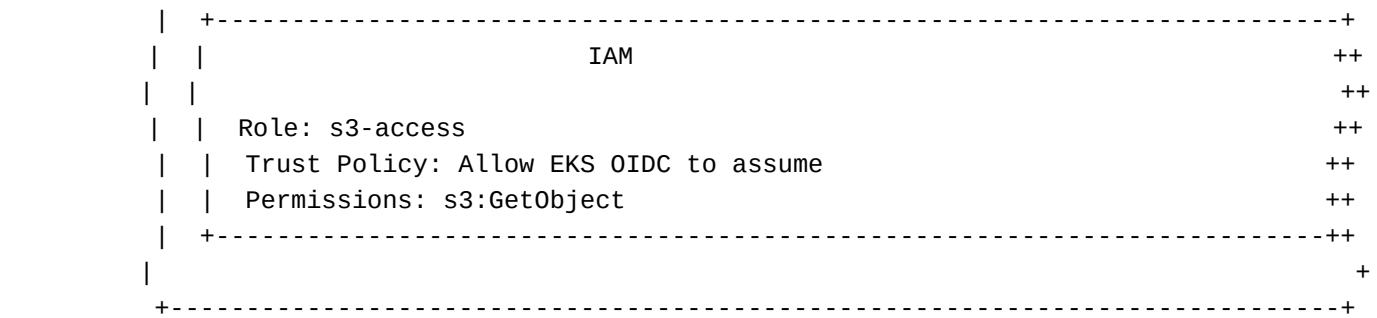
```
kind: Deployment
metadata:
  name: cluster-autoscaler
  namespace: kube-system
spec:
  replicas: 1
  selector:
    matchLabels:
      app: cluster-autoscaler
  template:
    metadata:
      labels:
        app: cluster-autoscaler
    spec:
      serviceAccountName: cluster-autoscaler
      containers:
      - name: cluster-autoscaler
        image: k8s.gcr.io/autoscaling/cluster-autoscaler:v1.27.0
        command:
        - ./cluster-autoscaler
        - --v=4
        - --stderrthreshold=info
        - --cloud-provider=aws
        - --skip-nodes-with-local-storage=false
        - --expander=priority
        - --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,k8s.io/cluster
```

Pod Topology Spread Constraints

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 6
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: DoNotSchedule
        labelSelector:
          matchLabels:
            app: my-app
      containers:
      - name: app
        image: myapp:latest
```

Chapter 64: IAM Roles for Service Accounts (IRSA)
What is IRSA?
Allows Kubernetes pods to assume IAM roles directly, without storing credentials in the pod.
Real Scenario
Your pod needs to access S3. You don't want to store AWS credentials in environment variables or as secrets.
Without IRSA:
- Store AWS credentials in pod (security risk)
- Long-lived access keys
- Hard to rotate
With IRSA:
- Pod assumes IAM role
- Temporary credentials via OIDC
- No credentials stored in pod
Diagram: IRSA Flow





IRSA Configuration

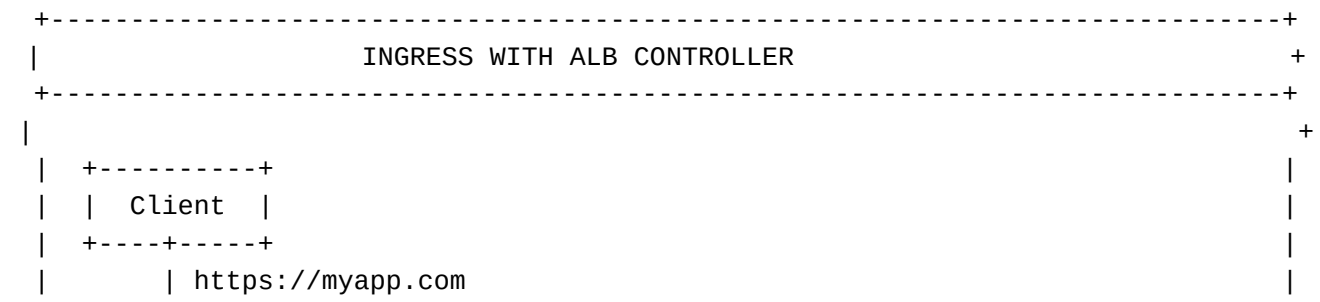
```
# 1. Create IAM role for service account
resource "aws_iam_role" "s3_access" {
name = "s3-access-role"
assume_role_policy = jsonencode({
                                    Version = "2012-10-17"
Statement = [
{
Effect = "Allow"
Principal = {
                         Federated = aws_iam_openid_connect_provider.eks.arn
}
Action = "sts:AssumeRoleWithWebIdentity"
Condition = {
StringEquals = {
                "${aws_iam_openid_connect_provider.eks.url}:aud" = "sts.amazonaws.com"
              "${aws_iam_openid_connect_provider.eks.url}:sub" = "system:serviceaccount:default:my-app-sa"
}
}
}
]
})
}
resource "aws_iam_role_policy_attachment" "s3_access" {
role       = aws_iam_role.s3_access.name
policy_arn = "arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess"
}
# 2. Create Kubernetes service account
resource "kubernetes_service_account" "my_app" {
metadata {
                                      name = "my-app-sa"
annotations = {
                    "eks.amazonaws.com/role-arn" = aws_iam_role.s3_access.arn
}
}
}
```

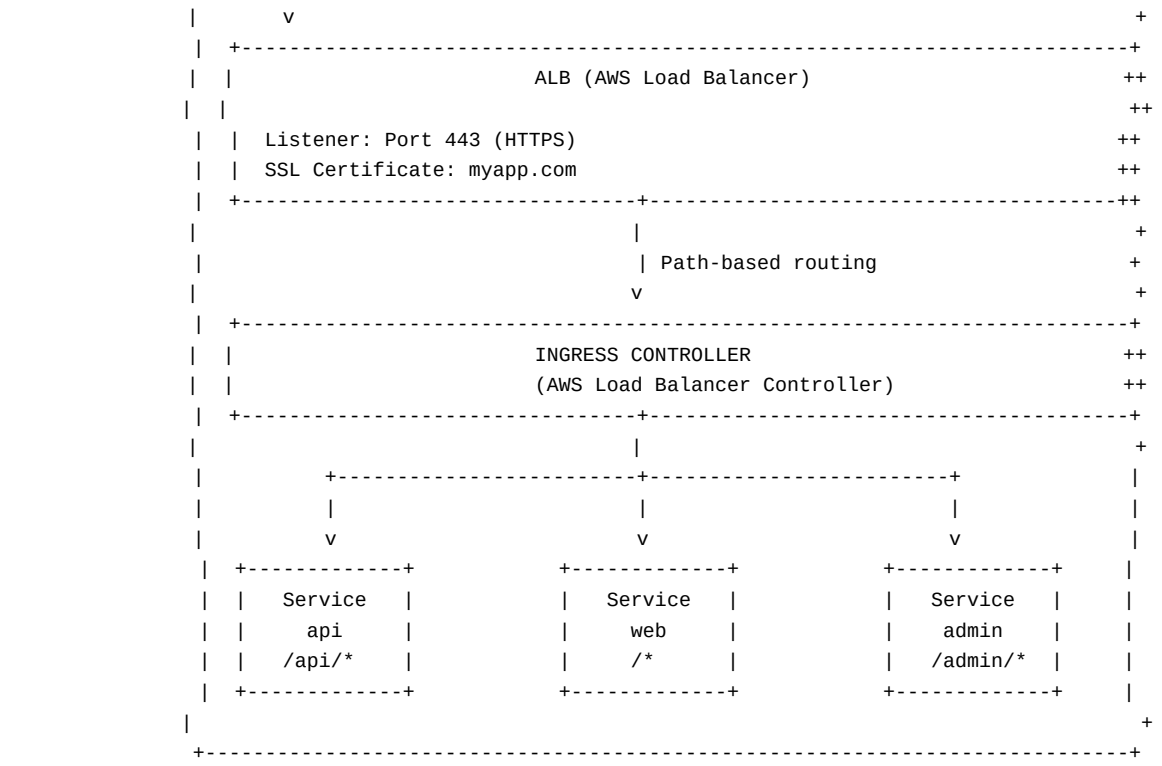
Pod using IRSA

```
apiVersion: v1
kind: Pod
metadata:
name: my-app
spec:
serviceAccountName: my-app-sa
containers:
- name: app
```

image: myapp:latest
AWS SDK will automatically use the IRSA credentials
AWS SDK automatically picks up credentials from environment variables injected by EKS

[Page 37]
Chapter 65: Ingress (ALB Controller)
What is Ingress?
Kubernetes resource that manages external access to services, typically HTTP/HTTPS traffic.
Real Scenario
You have multiple services (api, web, admin) and need to route traffic to them based on URL path (api/*, admin/*, /*).
Without Ingress:
- Each service needs its own load balancer (expensive)
- Manual routing configuration
- No SSL termination at cluster level
With Ingress:
- One load balancer for multiple services
- Path-based routing
- SSL termination at Ingress
Diagram: Ingress with ALB Controller



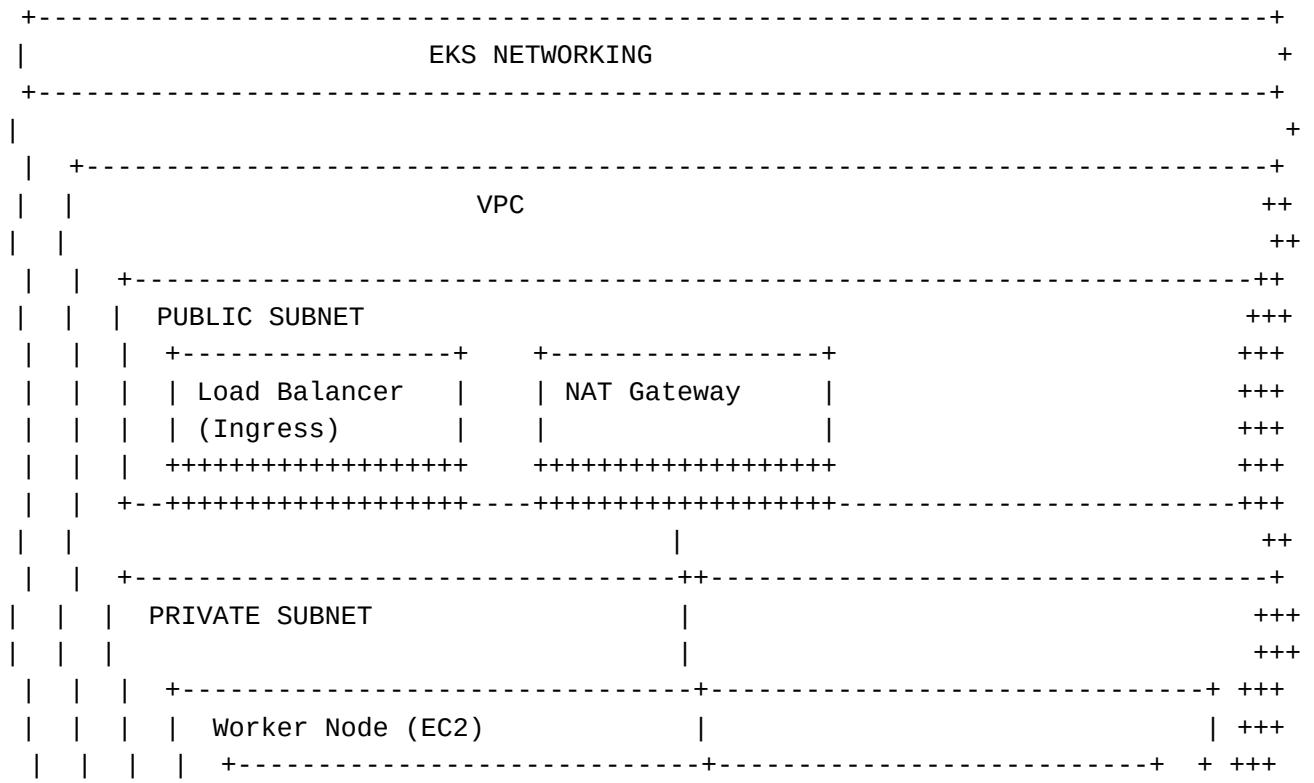


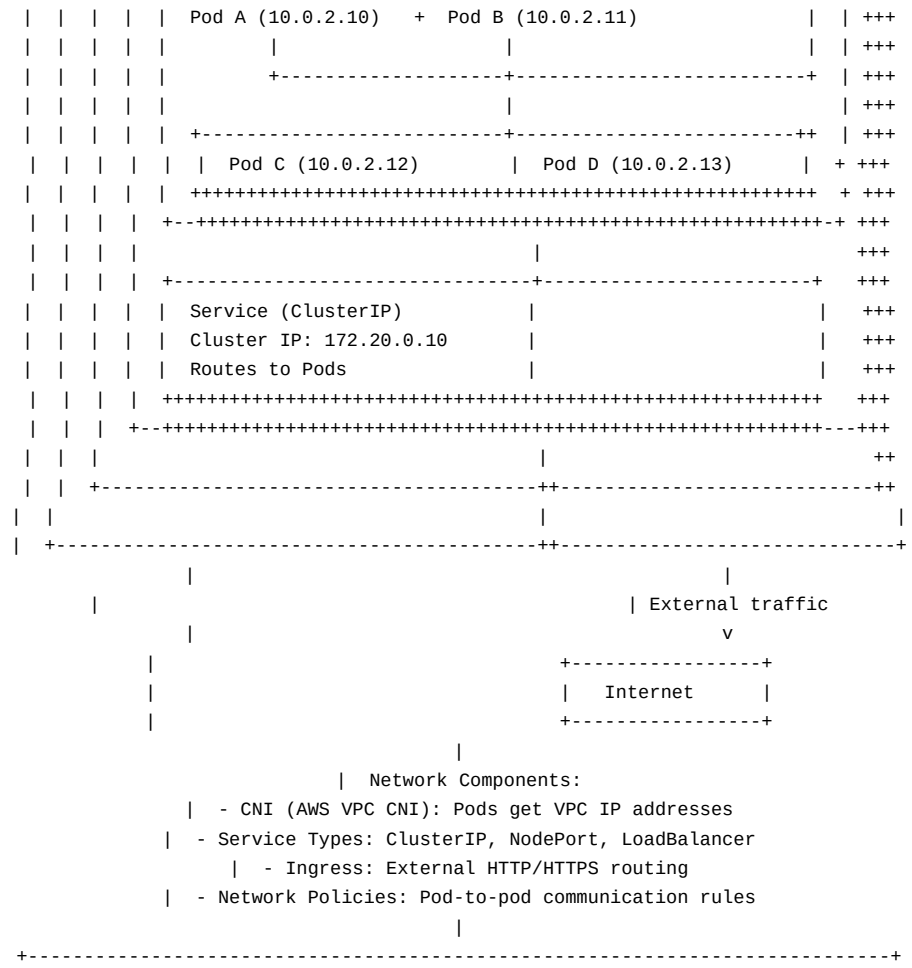
Ingress Resource

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
  annotations:
    kubernetes.io/ingress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/listen-ports: '[{"HTTP":80,"HTTPS":443}]'
    alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-east-1:123456789012:certificate/xxxx
    alb.ingress.kubernetes.io/healthcheck-path: /health
    alb.ingress.kubernetes.io/healthcheck-interval-seconds: '30'
    alb.ingress.kubernetes.io/target-type: ip
spec:
  rules:
  - host: myapp.com
    http:
      paths:
        - path: /api/*
          pathType: Prefix
          backend:
            service:
              name: api-service
              port:
                number: 8080
        - path: /admin/*
          pathType: Prefix
          backend:
            service:
              name: admin-service
              port:
                number: 8080
        - path: /*
          pathType: Prefix
          backend:
            service:
```

```
                                        name: web-service
port:
number: 80
```
Install AWS Load Balancer Controller
```
# Create IAM policy for controller
curl -o iam-policy.json
https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.jso
aws iam create-policy \
--policy-name AWSLoadBalancerControllerIAMPolicy \
--policy-document file://iam-policy.json
# Install controller via Helm
helm repo add eks https://aws.github.io/eks-charts
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
--set clusterName=my-cluster \
--set serviceAccount.create=true \
--set serviceAccount.name=aws-load-balancer-controller \
-n kube-system
```

[Page 38]
Chapter 66: Networking in EKS
What is EKS Networking?
How pods communicate with each other, with services, and with external resources.
Diagram: EKS Networking





Service Types

Type
Description
Use Case

ClusterIP
Internal only
Internal services
NodePort
Exposes on node port
Testing
LoadBalancer
Creates ALB/NLB
External access
ExternalName
External service alias
Third-party services

Network Policy (Zero Trust)

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
name: api-network-policy
spec:
podSelector:
matchLabels:
app: api
policyTypes:
- Ingress
- Egress
ingress:

```
  - from:
    - podSelector:
        matchLabels:
          app: web
    ports:
    - protocol: TCP
      port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - protocol: TCP
      port: 5432
  - to:
    - namespaceSelector:
        matchLabels:
          name: kube-system
    ports:
    - protocol: UDP
      port: 53
```

VPC CNI Configuration

```
# Custom networking configuration
resource "aws_eks_cluster" "main" {
  name     = "my-cluster"
  role_arn = aws_iam_role.eks_cluster.arn

  vpc_config {
    subnet_ids              = concat(var.public_subnet_ids, var.private_subnet_ids)
    endpoint_private_access = true
    endpoint_public_access  = false
  }
}

# VPC CNI custom networking (pods get IPs from secondary CIDR)
resource "aws_eks_addon" "vpc_cni" {
  cluster_name = aws_eks_cluster.main.name
  addon_name   = "vpc-cni"

  addon_version = "v1.14.0-eksbuild.1"

  configuration_values = jsonencode({
    env = {
      ENABLE_PREFIX_DELEGATION = "true"
      WARM_IP_TARGET           = "5"
      WARM_ENI_TARGET          = "2"
    }
  })
}
```

PART TWENTY-TWO: ADVANCED NETWORKING CONCEPTS

Chapter 67: Public vs Private Subnets (Deep Dive)

What are Subnets?

Subnets are subdivisions of a VPC's IP address range. They group resources and control network access.

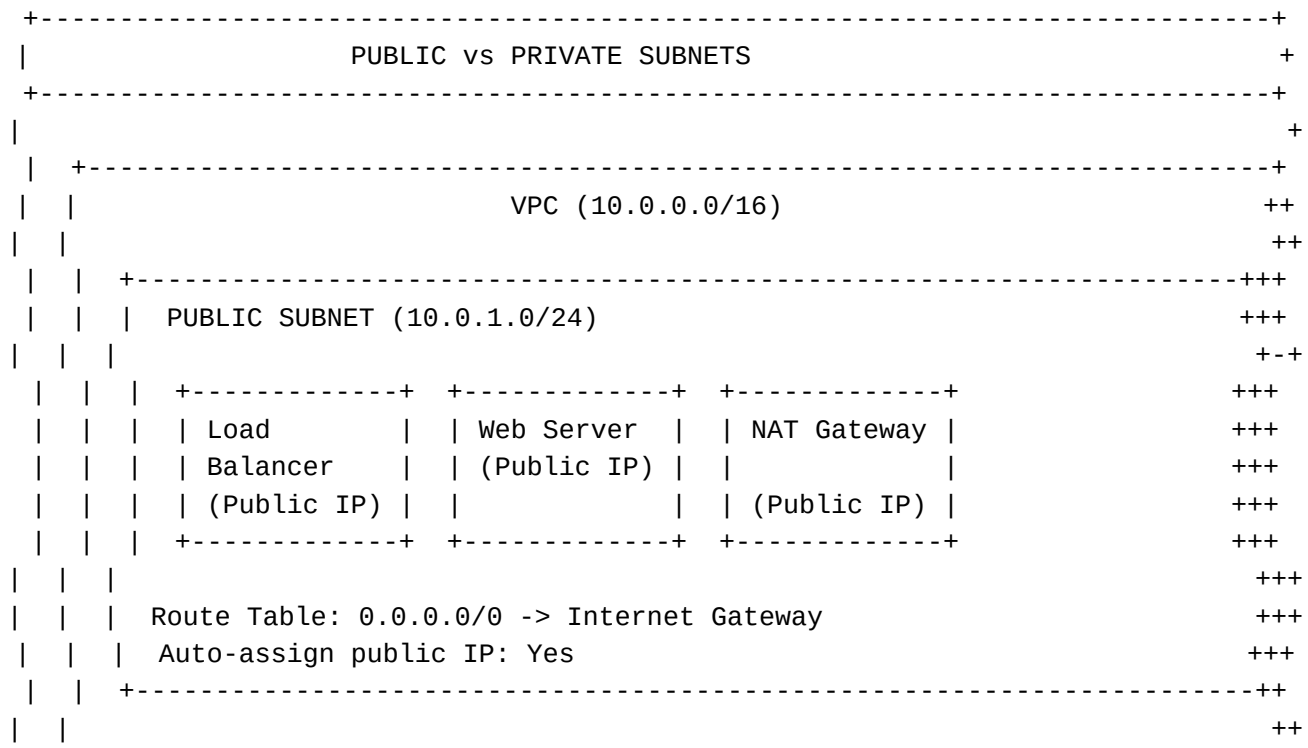
Public Subnet

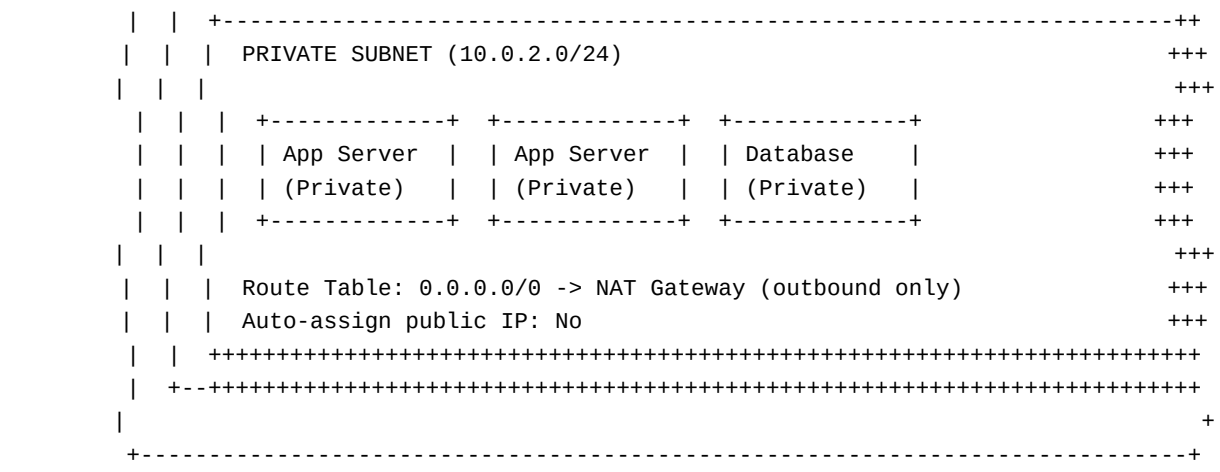
- Has direct route to Internet Gateway
- Resources get public IPs (optional)
- Can receive inbound traffic from internet
- Use for: Load balancers, web servers, NAT gateways

Private Subnet

- No direct route to Internet Gateway
- Resources have private IPs only
- Cannot receive inbound traffic from internet
- Use for: Application servers, databases

Diagram: Public vs Private Subnets





Subnet Use Cases

Type	Use Case	Internet Access	Public IP
Public	Load balancers	Inbound + Outbound	Yes
Public	Web servers	Inbound + Outbound	Yes (or via LB)
Public	NAT gateways	Outbound only	Yes
Private	Application servers	Outbound only (via NAT)	No
Private	Databases	None	No
Private	Redis/ElastiCache	None	No

[Page 68]

Chapter 68: NAT Gateway

What is NAT Gateway?

Network Address Translation (NAT) Gateway allows instances in private subnets to access the internet (for updates, patches, external APIs) while preventing inbound traffic from the internet.

Real Scenario

Your application servers in private subnets need to download security patches and call external APIs, but should not receive inbound internet traffic.

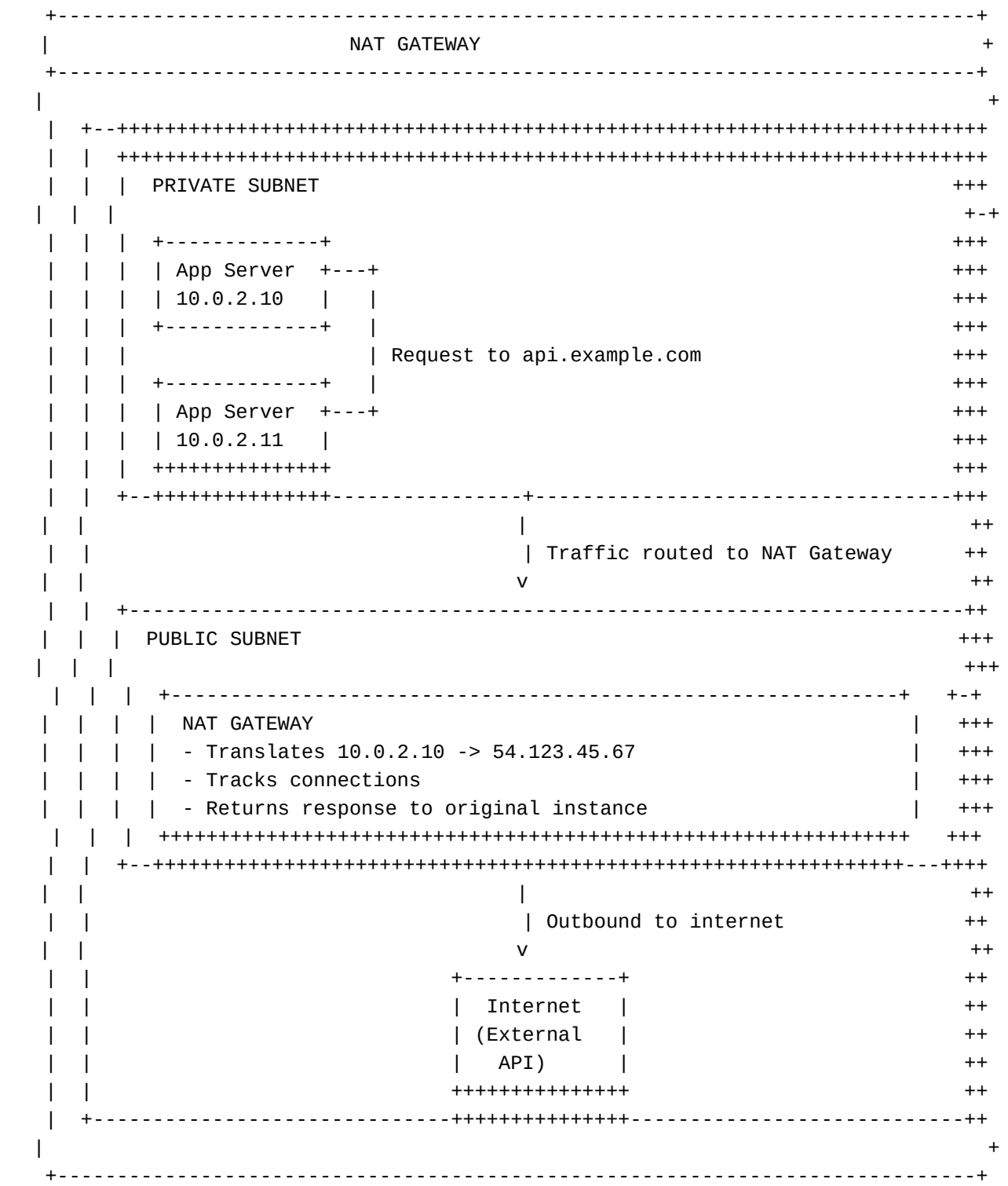
Without NAT Gateway:

- Private subnets can't reach internet
- Can't download updates
- Can't call external APIs

With NAT Gateway:

- Private instances can initiate outbound connections
- Inbound connections blocked
- Single public IP for all private instances

Diagram: NAT Gateway Flow



NAT Gateway Terraform

```
# Create Elastic IP for NAT Gateway
resource "aws_eip" "nat" {
  domain = "vpc"
  tags = {
    Name = "nat-gateway-eip"
  }
}

# Create NAT Gateway in public subnet
resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id

  tags = {
    Name = "main-nat-gateway"
  }

  depends_on = [aws_internet_gateway.main]
}

# Update private subnet route table
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.main.id
  }

  tags = {
    Name = "private-route-table"
  }
}
```

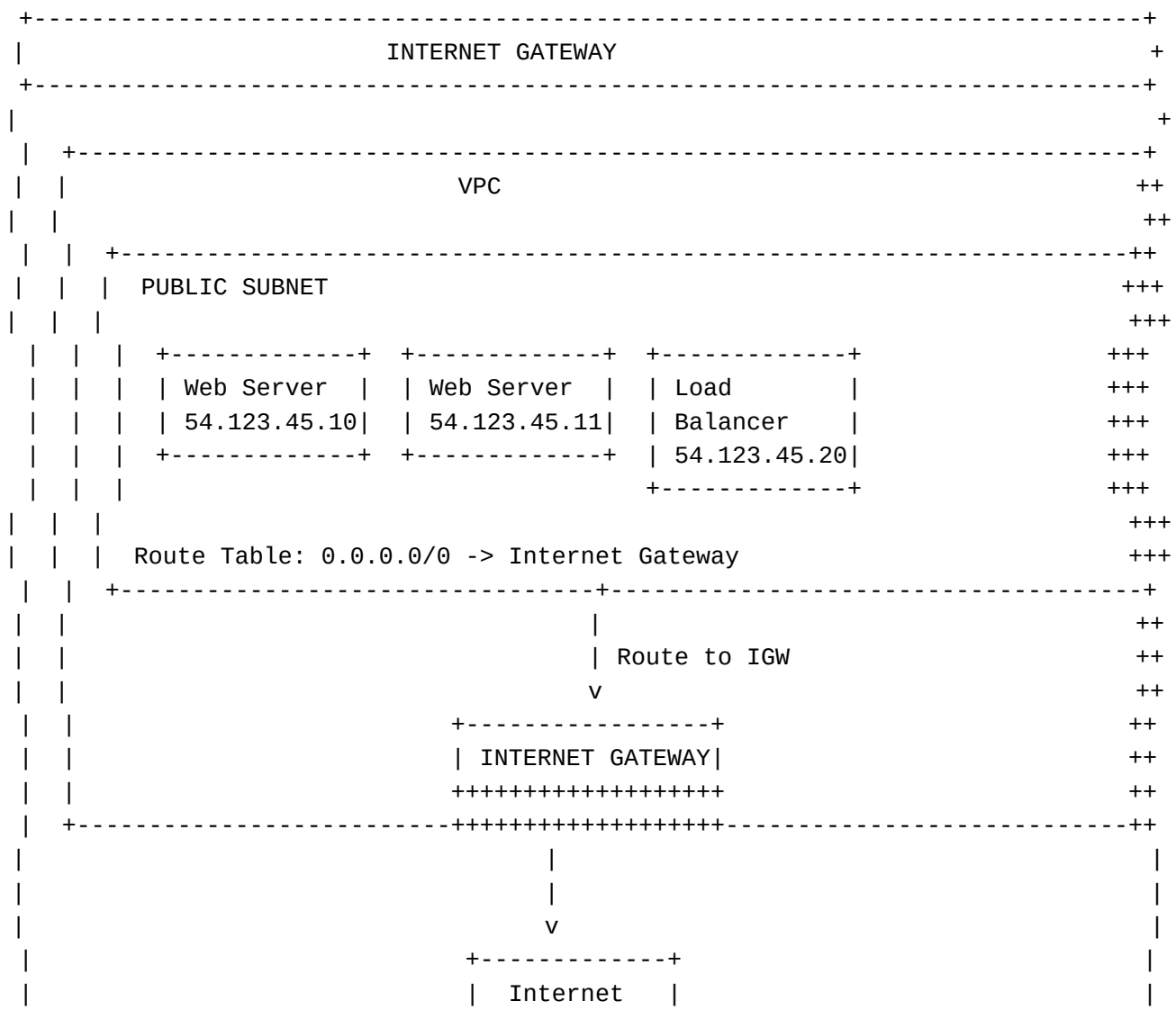
NAT Gateway vs NAT Instance

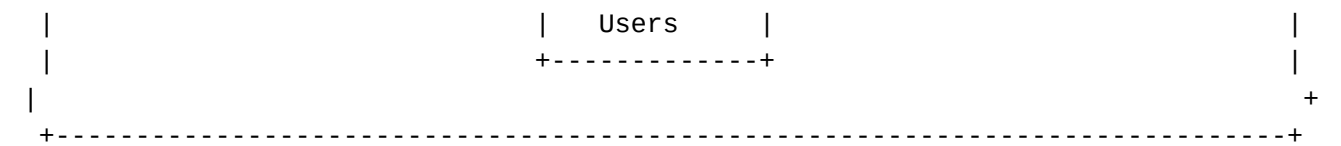
Aspect	NAT Gateway	NAT Instance
--------	------------	-------------
Managed	AWS managed	You manage
Availability	Highly available (within AZ)	You configure
Bandwidth	Up to 45 Gbps	Depends on instance type
Cost	Per hour + data processing	Instance cost + data
Maintenance	None	OS updates, patching
Scaling	Automatic	Manual
Recommendation	Preferred	Legacy

[Page 69]

Chapter 69: Internet Gateway

What is Internet Gateway?
Horizontally scaled, redundant, and highly available VPC component that allows communication between your VPC and the internet.
Real Scenario
Your web servers need to serve traffic to users on the internet.
Without Internet Gateway:
- No inbound internet access
- Resources can't be reached from internet
With Internet Gateway:
- Public subnets can receive inbound internet traffic
- Resources can have public IPs
- Required for public-facing applications
Diagram: Internet Gateway





Internet Gateway Terraform

```
# Create Internet Gateway
resource "aws_internet_gateway" "main" {
vpc_id = aws_vpc.main.id
tags = {
                                    Name = "main-igw"
}
}
# Public route table with IGW route
resource "aws_route" "internet_access" {
route_table_id         = aws_route_table.public.id
destination_cidr_block = "0.0.0.0/0"
gateway_id             = aws_internet_gateway.main.id
}
```

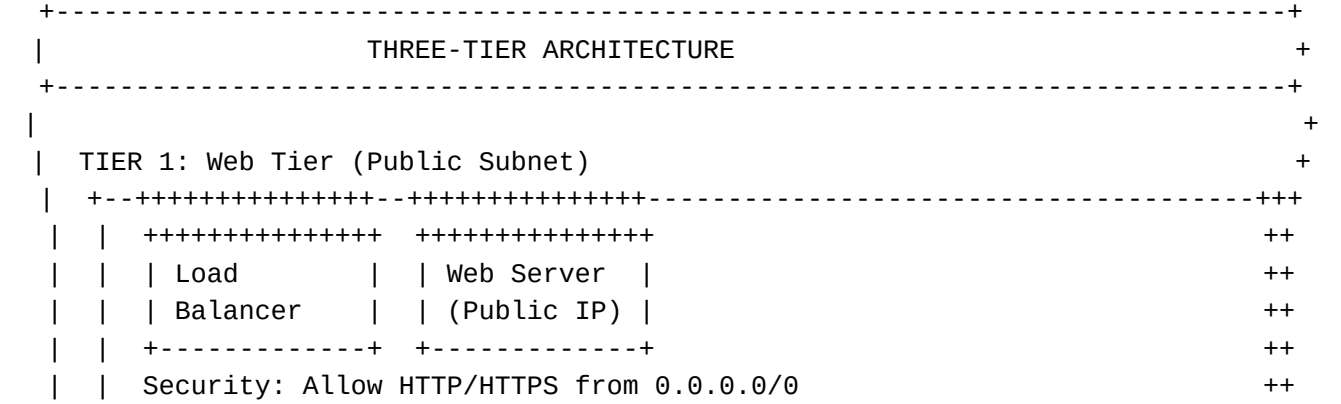
Internet Gateway vs NAT Gateway

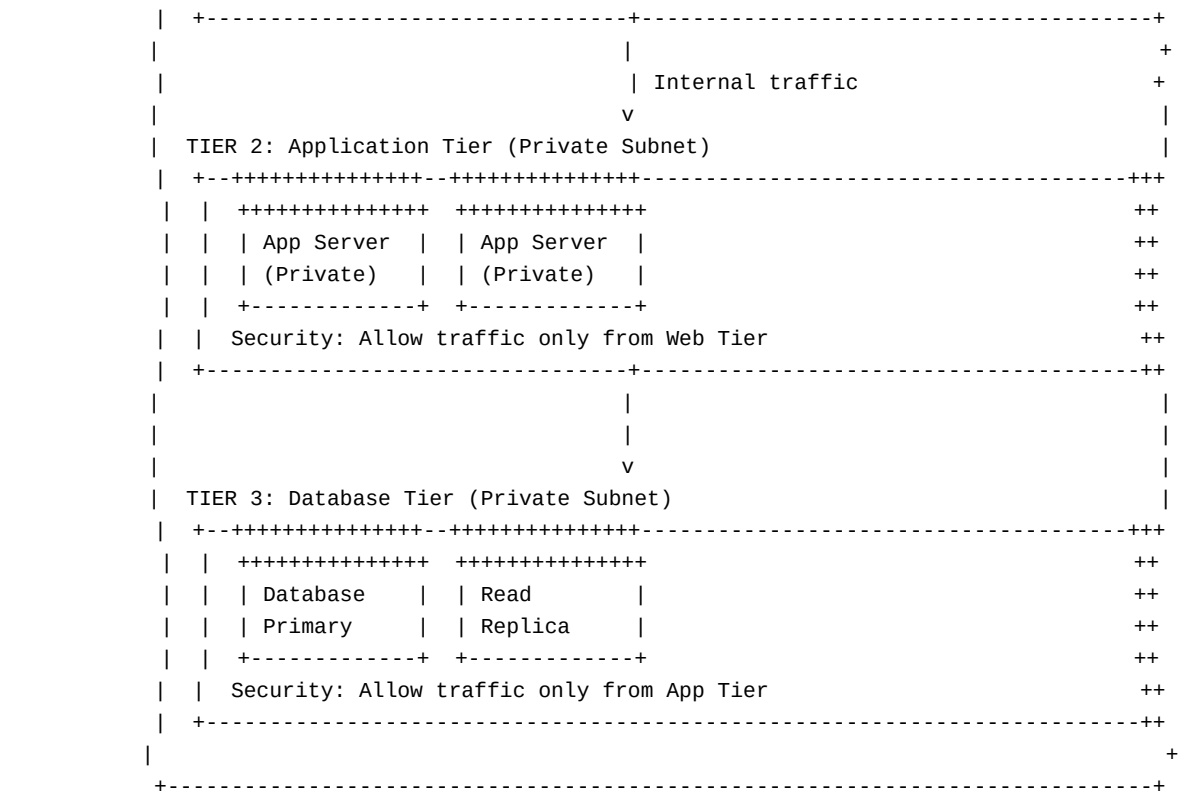
Aspect	Internet Gateway	NAT Gateway
Direction	Inbound + Outbound	Outbound only
Location	Public subnets	Public subnets (for NAT)
Use case	Public resources	Private resources accessing internet
IP translation	1:1 (public IP)	Many:1 (NAT)

[Page 70]

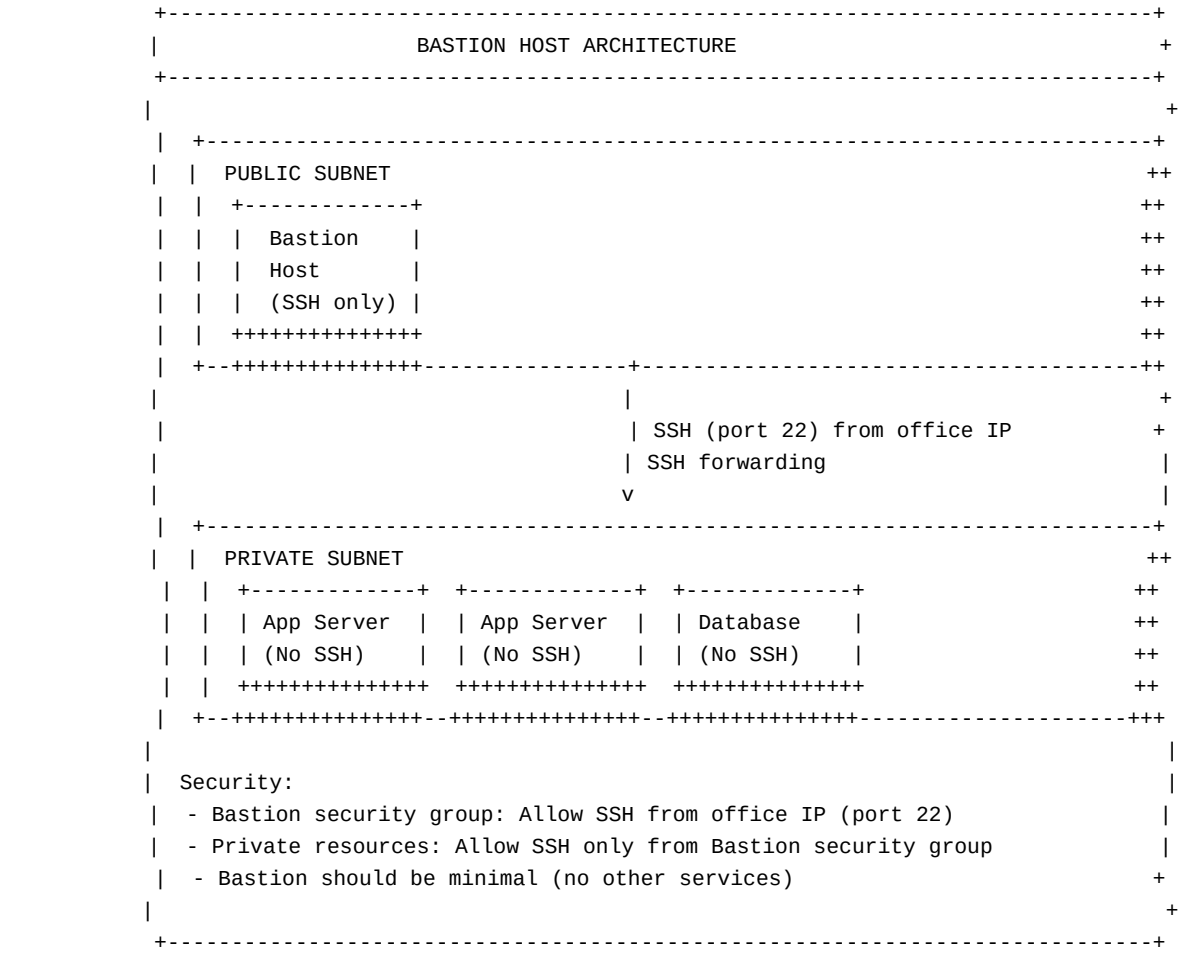
Chapter 70: Secure Network Design Patterns

Pattern 1: Three-Tier Architecture

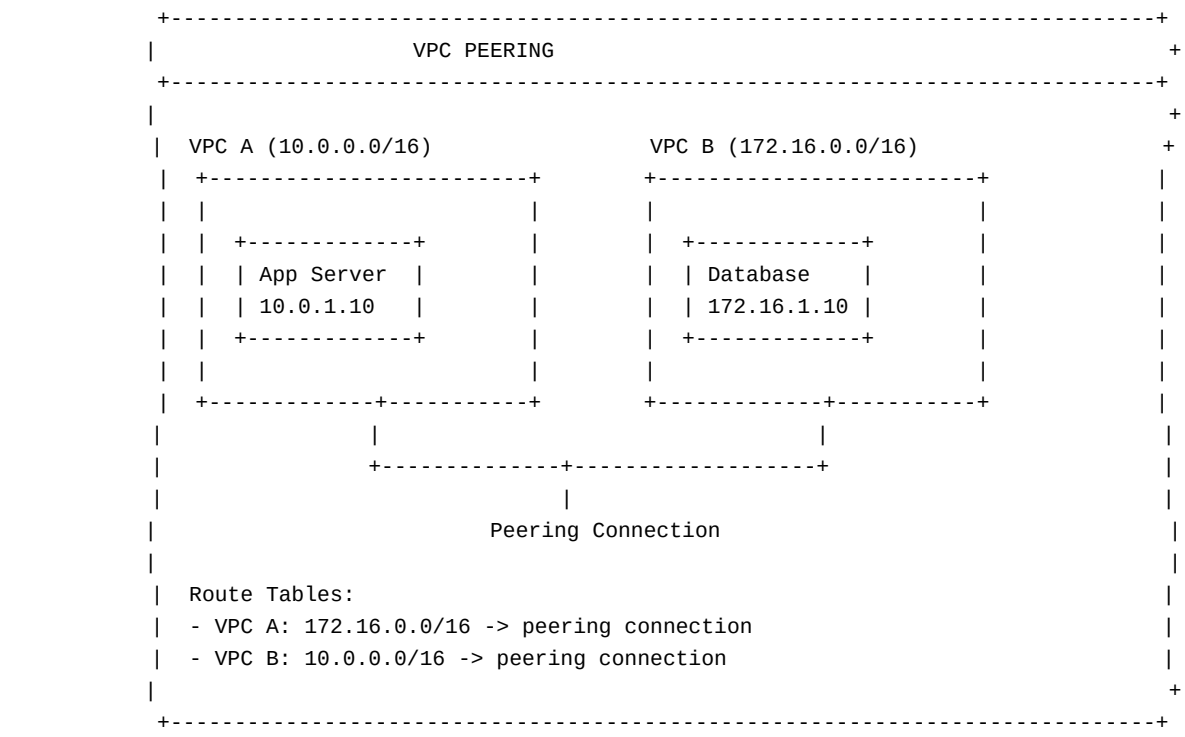




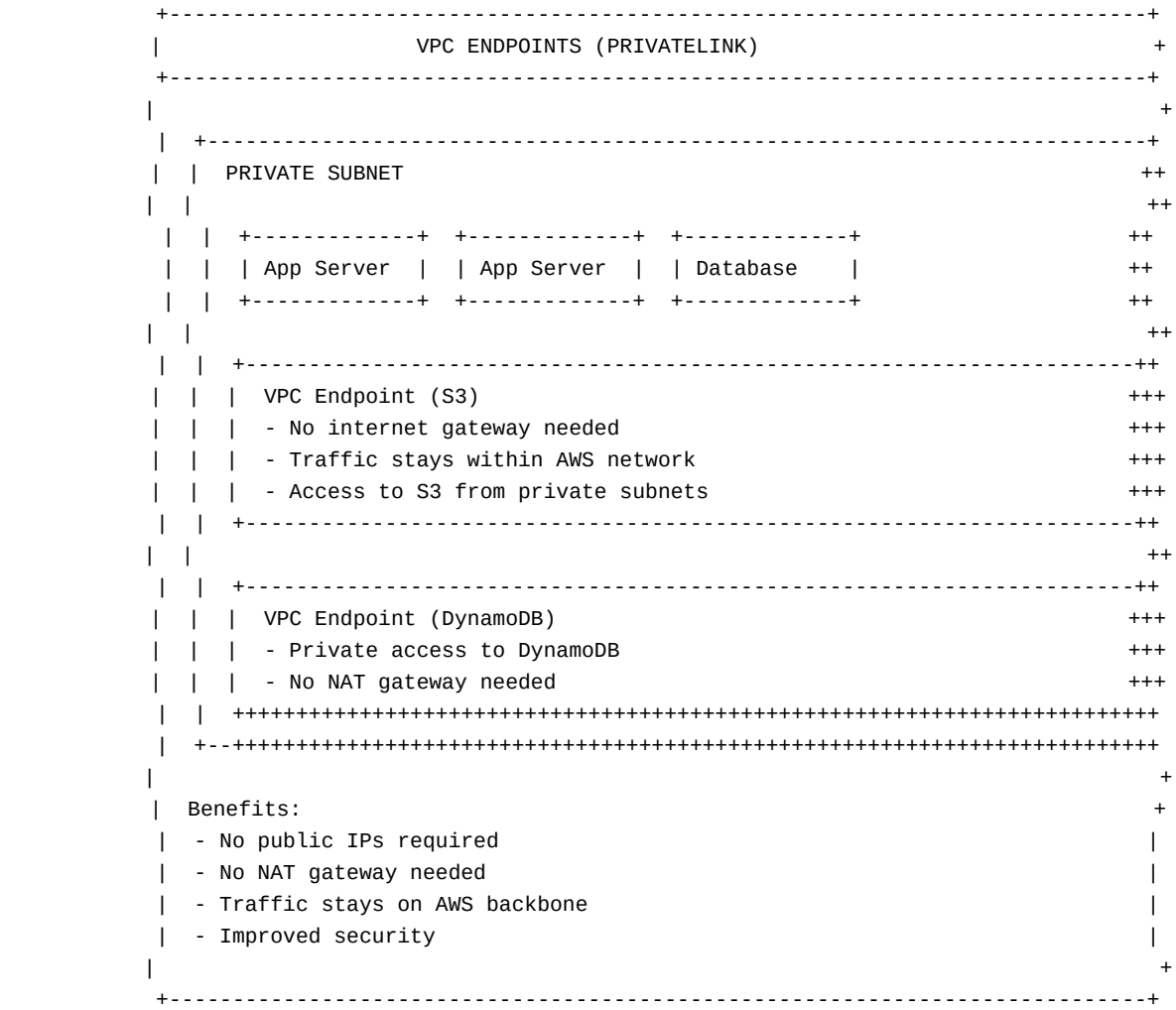
Pattern 2: Bastion Host (Jump Box)



Pattern 3: VPC Peering (Cross-VPC Communication)



Pattern 4: VPC Endpoints (PrivateLink)



VPC Endpoint Terraform

```
# Gateway endpoint for S3
resource "aws_vpc_endpoint" "s3" {
  vpc_id       = aws_vpc.main.id
  service_name = "com.amazonaws.us-east-1.s3"

  route_table_ids = [aws_route_table.private.id]

  tags = {
    Name = "s3-vpc-endpoint"
  }
}

# Interface endpoint for DynamoDB
resource "aws_vpc_endpoint" "dynamodb" {
  vpc_id              = aws_vpc.main.id
  service_name        = "com.amazonaws.us-east-1.dynamodb"
  vpc_endpoint_type   = "Interface"
  subnet_ids          = aws_subnet.private[*].id
  security_group_ids  = [aws_security_group.vpc_endpoint.id]

  tags = {
    Name = "dynamodb-vpc-endpoint"
  }
}
```

[Page 71]

PART TWENTY-THREE: COST OPTIMIZATION STRATEGIES

Chapter 71: Spot Instances

What are Spot Instances?

EC2 instances that use unused capacity at up to 90% discount, but can be reclaimed with 2-minute notice.

Real Scenario

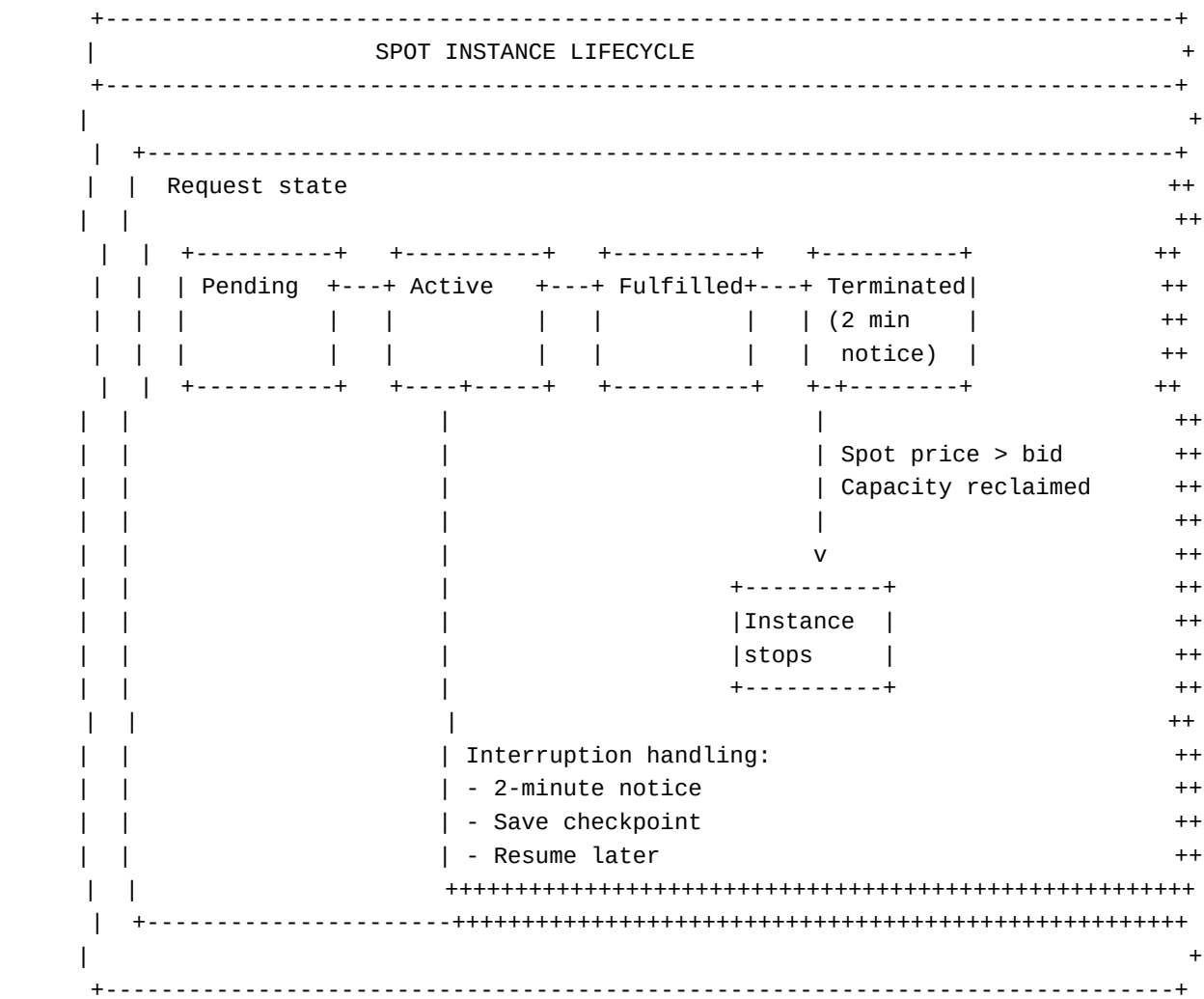
You have batch processing jobs that can be interrupted. You want to save costs.

Without Spot:

- On-Demand instances: $0.10/hour

- 1000 instances × 100 hours = $10,000

With Spot:
- Spot instances: $0.01/hour (90% savings)
- 1000 instances × 100 hours = $1,000
Diagram: Spot Instance Lifecycle



Spot Instance Terraform

```
resource "aws_spot_instance_request" "batch" {
ami                 = "ami-0c55b159cbfafe1f0"
instance_type       = "c5.large"
spot_price          = "0.05"
spot_type           = "one-time"
wait_for_fulfillment   = true

tags = {
                                    Name = "spot-batch-worker"
                                   Purpose = "batch-processing"
}

                                       user_data = <<-EOF
#!/bin/bash
# Install checkpointing for interruption handling
                                              apt-get update
                                    apt-get install -y checkpoint-criu

# Start batch job with checkpoint support
                                    /usr/local/bin/start-batch-job.sh
```

```
  EOF
}

# Spot fleet (multiple instance types)
resource "aws_spot_fleet_request" "cheap_compute" {
  iam_fleet_role      = aws_iam_role.spot_fleet.arn
  spot_price          = "0.10"
  target_capacity     = 10
  valid_until         = "2024-12-31T00:00:00Z"
  terminate_instances_with_expiration = true

  launch_specification {
    instance_type = "c5.large"
    ami           = "ami-0c55b159cbfafe1f0"
    subnet_id     = aws_subnet.private[0].id
  }

  launch_specification {
    instance_type = "c4.large"
    ami           = "ami-0c55b159cbfafe1f0"
    subnet_id     = aws_subnet.private[0].id
  }

  launch_specification {
    instance_type = "m5.large"
    ami           = "ami-0c55b159cbfafe1f0"
    subnet_id     = aws_subnet.private[0].id
  }
}
```

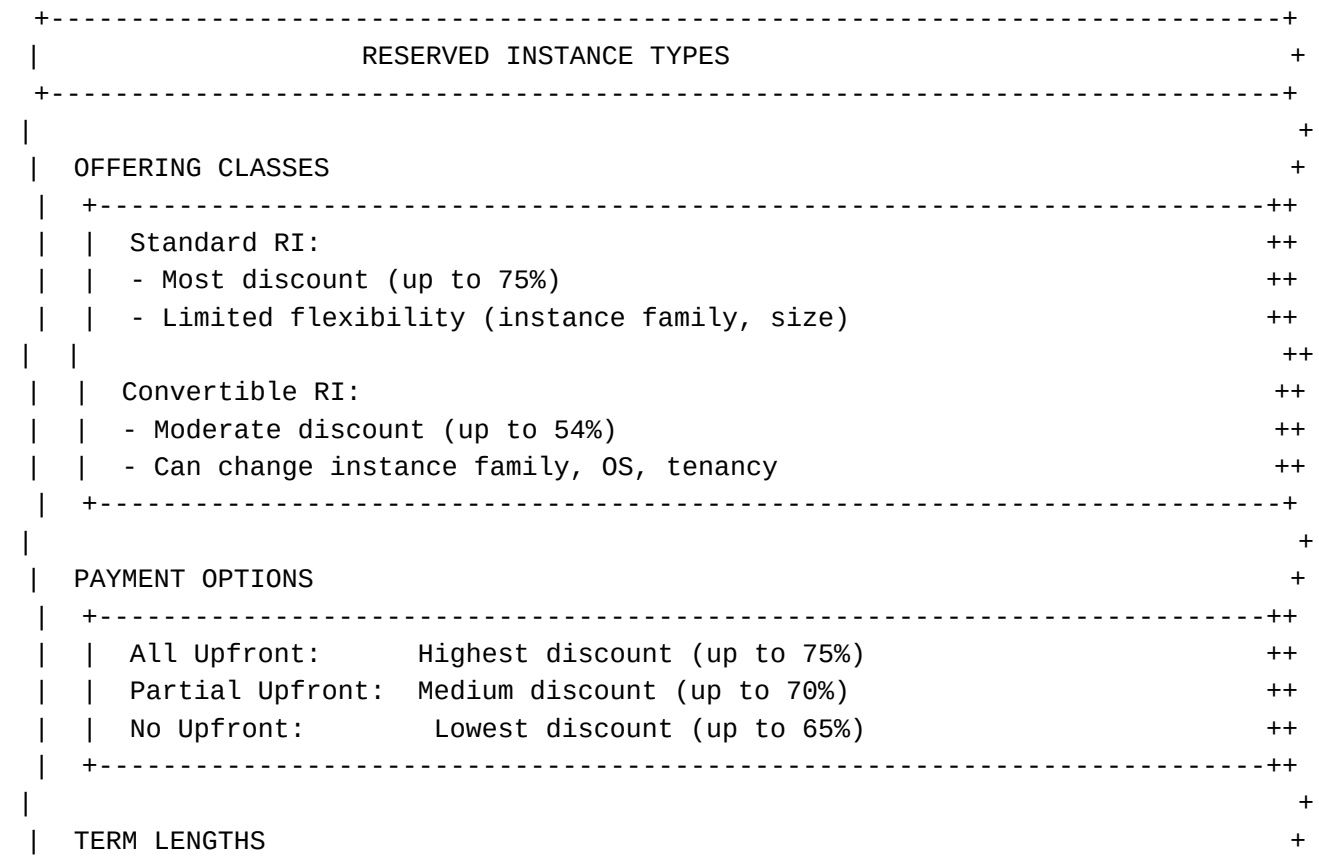
When to Use Spot Instances

Use Case	Suitability
----------	----------
Batch processing	■ Highly suitable
CI/CD workers	■ Suitable
Dev/test environments	■ Suitable
Stateless web apps	■■ Can use with ASG mixed
Stateful databases	■ Not suitable

Spot Instance Best Practices

Practice	Why
----------	-----
Use multiple instance types	More capacity available
Use multiple AZs	Better availability
Implement checkpointing	Save progress on interruption
Use Spot Fleet	Manages diversity
Set max price	Control costs
Handle interruption gracefully	2-minute notice

[Page 72]
Chapter 72: Reserved Instances
What are Reserved Instances?
Discount commitment for 1 or 3 years, offering up to 75% savings compared to On-Demand.
Real Scenario
Your production workload runs 24/7. You want to reduce costs.
Without Reserved Instances:
- On-Demand: 10 × t3.large × 24 hours × 30 days × $0.0832 = $5,990/month
With Reserved Instances (3-year, all upfront):
- Reserved: 10 × t3.large × $0.0208/hour = $1,498/month
- **Savings: 75%**
Diagram: Reserved Instance Types



```
 |  +-------------------------------------------------------------------------++
 |  |  1 Year:     Good for stable workloads                                   ++
 |  |  3 Years:    Best discount, for long-term workloads                      ++
 |  +-------------------------------------------------------------------------++
|                                                                               +
 +-------------------------------------------------------------------------------+
```

Reserved Instance Purchase (AWS CLI)

```
# Purchase Standard RI (1 year, all upfront)
aws ec2 purchase-reserved-instances-offering \
--reserved-instances-offering-id 1234567890 \
--instance-count 10 \
--limit-price 0.50
# Purchase Convertible RI (3 year, partial upfront)
aws ec2 purchase-reserved-instances-offering \
--reserved-instances-offering-id 0987654321 \
--instance-count 5 \
--offering-class convertible
```

Reserved Instance Recommendations (Cost Explorer)

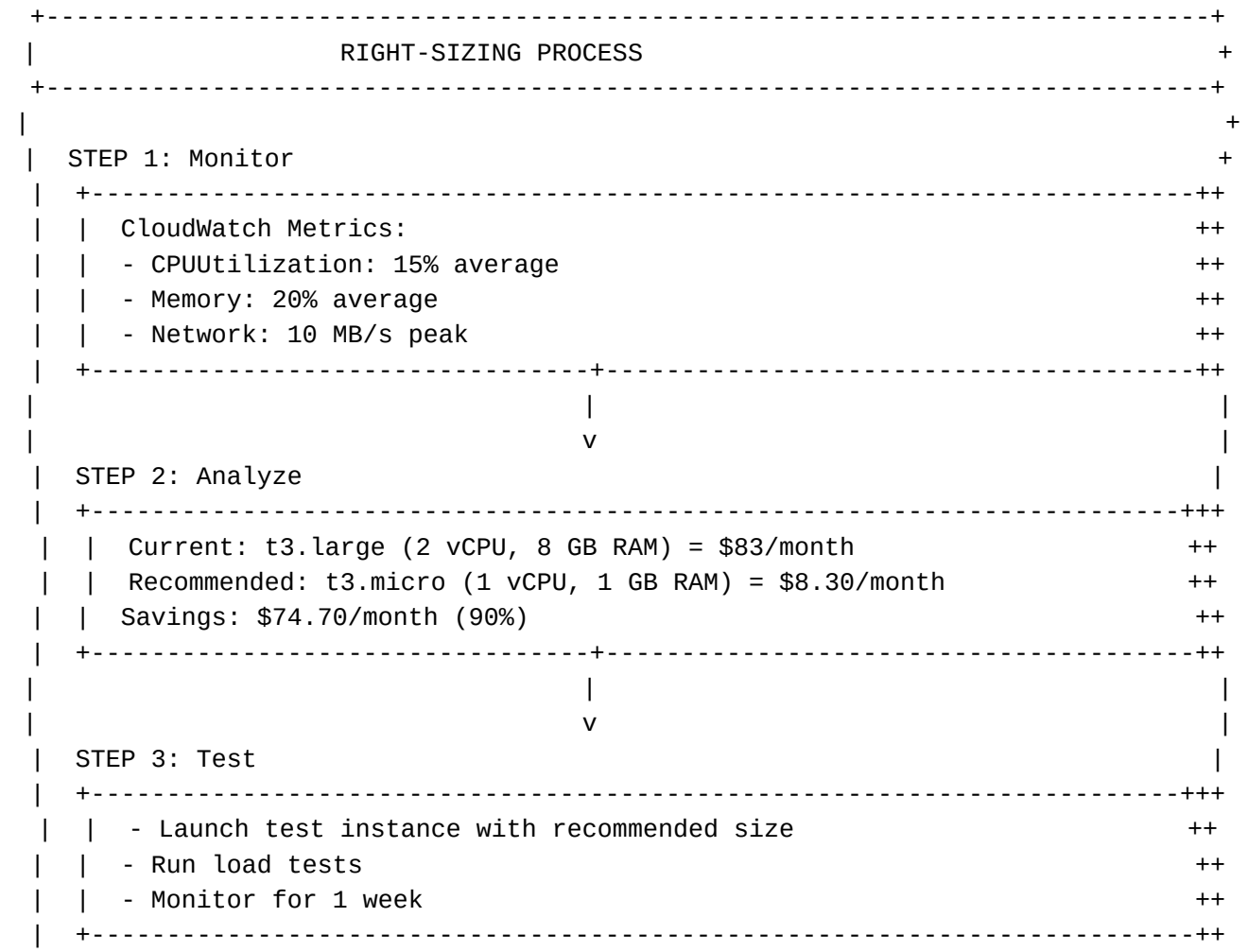
```
# Get RI recommendations
aws ce get-reservation-purchase-recommendation \
--service "Amazon Elastic Compute Cloud - Compute"
# Get RI utilization
aws ce get-reservation-utilization \
--time-period Start=2024-01-01,End=2024-01-31 \
--group-by Type=DIMENSION,Key=RESERVATION_ID
```

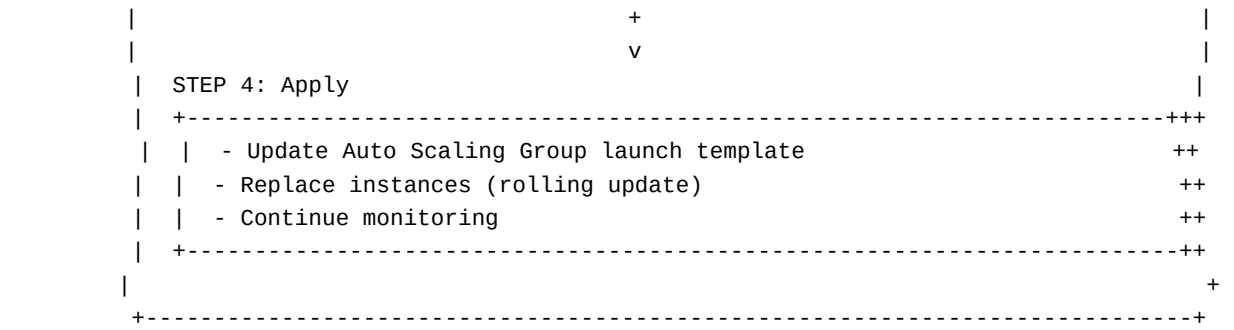
When to Use Reserved Instances

Workload
Recommendation

24/7 production
```
                         + Standard RI (3 year, all upfront)
```
Seasonal workload
```
                             ++ Convertible RI (1 year)
```
Variable workload
```
                               + Use Spot + On-Demand
```
New application
```
                           + Start with On-Demand, then reserve
```
Container workloads
```
                             + Savings Plans (more flexible)
```

What is Right-Sizing?
Optimizing resource size (instance type, storage, etc.) to match actual workload requirements.
Real Scenario
You provisioned t3.large instances because you thought you needed 2 vCPUs and 8 GB RAM. Monitoring shows CPU at 10%, memory at 20%.
Without Right-Sizing:
- Over-provisioned resources
- Paying for unused capacity
- 10 × t3.large = $830/month
With Right-Sizing:
- Downsize to t3.micro
- 10 × t3.micro = $83/month
- **Savings: 90%**
Diagram: Right-Sizing Process





Compute Optimizer (Right-Sizing Recommendations)

```
# Enable Compute Optimizer
resource "aws_computeoptimizer_recommendation_preferences" "example" {
resource_type = "Ec2Instance"

scope {
name  = "AccountId"
value = "123456789012"
}

preferred_resource {
name      = "instanceType"
include   = ["m5.large", "m5.xlarge", "c5.large"]
exclude   = ["t2.*", "t3.nano"]
}
}
```

Right-Sizing Tools

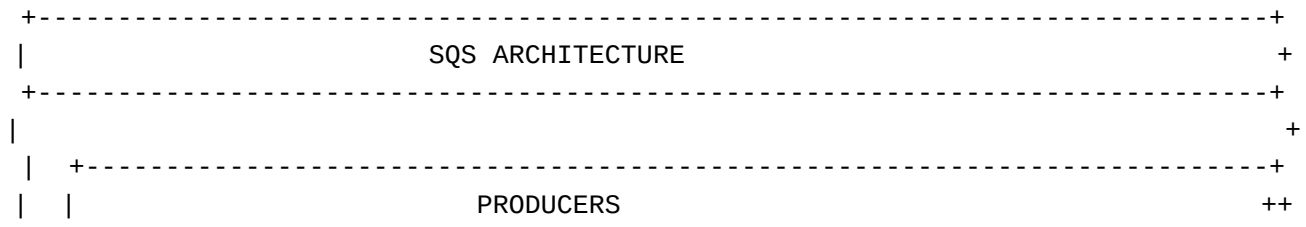
Tool	Purpose
AWS Compute Optimizer	ML-based recommendations
CloudWatch Metrics	Current utilization
Cost Explorer	Cost analysis
Trusted Advisor	Idle resources
Third-party (CloudHealth)	Advanced analytics

Right-Sizing Best Practices

Practice	Why
Monitor for 2 weeks	Capture peak usage
Consider future growth	Don't over-optimize
Test before changing	Avoid performance issues
Use Auto Scaling	Scale with demand
Review quarterly	Workloads change
Use Spot for batch	Maximize savings

Practice
Why
Consider Graviton
Better price/performance

[Page 74]
PART TWENTY-FOUR: EVENT-DRIVEN ARCHITECTURE
Chapter 74: SQS (Simple Queue Service)
What is SQS?
Fully managed message queuing service for decoupling application components.
Real Scenario
Your order processing system receives 1000 orders per second. You need to process them without overwhelming the backend.
Without SQS:
- Direct calls to backend (synchronous)
- Backend gets overwhelmed
- Orders dropped
With SQS:
- Orders go to queue
- Workers pull at their own pace
- No orders dropped
Diagram: SQS Architecture





SQS Terraform

```
# Standard queue (high throughput)
resource "aws_sqs_queue" "standard_queue" {
name                  = "order-queue"
delay_seconds         = 0
max_message_size      = 262144
message_retention_seconds = 86400
receive_wait_time_seconds = 20
visibility_timeout_seconds = 30
redrive_policy = jsonencode({
                        deadLetterTargetArn = aws_sqs_queue.dead_letter_queue.arn
maxReceiveCount     = 3
})
tags = {
Environment = "production"
}
}
# FIFO queue (strict ordering)
resource "aws_sqs_queue" "fifo_queue" {
name                  = "order-queue.fifo"
fifo_queue            = true
content_based_deduplication = true
visibility_timeout_seconds  = 30
tags = {
```

```
    Environment = "production"
  }
}

# Dead letter queue
resource "aws_sqs_queue" "dead_letter_queue" {
  name = "order-queue-dlq"
  message_retention_seconds = 1209600  # 14 days
}
```

SQS Lambda Integration

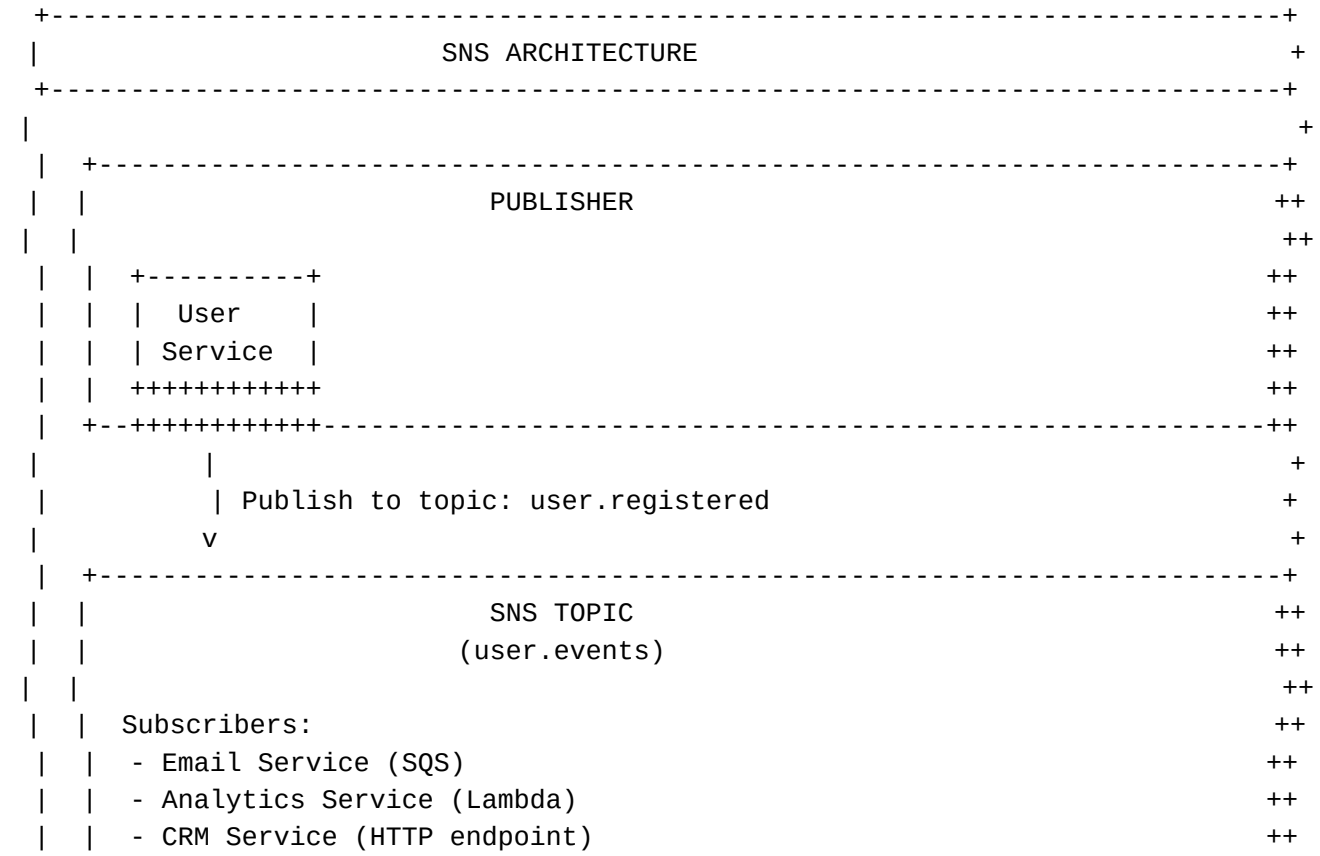
```
# Lambda function with SQS trigger
ProcessOrderFunction:
  Type: AWS::Lambda::Function
  Properties:
    Handler: index.handler
    Runtime: nodejs18.x
    Code:
      S3Bucket: my-bucket
      S3Key: function.zip
    Environment:
      Variables:
        QUEUE_URL: !Ref OrderQueue

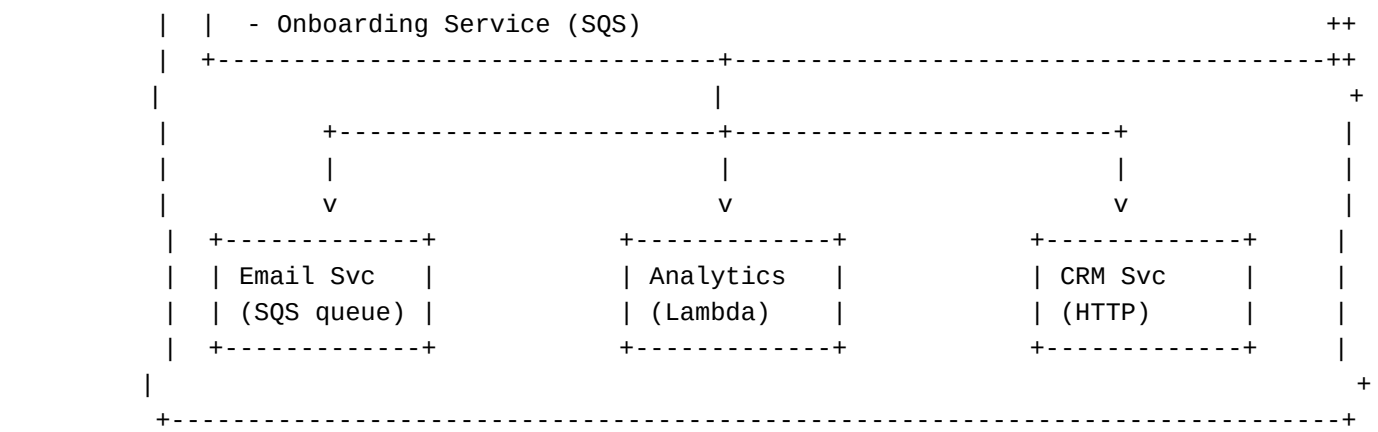
# SQS trigger for Lambda
EventSourceMapping:
  Type: AWS::Lambda::EventSourceMapping
  Properties:
    EventSourceArn: !GetAtt OrderQueue.Arn
    FunctionName: !Ref ProcessOrderFunction
    BatchSize: 10
    MaximumBatchingWindowInSeconds: 60
```

SQS Best Practices

Practice	**Why**
----------	-----
Use DLQ for failures	Prevent poison messages
Set visibility timeout	Based on processing time
Use long polling	Reduce empty responses
Monitor queue depth	Alert on backlog
Batch operations	Reduce API calls
Use FIFO for ordering	When order matters
Idempotent consumers	Handle duplicate messages

Chapter 75: SNS (Pub/Sub)
What is SNS?
Fully managed publish/subscribe messaging service for broadcasting messages to multiple subscribers.
Real Scenario
When a new user registers, you need to send welcome email, update analytics, notify CRM, and trigger onboarding.
Without SNS:
- User service calls each service directly (tight coupling)
- Adding new subscriber requires code change
- Subscriber failures affect user service
With SNS:
- User service publishes one message to topic
- All subscribers receive message independently
- Add new subscribers without code change
Diagram: SNS Architecture





SNS Terraform

```
# Create SNS topic
resource "aws_sns_topic" "user_events" {
name = "user-events"
tags = {
Environment = "production"
}
}
# SQS subscription (email service)
resource "aws_sns_topic_subscription" "email_subscription" {
topic_arn = aws_sns_topic.user_events.arn
protocol  = "sqs"
endpoint  = aws_sqs_queue.email_queue.arn
filter_policy = jsonencode({
                    event_type = ["user.registered", "user.updated"]
})
}
# Lambda subscription (analytics)
resource "aws_sns_topic_subscription" "analytics_subscription" {
topic_arn = aws_sns_topic.user_events.arn
protocol  = "lambda"
endpoint  = aws_lambda_function.analytics.arn
filter_policy = jsonencode({
                            event_type = ["user.registered"]
})
}
# HTTP subscription (CRM)
resource "aws_sns_topic_subscription" "crm_subscription" {
topic_arn = aws_sns_topic.user_events.arn
protocol  = "https"
endpoint  = "https://crm.example.com/webhook"
delivery_policy = jsonencode({
"healthyRetryPolicy" = {
"numRetries" = 3,
"minDelayTarget" = 1,
"maxDelayTarget" = 10,
"numNoDelayRetries" = 0,
"numMinDelayRetries" = 0,
"numMaxDelayRetries" = 0,
"backoffFunction" = "linear"
}
})
```

```
}
```

SNS Message Filtering

```
{
  "event_type": ["user.registered"],
  "user": {
    "plan": ["premium"]
  },
  "region": ["us-east-1", "eu-west-1"]
}
```

SQS vs SNS

Aspect	SQS	SNS
--------	-----	-----
Pattern	Queue (point-to-point)	Topic (pub/sub)
Consumers	One consumer per message	All subscribers
Delivery	Pull-based	Push-based (to subscribers)
Persistence	Messages stored (14 days)	Not stored (fanout)
Use case	Work distribution	Event notification

[Page 76]

Chapter 76: EventBridge

What is EventBridge?

Serverless event bus that connects applications using events from AWS services, SaaS partners, and custom applications.

Real Scenario

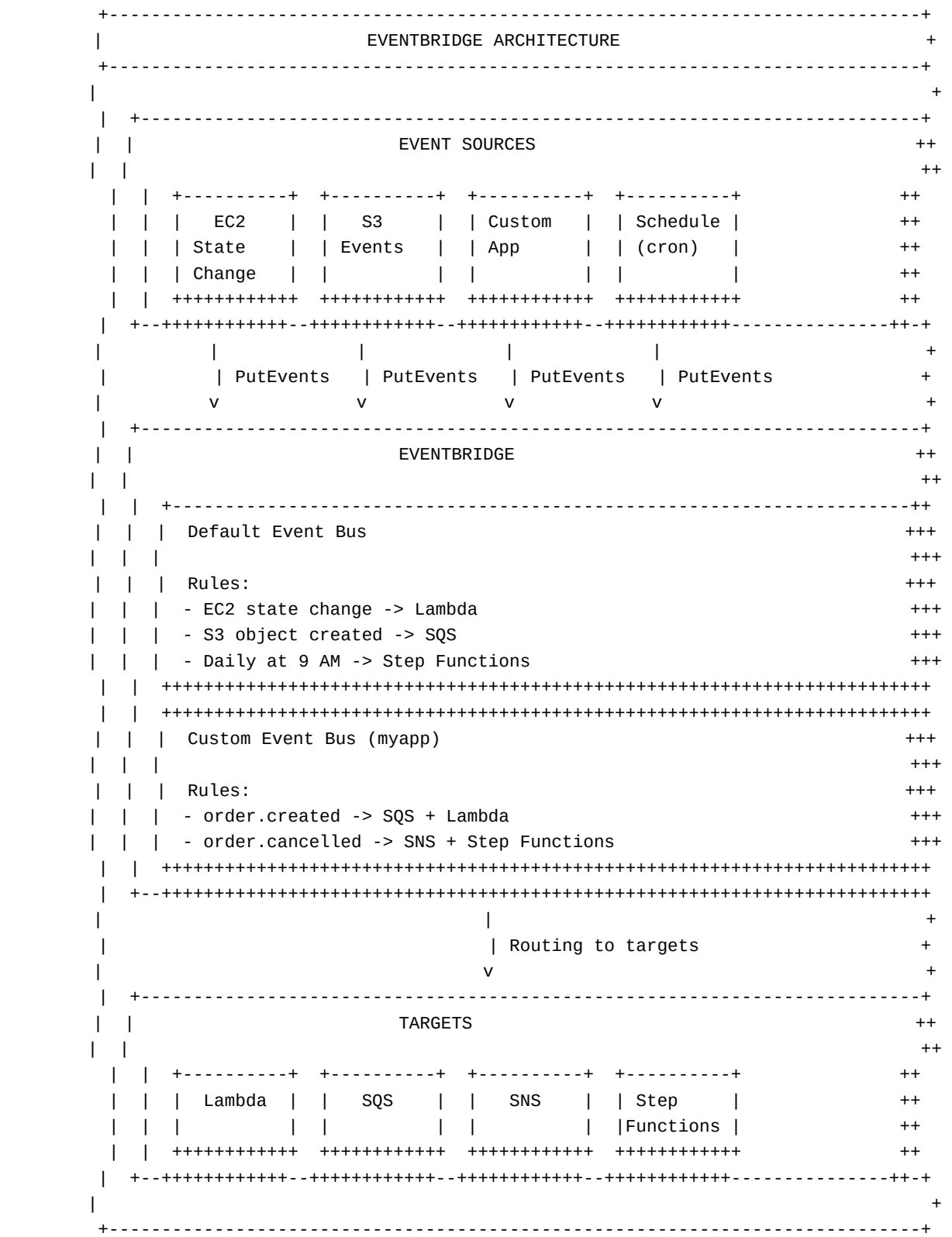
You need to react to EC2 instance state changes, S3 bucket events, and scheduled cron jobs, with routing to multiple targets.

Without EventBridge:

- Multiple CloudWatch rules

- Complex routing logic

- No event schema validation
With EventBridge:
- Single event bus
- Powerful filtering
- Schema registry
Diagram: EventBridge Architecture



EventBridge Rules

```
# EC2 state change rule
resource "aws_cloudwatch_event_rule" "ec2_state_change" {
  name        = "ec2-state-change"
  description = "Capture EC2 instance state changes"

  event_pattern = jsonencode({
    source = ["aws.ec2"]
    detail-type = ["EC2 Instance State-change Notification"]
    detail = {
      state = ["running", "stopped", "terminated"]
    }
  })
}

resource "aws_cloudwatch_event_target" "ec2_lambda" {
  rule      = aws_cloudwatch_event_rule.ec2_state_change.name
  target_id = "SendToLambda"
  arn       = aws_lambda_function.process_ec2_change.arn
}

# Scheduled rule (cron)
resource "aws_cloudwatch_event_rule" "daily_report" {
  name                = "daily-report"
  description         = "Trigger daily report at 9 AM UTC"
  schedule_expression = "cron(0 9 * * ? *)"
}

resource "aws_cloudwatch_event_target" "daily_step_functions" {
  rule      = aws_cloudwatch_event_rule.daily_report.name
  target_id = "StartStepFunctions"
  arn       = aws_sfn_state_machine.report_generation.arn
  role_arn  = aws_iam_role.eventbridge_stepfunctions.arn
}
```

Custom Event Bus

```
# Custom event bus for application events
resource "aws_cloudwatch_event_bus" "app_bus" {
  name = "myapp-events"
}

# Rule on custom bus
resource "aws_cloudwatch_event_rule" "order_created" {
  name           = "order-created"
  event_bus_name = aws_cloudwatch_event_bus.app_bus.name

  event_pattern = jsonencode({
    source = ["myapp.order"]
    detail-type = ["OrderCreated"]
  })
}

resource "aws_cloudwatch_event_target" "order_sqs" {
  rule           = aws_cloudwatch_event_rule.order_created.name
  event_bus_name = aws_cloudwatch_event_bus.app_bus.name
  target_id      = "SendToSQS"
  arn            = aws_sqs_queue.order_queue.arn
}
```

EventBridge vs SNS vs SQS

Aspect	EventBridge	SNS	SQS
--------	----------	-----	-----
Pattern	Event bus	Pub/Sub	Queue
Filtering	Rich (JSONPath)	Simple (attributes)	None
Schema validation	Yes (Schema Registry)	No	No
Targets	20+ AWS services	SQS, Lambda, HTTP	Lambda, EC2
Scheduling	Built-in	No	No
Use case	Complex event routing	Simple pub/sub	Work queues

[Page 77]

PART TWENTY-FIVE: MULTI-ACCOUNT & DISASTER RECOVERY

Chapter 77: AWS Organizations

What is AWS Organizations?

Centralized management for multiple AWS accounts, enabling consolidated billing, policy management, and governance.

Real Scenario

Your company has multiple teams (dev, prod, security, analytics). You need centralized billing and security policies.

Without Organizations:

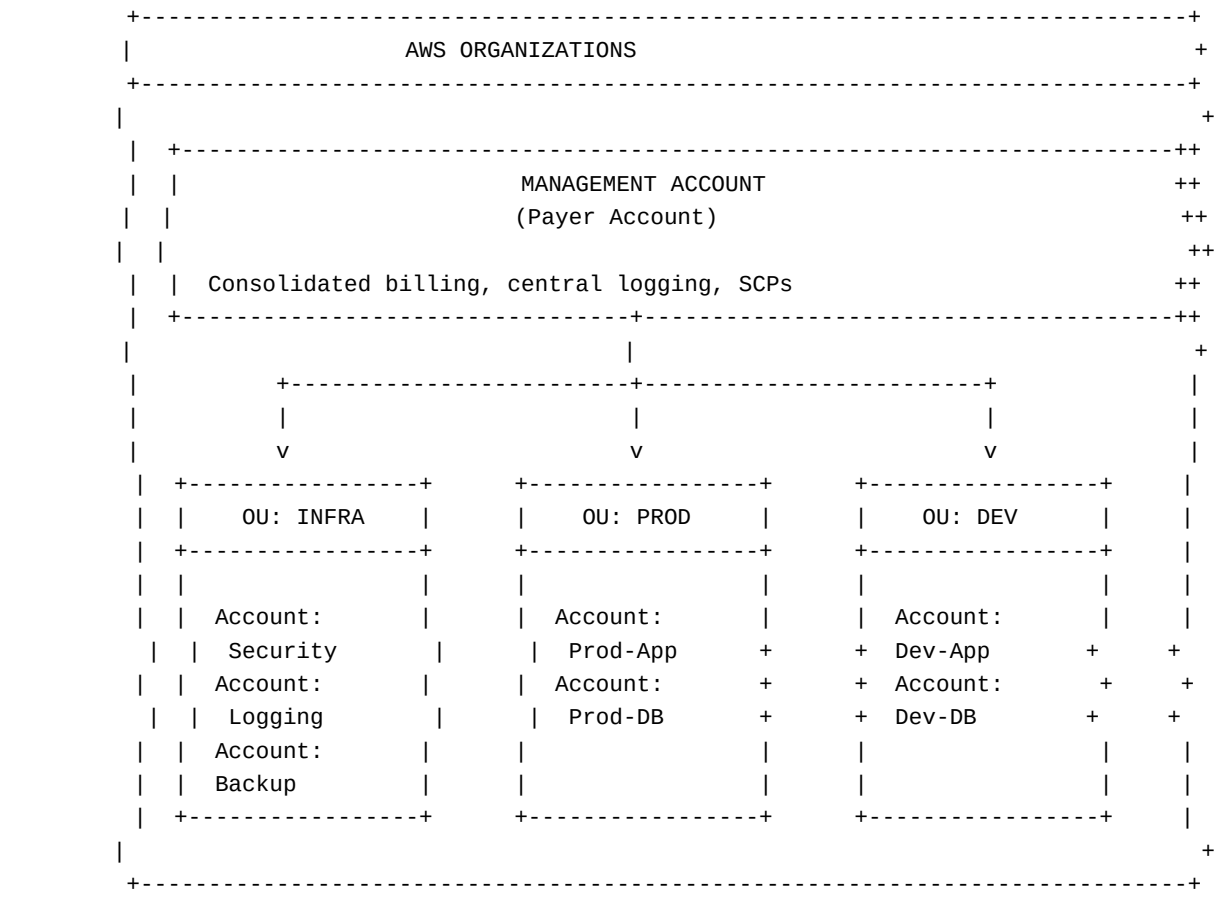
- Separate billing per account
- No central policy enforcement
- Hard to track costs across teams

With Organizations:

- Consolidated billing

- Service control policies (SCPs)
- Centralized logging (CloudTrail)

Diagram: AWS Organizations



Service Control Policy (SCP) Example

```
{
"Version": "2012-10-17",
"Statement": [
{
"Sid": "DenyRegionOutsideUSEast1",
"Effect": "Deny",
"Action": "*",
"Resource": "*",
"Condition": {
"StringNotEquals": {
                                    "aws:RequestedRegion": "us-east-1"
}
}
},
{
"Sid": "DenyLeavingOrganization",
"Effect": "Deny",
"Action": [
"organizations:LeaveOrganization",
"organizations:DeleteOrganization"
],
"Resource": "*"
},
{
"Sid": "RequireEncryptedS3Buckets",
```

```
      "Effect": "Deny",
      "Action": [
        "s3:CreateBucket",
        "s3:PutBucketEncryption"
      ],
      "Resource": "*",
      "Condition": {
        "Null": {
          "s3:x-amz-server-side-encryption": "true"
        }
      }
    }
  ]
}
```

Organizations Terraform

```
# Create organization
resource "aws_organizations_organization" "main" {
  aws_service_access_principals = [
    "cloudtrail.amazonaws.com",
    "config.amazonaws.com"
  ]
  feature_set = "ALL"
}

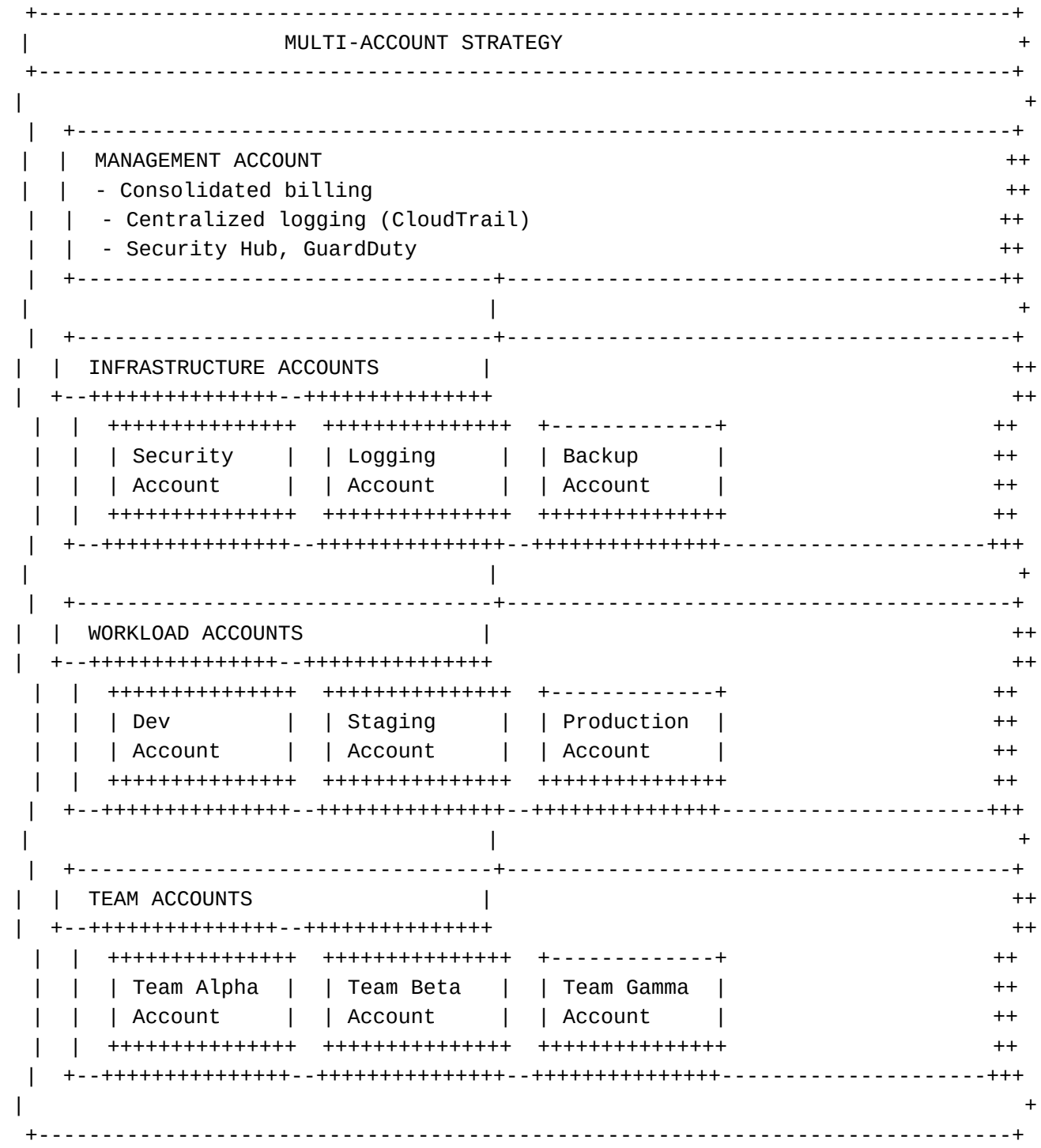
# Create organizational unit
resource "aws_organizations_organizational_unit" "prod" {
  name      = "Production"
  parent_id = aws_organizations_organization.main.roots[0].id
}

# Create member account
resource "aws_organizations_account" "prod_app" {
  name  = "Production-App"
  email = "prod-app@example.com"
  parent_id = aws_organizations_organizational_unit.prod.id
}

# Attach SCP
resource "aws_organizations_policy" "deny_region" {
  name        = "DenyRegionOutsideUSEast1"
  description = "Deny operations outside us-east-1"
  content = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Deny"
      Action = "*"
      Resource = "*"
      Condition = {
        StringNotEquals = {
          "aws:RequestedRegion" = "us-east-1"
        }
      }
    }]
  })
}

resource "aws_organizations_policy_attachment" "prod_ou" {
  policy_id = aws_organizations_policy.deny_region.id
  target_id = aws_organizations_organizational_unit.prod.id
}
```

[Page 78]
Chapter 78: Multi-Account Strategy
What is Multi-Account Strategy?
Organizing multiple AWS accounts to separate environments, teams, and workloads for security, billing, and governance.
Diagram: Multi-Account Strategy

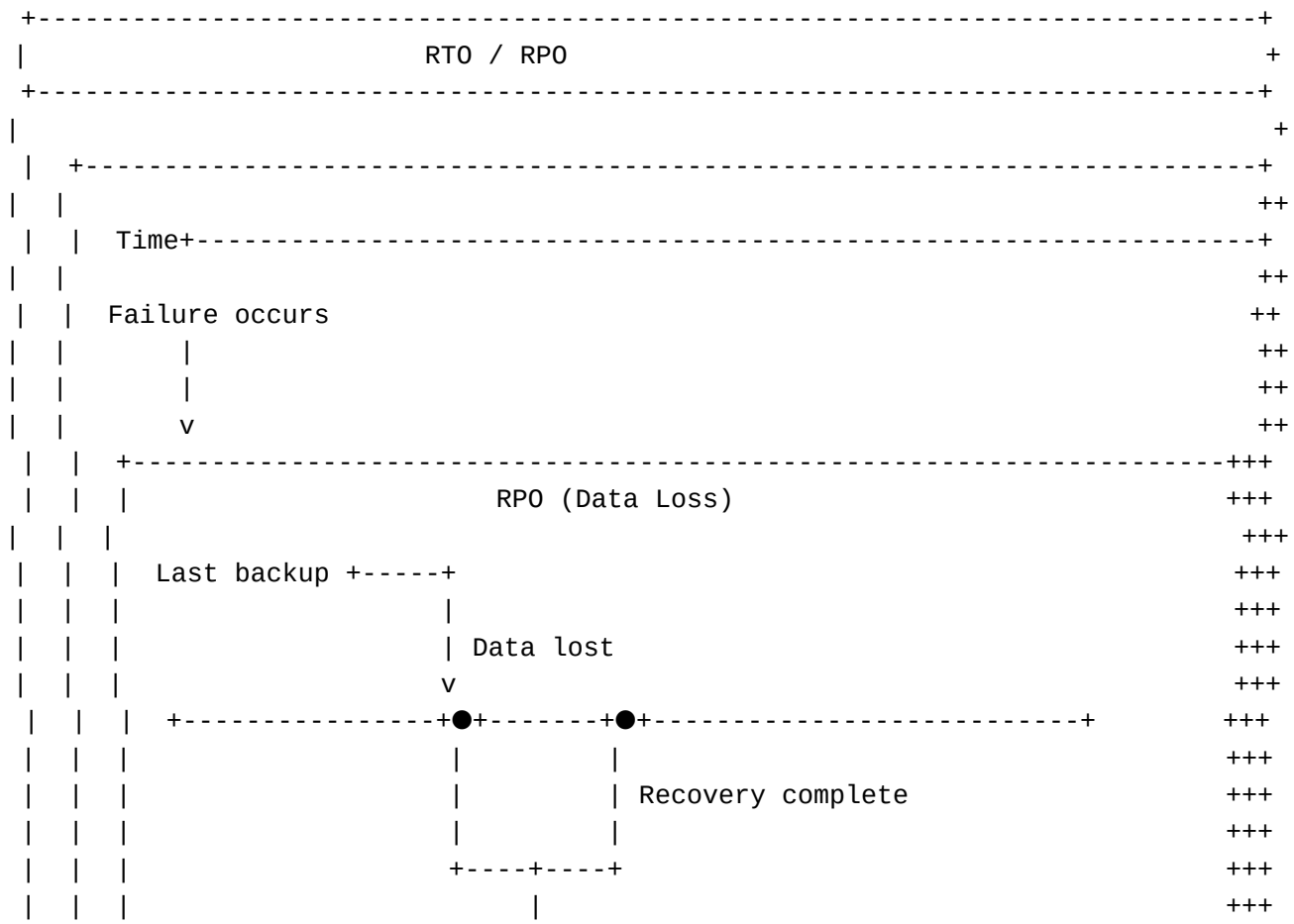


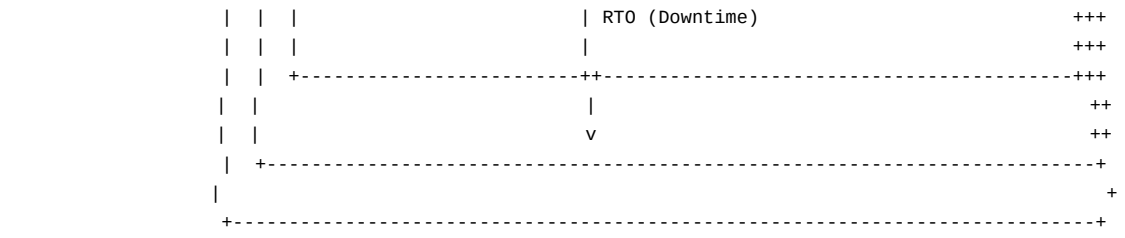
Multi-Account Best Practices

Practice
Why

Separate environments
Dev, staging, prod in different accounts
Centralize logging
Single CloudTrail bucket across accounts
Use IAM roles for cross-account
Assume role from management account
Tag all resources
Cost allocation
Implement SCPs
Guardrails for all accounts
Use AWS Control Tower
Automated multi-account setup
Centralize security
GuardDuty, Security Hub in management account

[Page 79]
Chapter 79: Backup & Disaster Recovery (RTO/RPO)
What are RTO and RPO?
- **RTO (Recovery Time Objective):** Maximum acceptable downtime
- **RPO (Recovery Point Objective):** Maximum acceptable data loss
Diagram: RTO/RPO

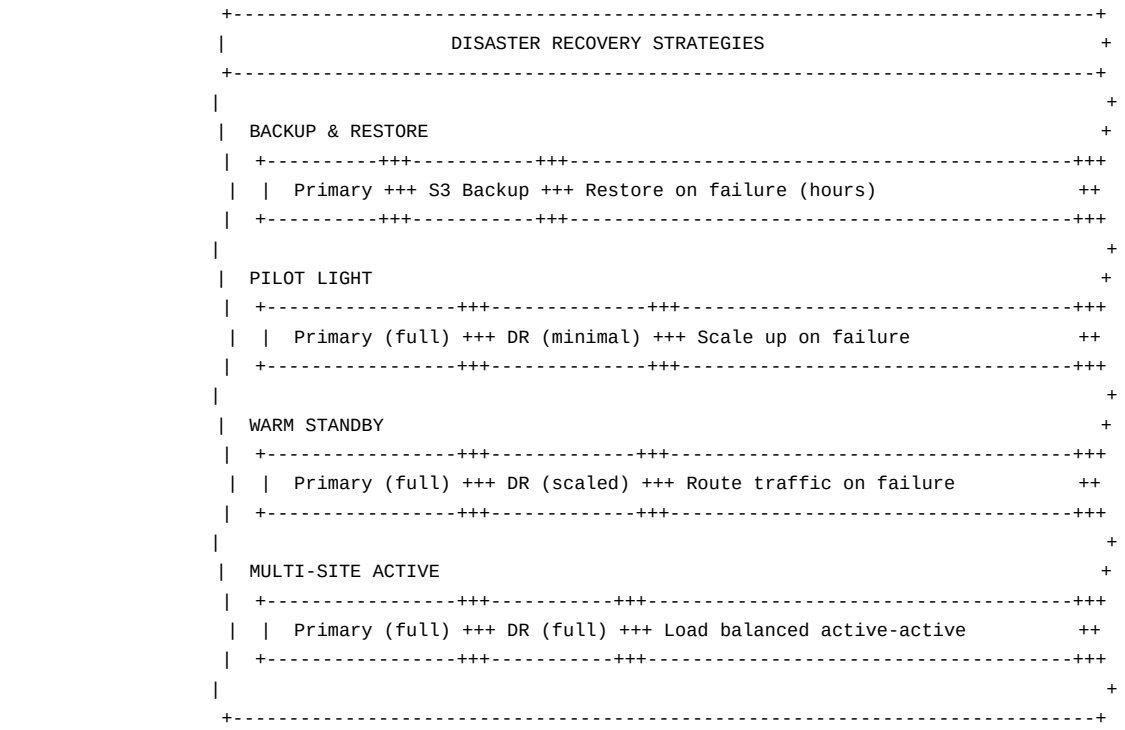




Disaster Recovery Strategies

Strategy	RTO	RPO	Cost	Complexity
Backup & Restore	Hours	Hours	Low	Low
Pilot Light	Minutes	Minutes	Medium	Medium
Warm Standby	Minutes	Seconds	High	Medium
Multi-Site Active	Seconds	Seconds	Very High	High

Diagram: DR Strategies



Backup Implementation (AWS Backup)

```
# AWS Backup plan
resource "aws_backup_plan" "daily_backup" {
name = "daily-backup-plan"
rule {
                                        rule_name         = "daily-backup"
                                    target_vault_name = aws_backup_vault.main.name
```

```
    schedule          = "cron(0 2 * * ? *)"  # 2 AM daily
    start_window      = 60
    completion_window = 120

    lifecycle {
      delete_after = 30
    }

    copy_action {
      destination_vault_arn = aws_backup_vault.dr.arn
    }
  }
}

# Backup selection (what to backup)
resource "aws_backup_selection" "rds" {
  name         = "backup-rds"
  plan_id      = aws_backup_plan.daily_backup.id
  iam_role_arn = aws_iam_role.backup_role.arn

  resources = [
    aws_db_instance.main.arn
  ]

  condition {
    string_equals {
      key   = "aws:ResourceTag/Environment"
      value = "production"
    }
  }
}
```

Cross-Region Replication (S3)

```
resource "aws_s3_bucket_replication_configuration" "main" {
  bucket = aws_s3_bucket.primary.id
  role   = aws_iam_role.replication.arn

  rule {
    id     = "cross-region-replication"
    status = "Enabled"

    destination {
      bucket        = aws_s3_bucket.dr.arn
      storage_class = "STANDARD_IA"
    }

    source_selection_criteria {
      sse_kms_encrypted_objects {
        status = "Enabled"
      }
    }
  }
}
```

RDS Cross-Region Read Replica

```
resource "aws_db_instance" "dr_replica" {
  identifier             = "main-dr-replica"
  replicate_source_db    = aws_db_instance.main.id
  instance_class         = "db.t3.micro"
```

```
  availability_zone      = "us-west-2a"
  backup_retention_period = 7
  skip_final_snapshot    = true

  tags = {
    Environment = "dr"
    Purpose     = "disaster-recovery"
  }
}
```

DR Best Practices

Practice	Why
----------	-----
Define RTO/RPO	Business requirements
Test DR plan regularly	Ensure it works
Automate failover	Reduce manual errors
Document runbooks	Enable on-call engineers
Cross-region backups	Regional failure protection
Use infrastructure as code	Rebuild quickly
Monitor backup success	Detect failures early

[Page 80]

CONCLUSION

Key Takeaways from All Additional Chapters

Compute Scaling

- Auto Scaling Groups provide automatic capacity adjustment

- Target tracking is simplest scaling policy

- Health checks enable self-healing systems

- Multi-AZ design is essential for high availability

Infrastructure as Code

- Terraform enables declarative infrastructure

- Remote state (S3 + DynamoDB) enables team collaboration

- Modules enable reusable infrastructure components

Container Ecosystem

- ECR provides managed container registry
- Tag images with semantic version and commit hash
- Scan images for vulnerabilities before deployment

Deployment Strategies

- Blue-green for zero downtime and instant rollback
- Canary for risk reduction with real traffic
- Rolling for cost-sensitive workloads

Observability

- Metrics for performance, logs for debugging
- Alarms should be actionable
- X-Ray for distributed tracing

Auditing & Security

- CloudTrail for API auditing (who did what)
- AWS Config for compliance and drift detection
- GuardDuty for threat detection
- Security Hub for centralized view

IAM & Security

- Least privilege principle
- Use roles, not users, for services
- Cross-account access via role assumption

Encryption

- At rest (EBS, S3) and in transit (TLS)
- Envelope encryption reduces KMS calls
- Rotate keys regularly

CI/CD Security

- Scan code, dependencies, containers

- Never store secrets in code
- Environment approvals for production

Kubernetes (EKS)
- Node groups for different workloads
- IRSA for pod IAM permissions
- Ingress for external HTTP traffic
- Network policies for zero trust

Networking
- Public subnets for internet-facing resources
- Private subnets for internal resources
- NAT Gateway for private subnet internet access
- VPC endpoints for private AWS service access

Cost Optimization
- Spot instances for fault-tolerant workloads
- Reserved instances for steady-state
- Right-size resources based on metrics

Event-Driven
- SQS for work queues
- SNS for pub/sub
- EventBridge for complex event routing

Multi-Account & DR
- Separate accounts for environments
- Centralized logging and security
- Define RTO/RPO based on business needs

Your Complete AWS Interview Preparation Checklist

- [ ] EC2, S3, Lambda, RDS, DynamoDB

- [ ] VPC, subnets, security groups, NACLs
- [ ] Load balancers (ALB vs NLB)
- [ ] Auto Scaling Groups and scaling policies
- [ ] IAM (users, groups, roles, policies, SCPs)
- [ ] CloudFormation/Terraform (IaC)
- [ ] CI/CD (CodePipeline, CodeBuild, CodeDeploy)
- [ ] Container services (ECS, EKS, ECR)
- [ ] Monitoring (CloudWatch, X-Ray)
- [ ] Logging (CloudTrail, CloudWatch Logs)
- [ ] Security (GuardDuty, Security Hub, Config)
- [ ] Encryption (KMS, envelope encryption)
- [ ] Networking (NAT Gateway, VPC endpoints)
- [ ] Cost optimization (Spot, RI, right-sizing)
- [ ] Event-driven (SQS, SNS, EventBridge)
- [ ] Multi-account (Organizations, Control Tower)
- [ ] Disaster recovery (RTO, RPO, backup strategies)

Final Tips for Your Interview

1. **Draw architectures:** Show you understand how services connect
2. **Know trade-offs:** EC2 vs Lambda, RDS vs DynamoDB, etc.
3. **Practice failure scenarios:** What happens when X fails?
4. **Design for cost:** Mention Spot, RI, right-sizing
5. **Security first:** Always mention least privilege, encryption
6. **Real-world examples:** Share your experience

Good luck with your interview!

About These Additional Chapters

These chapters complete the AWS curriculum, covering compute scaling, IaC, containers, deployment strategies, observability, auditing, security, encryption, CI/CD security,

Kubernetes, advanced networking, cost optimization, event-driven architecture, and multi-account disaster recovery. Use this guide alongside the main AWS book for comprehensive interview preparation.

[End of Additional Chapters]